



UNIVERSIDAD DE TALCA
FACULTAD DE INGENIERÍA
ESCUELA DE INGENIERÍA CIVIL EN COMPUTACIÓN

**Sistema IoT de asistencia con autenticación
biométrica e integración empresarial**

AGUSTIN EDUARDO MEZA MELLA

Profesor Guía: RICARDO PÉREZ GUZMÁN

Memoria para optar al título de
Ingeniero Civil en Computación

Curicó – Chile

Junio, 2026



UNIVERSIDAD DE TALCA
FACULTAD DE INGENIERÍA
ESCUELA DE INGENIERÍA CIVIL EN COMPUTACIÓN

**Sistema IoT de asistencia con autenticación
biométrica e integración empresarial**

AGUSTIN EDUARDO MEZA MELLA

Profesor Guía: RICARDO PÉREZ GUZMÁN

El presente documento fue calificado con nota: _____

Memoria para optar al título de
Ingeniero Civil en Computación

Curicó – Chile

Junio, 2026

Dedicado a mi familia.

Agradecimientos

En primer lugar, quiero agradecer a mis padres por haber sido quienes me han apoyado desde el principio de mi carrera. A mi padre, por ayudarme en todo lo que necesité para poder llegar a este punto, y a mi madre, por estar en los momentos más difíciles entregándome su amor y cariño. Agradezco a mi hermano Maximiliano por estar siempre ahí para resolver mis dudas y ayudarme a visualizar el mañana sin perderme en el camino.

Quiero dar las gracias a mi amigo Gonzalo por guiarme en el camino de la vida, enseñarme a mejorar cada día y ser un ejemplo de superación, perseverancia y calidad humana. Agradezco también a mis amigos de la universidad, con quienes compartí innumerables momentos de estudio, trabajo y diversión, que hicieron de esta una grata experiencia universitaria.

Quiero agradecer de manera especial a Adrián Espinoza por su buena voluntad de transportarme en los días en que no tenía cómo llegar a la universidad sin nunca pedir algo a cambio, y por todos los momentos que vivimos juntos: fiestas, trabajos, entre tantas otras cosas. Gracias por todos esos recuerdos que nos llevamos. Junto a él, agradezco a Nelson Silva y Benjamín Puebla, de quienes me llevo grandes recuerdos y experiencias que me ayudaron a crecer como persona y profesional.

Por último, quiero agradecer a mi primer compañero de universidad, quien siempre estuvo ahí para apoyarme en todo lo que necesitaba y fue quien me ayudó a ingresar al mundo universitario. Gracias, Luis Bernal, por ser un gran amigo y compañero que, a pesar de no poder acompañarme durante toda la carrera, siempre estuvo presente.

TABLA DE CONTENIDOS

Agradecimientos	2
Tabla de Contenidos	3
Índice de Figuras	6
Índice de Tablas	8
Resumen	9
1 Introducción	11
1.1 Contexto	11
1.2 Descripción del problema	12
1.3 Propuesta de solución	13
1.4 Objetivos	13
1.4.1 Objetivo general	14
1.4.2 Objetivos específicos	14
1.5 Alcances	15
1.6 Resumen capítulo	16
2 Marco teórico	17
2.1 Conceptos básicos	17
2.1.1 Conceptos globales	17
2.1.2 Conceptos de hardware y software	18
2.2 Tecnologías utilizadas	20
2.2.1 Microcontrolador ESP32-CAM	20
2.2.2 Lector de huella digital AS608	20
2.2.3 Protocolos de comunicación HTTP	21
2.2.4 Protocolo de comunicación MQTT	21
2.2.5 Contenedores Docker	21
2.2.6 Sistemas de bases de datos	21
2.2.7 Sensor PIR (HC-SR501)	22
2.2.8 Framework DeepFace	22
2.2.9 Broker Mosquitto en contenedor Docker	23
2.3 Estado del arte	24
2.3.1 Soluciones basadas en hardware	24
2.3.2 Soluciones basadas en software	24
2.4 Metodologías	25

2.4.1	Metodología de desarrollo	25
2.4.2	Metodología de evaluación	26
2.5	Resumen del capítulo	28
3	Metodologías	30
3.1	Metodología de desarrollo	30
3.1.1	Prototipado Evolutivo Incremental	30
3.2	Plan de trabajo	32
3.3	Metodología de evaluación	33
3.3.1	Pruebas de integración	33
3.3.2	Pruebas de usabilidad	34
3.3.3	Pruebas de confiabilidad y disponibilidad	35
3.3.4	Pruebas de caja negra	36
3.3.5	Pruebas unitarias automatizadas	36
3.3.6	Resultados esperados de las pruebas	37
3.4	Resultados Esperados de los Objetivos Específicos	40
3.5	Resumen del capítulo	42
4	Desarrollo e implementación	43
4.1	Diseño de software	43
4.1.1	Arquitectura física	43
4.1.2	Arquitectura lógica	44
4.2	Implementación	46
4.2.1	Iteración 1: Integración de hardware y servidor embebido	46
4.2.2	Iteración 2: Almacenamiento local y modo offline	62
4.2.3	Iteración 3: Backend, base de datos y comunicación	68
4.2.4	Iteración 4: Reconocimiento facial, anti-spoofing y cifrado biométrico	81
4.2.5	Iteración 5: Autenticación, multi-tenant y enrolamiento de dispositivos	88
4.2.6	Iteración 6: Módulo antifraude y validación previa a la captura	95
4.2.7	Iteración 7: Panel web backend e integración con ERP	101
4.2.8	Iteración 8: Sincronización diferida, logs y cierre del proyecto . .	111
4.3	Mecanismos de seguridad	121
4.3.1	Autenticación y control de acceso	121
4.3.2	Enrolamiento seguro de dispositivos	122
4.3.3	Protección de datos biométricos	123
4.4	Arquitectura de tiempo real (SSE)	123
4.4.1	Endpoints de streaming	123

4.4.2	Tres capas de detección de desconexión	124
4.4.3	Consumo en el frontend	125
4.5	Resumen del capítulo	125
5	Análisis de resultados	126
5.1	Costo total del prototipo	126
5.2	Resultados de encuestas SUS	126
5.2.1	Usuarios no técnicos	127
5.2.2	Usuarios técnicos	127
5.2.3	Análisis comparativo	128
5.3	Métricas de reconocimiento facial	128
5.3.1	Tiempo de respuesta del ESP32-CAM	129
5.3.2	Precisión del reconocimiento	129
5.3.3	Comparación de detectores faciales: MTCNN vs. RetinaFace . .	129
5.3.4	Efecto de la caché de embeddings	130
5.4	Resultados de pruebas de estrés offline	130
5.4.1	Escenario 1: Desconexión total	130
5.4.2	Escenario 2: Conectividad intermitente	131
5.4.3	Escenario 3: Reconexión prolongada	131
5.4.4	Escenario 4: Corte de energía durante la sincronización	131
5.4.5	Escenario 5: Backend inaccesible con Wi-Fi activo	131
5.5	Análisis de accesibilidad	132
5.6	Análisis de sostenibilidad	133
5.7	Resumen del capítulo	134
6	Conclusiones	135
6.1	Recapitulación del trabajo desarrollado	135
6.2	Cumplimiento de los objetivos planteados	135
6.3	Discusión y limitaciones	137
6.4	Proyecciones futuras	137
	Bibliografía	139
	Anexo A: Preguntas complementarias de usabilidad	145
	Anexo B: Tabla de endpoints REST y tópicos MQTT	148
	Anexo C: Diagramas ampliados	153
	Anexo D: Detalle de casos de prueba y resultados de la suite automatizada	158

**Anexo E: Respuestas individuales de las encuestas SUS y formulario
aplicado**

161

ÍNDICE DE FIGURAS

3.1	Metodología prototipado evolutivo con enfoque incremental	31
4.1	Arquitectura física.	44
4.2	Arquitectura lógica.	45
4.3	Componentes del ESP32-CAM: hardware y firmware.	46
4.4	Mockup menú principal en el ESP32-CAM.	48
4.5	Mockup panel de configuración Wi-Fi.	48
4.6	Mockup registro de usuario en el ESP32-CAM.	49
4.7	Mockup de usuarios almacenados en el ESP32-CAM.	49
4.8	Mockup de visualización de turnos en el ESP32-CAM.	50
4.9	Mockup de creación de turnos en el ESP32-CAM.	50
4.10	Mockup de asignación de turnos en el ESP32-CAM.	51
4.11	Conexiones entre el ESP32-CAM y el lector de huellas AS608.	52
4.12	Menú principal del sistema web embebido.	55
4.13	Interfaz de configuración Wi-Fi.	56
4.14	Vista para el registro de personas en el dispositivo.	57
4.15	Vista de asistencias locales almacenadas en el dispositivo.	58
4.16	Vista de personas registradas en el dispositivo.	59
4.17	Vista de gestión y creación de turnos en el dispositivo.	60
4.18	Vista de gestión de asignaciones en el dispositivo.	61
4.19	Diagrama entidad-relación de la base de datos (14 tablas).	72
4.20	Flujo MQTT pub/sub: tópicos, direcciones y QoS entre el ESP32-CAM, el broker Mosquitto y el backend.	73
4.21	Máquina de estados del registro biométrico en el ESP32-CAM.	75
4.22	Arquitectura de despliegue Docker: servicios, puertos, volúmenes y dependencias en la red.	78
4.23	Pipeline de cifrado biométrico	85
4.24	Diagrama de secuencia del login multi-empresa con JWT.	91
4.25	Diagrama de secuencia de la integración ERP: push asíncrono con field mapping.	104
4.26	Diagrama de actividad: sincronización bidireccional entre los modos online y offline del ESP32-CAM.	113
4.27	Flujo operacional consolidado para el registro de asistencia.	118
6.3	Enrolamiento de dispositivo: vinculación del ESP32-CAM mediante PIN de un solo uso y configuración Wi-Fi.	153
6.1	Diagrama de secuencia del enrolamiento biométrico completo.	154

6.2	Marcaje automático: centinela facial con identificación 1:N y push asíncrono a ERP.	155
6.4	Registro de personas: alta de datos, consentimiento biométrico y captura de huella y rostro.	156
6.5	Gestión de turnos: creación de turnos y asignación a personas registradas.	156
6.6	Sincronización diferida: vaciado de la cola local de asistencias pendientes tras reconexión.	157
6.7	Panel web administrativo: monitoreo en tiempo real, gestión de usuarios y notificación a ERP.	157

ÍNDICE DE TABLAS

3.1	Plan de trabajo del proyecto: 8 iteraciones de 3 semanas cada una (24 semanas total). El “Peso” refleja la complejidad relativa estimada de cada iteración y el “Avance” es la suma acumulada de los pesos.	32
3.2	Cuestionario SUS estándar (Brooke, 1996), traducción al español. . . .	35
3.3	Resultados esperados de las pruebas de integración.	38
3.4	Resultados esperados de las pruebas de usabilidad (SUS).	38
3.5	Resultados esperados de las pruebas de confiabilidad y disponibilidad. .	39
3.6	Resultados esperados de las pruebas de caja negra.	40
5.1	Desglose de costos del prototipo.	126
5.2	Resultados de encuesta SUS, usuarios no técnicos (n=6).	127
5.3	Resultados de encuesta SUS, usuarios técnicos (n=3).	128
5.4	Tiempos de respuesta del ciclo de marcación facial.	129
5.5	Métricas de precisión del reconocimiento facial.	129
5.6	Comparación empírica de detectores faciales sobre el mismo conjunto de imágenes de prueba.	130
5.7	Efecto de la caché de embeddings en memoria sobre el tiempo de identificación.	130
5.8	Resultados de pruebas de estrés offline ante variaciones de conectividad.	132
5.9	Resultados de pruebas de estrés offline ante fallas de sincronización y de backend	132
6.1	Endpoints de la API REST del backend.	151
6.2	Tópicos MQTT utilizados en la comunicación ESP32-CAM ↔ backend.	152
6.3	Casos de prueba de la suite automatizada por archivo y módulo cubierto.	159
6.4	Resultados de la ejecución completa de la suite ampliada, con PostgreSQL (Docker) activo.	160
6.5	Cobertura de código por módulo del backend tras la ampliación de la suite (solo código de producción).	160
6.6	Respuestas individuales SUS, usuarios no técnicos (n=6).	161
6.7	Respuestas individuales SUS, usuarios técnicos (n=3).	162

RESUMEN

El presente proyecto aborda el desarrollo de un sistema IoT de control de asistencia biométrico, diseñado para responder a las limitaciones actuales de las soluciones comerciales y cumplir con los requisitos técnicos y legales establecidos por la Dirección del Trabajo de Chile, en particular la Resolución Exenta N.º38 (2025), que regula los sistemas electrónicos de registro de asistencia. El trabajo propone una alternativa modular, de bajo costo, código abierto y adaptable, orientada principalmente a pequeñas y medianas empresas que requieren un sistema flexible, con capacidad de funcionamiento autónomo y con posibilidades reales de integración con plataformas ERP.

La solución desarrollada utiliza como dispositivo central un ESP32-CAM, microcontrolador que incorpora cámara, conectividad Wi-Fi y capacidades de procesamiento suficientes para realizar captura de imágenes y ejecutar tareas de apoyo asociadas a la autenticación. A este módulo se integra un lector de huellas AS608, que permite una verificación biométrica complementaria frente a situaciones donde la captura facial no sea óptima. El sistema se estructura de manera híbrida, permitiendo registrar asistencia tanto mediante reconocimiento facial como mediante verificación dactilar, de acuerdo con las condiciones de operación.

Uno de los principales aportes del proyecto es la capacidad del dispositivo para operar en modo offline, utilizando SPIFFS como sistema de almacenamiento local para gestionar usuarios, turnos, asignaciones y asistencias en formato JSON. Cuando la conectividad se restablece, el sistema ejecuta un proceso de sincronización diferida que garantiza la integridad de los datos y la continuidad operativa del dispositivo, incluso en entornos con conectividad inestable o intermitente, situación común en empresas pequeñas o en faenas remotas. Asimismo, la arquitectura lógica y física diseñada permite incorporar módulos de seguridad adicionales, como un sensor PIR combinado con anti-spoofing por software, destinado a detectar intentos de suplantación mediante fotografías o pantallas.

El desarrollo del proyecto se llevó a cabo utilizando una metodología de prototipado evolutivo incremental, seleccionada por su capacidad para integrar hardware, firmware, backend y módulos biométricos de manera progresiva. A lo largo de ocho iteraciones se construyeron y validaron componentes funcionales independientes que luego fueron integrados: capturas biométricas, almacenamiento local, comunicación HTTP/MQTT, panel web embebido, mecanismos antifraude, backend con base de datos PostgreSQL y módulo de comunicación estándar para ERP. Cada iteración incluyó análisis, diseño, implementación y pruebas, permitiendo detectar fallas tempranas y optimizar el prototipo de manera continua.

La etapa de evaluación incorporó pruebas de integración, usabilidad, confiabilidad y

disponibilidad, fundamentales para determinar el desempeño del sistema en escenarios reales. Las pruebas de integración verificaron la correcta comunicación entre los distintos módulos del prototipo, incluyendo la interoperabilidad entre el ESP32-CAM, el lector AS608, el almacenamiento SPIFFS, el backend y la base de datos. Las pruebas de usabilidad, aplicadas tanto a usuarios técnicos como no técnicos mediante el método System Usability Scale (SUS), permitieron evaluar la claridad de las interfaces y la facilidad de uso del sistema. Complementariamente, se desarrollaron pruebas de confiabilidad y disponibilidad destinadas a medir la robustez del sistema frente a desconexiones, reconexiones y funcionamiento prolongado sin supervisión.

El sistema resultante es un prototipo funcional que permite registrar asistencia, almacenar datos localmente, sincronizar registros con un backend cuando existe conectividad, operar de manera continua bajo distintos escenarios y comunicar información hacia sistemas ERP mediante una API estándar. A través del análisis de los resultados obtenidos, se concluye que la solución propuesta es técnicamente viable, cumple con los requisitos de funcionamiento planteados en los objetivos específicos y representa una alternativa accesible y adaptable frente a las restricciones de los dispositivos biométricos comerciales actuales.

1. Introducción

En el siguiente capítulo se presenta el contexto, la problemática actual y la propuesta de solución que fundamentan el desarrollo de este sistema IoT de control de asistencia biométrico.

1.1 Contexto

El control de asistencia laboral es un requisito obligatorio para todas las empresas en Chile, conforme a lo dispuesto por la Dirección del Trabajo. Este registro puede realizarse de manera manual o mediante sistemas digitales que cumplan con la Resolución Exenta N°38 (2025) [40], la cual establece los requisitos técnicos y legales que deben cumplir los sistemas electrónicos para ser considerados válidos ante la autoridad.

El cumplimiento de estas normas busca garantizar la trazabilidad y transparencia de las jornadas laborales, e impacta directamente en el cálculo de remuneraciones y en la gestión administrativa de la empresa cuando el registro no incluye los métodos que garanticen la asistencia del trabajador exigidos por la norma [14]. Por ello, muchas organizaciones optan por integrar sus sistemas de asistencia con plataformas ERP (EntERPrise Resource Planning), las cuales centralizan distintos procesos como recursos humanos, inventario, finanzas y contabilidad dentro de un mismo entorno [58].

Esta integración permite que los datos de asistencia alimenten automáticamente los módulos de remuneraciones o de gestión de personal, evitando errores y reduciendo los tiempos administrativos [7]. En este contexto, los sistemas de asistencia pueden ser parte del propio ERP o soluciones externas especializadas, las cuales se comunican mediante APIs o servicios web.

Las máquinas de control de asistencia digital, por su parte, permiten registrar las marcaciones de entrada y salida de los trabajadores de forma autónoma [22]. Sin embargo, muchas de las soluciones disponibles en el mercado presentan limitaciones: operan únicamente en red local, exportan datos en formatos cerrados (como CSV o Excel) y no siempre permiten integrarse de manera flexible con otros sistemas, como los ERP corporativos.

Este proyecto propone el desarrollo de un sistema IoT de control de asistencia basado en ESP32-CAM, capaz de realizar reconocimiento facial y enviar los registros al backend, permitiendo su posterior integración con un ERP o funcionamiento independiente en modo local.

1.2 Descripción del problema

Aunque la normativa chilena exige que los sistemas de control de asistencia garanticen trazabilidad, integridad y disponibilidad de los registros de acuerdo con la Resolución Exenta N.º 38 (2025), una parte importante de los dispositivos utilizados actualmente especialmente los relojes de control biométricos de bajo costo presentan limitaciones que dificultan su adopción en empresas pequeñas y medianas. Muchos de estos equipos requieren conexión permanente a internet, operan con software propietario o solo permiten exportar datos en formatos estáticos como CSV, lo que genera dependencia de procesos manuales y restringe la interoperabilidad con otros sistemas.

Al mismo tiempo, la mayoría de los ERP utilizados por las empresas no incorpora módulos nativos para capturar asistencia desde hardware, por lo que su integración con dispositivos externos resulta costosa o directamente incompatible. La ausencia de APIs abiertas, la falta de mecanismos de sincronización automática y la poca flexibilidad para adaptar el hardware al contexto operativo dificultan la incorporación de tecnología biométrica en entornos reales.

Estas limitaciones se vuelven más evidentes en escenarios donde la conectividad es inestable. Muchos relojes de control no pueden almacenar marcaciones localmente de manera segura ni realizar una sincronización diferida una vez restablecida la red, lo que expone a las empresas a la pérdida de datos y al incumplimiento normativo. Asimismo, la mayoría de las soluciones comerciales no permite personalizar la lógica de validación biométrica, integrar módulos anti fraude ni extender el sistema a necesidades particulares, como gestión de turnos o funcionamiento híbrido.

Actualmente no existe en el mercado una alternativa accesible, modular y adaptable que permita capturar asistencia mediante biometría (huella o reconocimiento facial), operar de forma autónoma sin internet, almacenar registros en el dispositivo, detectar intentos de suplantación y sincronizar datos con sistemas ERP mediante APIs estándar. Esta brecha tecnológica afecta especialmente a organizaciones que requieren una solución flexible, de bajo costo y consistente con las exigencias de la autoridad laboral.

En síntesis, el problema consiste en la falta de un reloj control biométrico capaz de operar offline, almacenar y sincronizar marcaciones de manera segura, y conectarse a sistemas ERP mediante interfaces estándar cumpliendo la normativa vigente.

1.3 Propuesta de solución

La propuesta consiste en diseñar e implementar un prototipo de control de asistencia utilizando un módulo ESP32-CAM junto con un lector de huellas dactilares. El ESP32-CAM es una placa de bajo costo que incorpora una cámara, conectividad Wi-Fi y Bluetooth, lo que permite capturar imágenes, procesarlas y comunicarse con otros sistemas sin necesidad de equipos adicionales.

El objetivo es que el dispositivo pueda registrar la asistencia de los trabajadores mediante reconocimiento facial y huella digital, funcionando tanto con conexión a internet como sin ella. En modo offline, el equipo almacenará temporalmente los registros en su propia memoria interna, lo que le permitirá operar de manera autónoma durante varios días, dependiendo de la cantidad de marcaciones diarias. Cuando se recupere la conexión, los datos se sincronizarán automáticamente con el servidor central, el cual se encarga de reenviarlos hacia el sistema ERP de la empresa cuando corresponda, según la configuración establecida por el usuario.

El lector de huellas permitirá complementar la verificación facial, ofreciendo una alternativa en lugares donde las condiciones de luz o el ángulo de la cámara dificulten el reconocimiento.

El prototipo contará además con una interfaz web integrada, accesible desde la misma red local, que permitirá configurar parámetros de red, administrar usuarios y definir el destino de los datos. Junto con ello, se desarrollará una documentación técnica de apoyo, donde se describirá el funcionamiento general del sistema, el formato de los registros generados y las opciones de integración con diferentes plataformas ERP. Esta integración se realizará mediante un endpoint o API estándar, lo que permitirá la conexión con cualquier sistema ERP que acepte intercambio de datos a través de solicitudes HTTP o JSON, brindando flexibilidad sin depender de un proveedor específico.

Finalmente, para mejorar la seguridad en el proceso de marcación, se incorporarán métodos que eviten la suplantación de identidad, impidiendo que se utilicen fotografías o imágenes impresas para registrar asistencias falsas.

1.4 Objetivos

En el siguiente apartado se procederá a presentar el objetivo general del proyecto, junto con los objetivos específicos que guiarán el desarrollo y evaluación del prototipo.

1.4.1 Objetivo general

Desarrollar un prototipo funcional de sistema IoT de código abierto para el control de asistencia laboral en pequeñas y medianas empresas, basado en el microcontrolador ESP32-CAM, que registre marcaciones biométricas sin conexión a internet y las sincronice de forma automática con plataformas empresariales una vez recuperada la conexión. El cumplimiento de este objetivo se medirá con los criterios de aceptación definidos en los objetivos específicos.

1.4.2 Objetivos específicos

A partir del objetivo general, se establecen los siguientes objetivos específicos, los cuales permiten abordar de manera concreta y medible cada una de las dimensiones técnicas involucradas en el desarrollo del prototipo.

Objetivo específico 1

Diseñar la arquitectura física y lógica del prototipo IoT, seleccionando componentes que garanticen el funcionamiento autónomo y demostrando su viabilidad económica mediante un presupuesto de hardware total que no supere los \$45.000 CLP.

Objetivo específico 2

Implementar el módulo de autenticación biométrica híbrida, logrando un tiempo de procesamiento inferior a 5 segundos por marcación facial (incluyendo transmisión y respuesta del servidor) y garantizando una precisión superior al 90% en la identificación correcta durante las pruebas de laboratorio.

Objetivo específico 3

Integrar el sistema de archivos local (LittleFS) para la operación offline, asegurando una capacidad de almacenamiento segura de al menos 1.000 registros de asistencia en formato JSON, sin pérdida de datos ante simulaciones de corte de energía o desconexión de red.

Objetivo específico 4

Construir una interfaz web embebida y un panel de administración para la configuración del sistema, validando su facilidad de uso mediante la obtención de un puntaje igual o superior a 68 puntos en la evaluación de usabilidad SUS (System Usability Scale) aplicada a un grupo de usuarios de prueba.

Objetivo específico 5

Desarrollar una API RESTful que permita la integración automatizada con sistemas ERP externos, verificando que el sistema logre sincronizar y transmitir el 100% de los paquetes de datos generados en modo offline una vez restablecida la conectividad.

Objetivo específico 6

Evaluar la robustez y disponibilidad del prototipo integrado en escenarios de conectividad intermitente, comprobando la correcta ejecución de los mecanismos de reintento automático y analizando técnicamente las limitaciones del dispositivo para su futura escalabilidad comercial.

1.5 Alcances

A continuación se presentarán los alcances propuestos para este proyecto.

- Se diseñará la arquitectura de hardware y software del sistema, incluyendo los módulos de captura de imagen, registro de huella, almacenamiento local y comunicación con el servidor.
- Se implementará un prototipo funcional capaz de registrar asistencias mediante reconocimiento facial y huella digital, operando en modo offline con almacenamiento local y sincronización posterior.
- Se desarrollará una interfaz web embebida para la configuración del dispositivo, administración de usuarios y definición del destino de los datos.
- Se integrará una API RESTful para permitir la comunicación entre el dispositivo y un servidor o sistema ERP, utilizando formatos de intercambio JSON sobre HTTP.
- Se incorporarán medidas básicas de seguridad biométrica orientadas a evitar la suplantación de identidad mediante imágenes o fotografías.
- Se realizarán pruebas de funcionamiento offline, midiendo la autonomía del sistema (tiempo máximo de operación sin conexión y capacidad de almacenamiento).
- Se ejecutarán pruebas de integración entre los módulos de hardware, software y el ERP simulado, verificando el flujo completo de datos desde el registro hasta la sincronización.

- Se aplicarán pruebas de usabilidad para evaluar la facilidad de configuración y el nivel de satisfacción del usuario final con la interfaz embebida.
- Se efectuarán pruebas de confiabilidad e integridad, comprobando que los datos registrados sin conexión se transmitan correctamente una vez restablecida la conectividad.
- Los resultados se analizarán y documentarán para determinar la viabilidad técnica y operativa del sistema propuesto.
- El prototipo hará uso de la conexión a internet mediante Wi-Fi para realizar las sincronizaciones con los datos existentes en el servidor como así las funcionalidades de reconocimiento facial avanzado.

1.6 Resumen capítulo

El capítulo 1 presenta el contexto normativo del control de asistencia en Chile y expone las limitaciones de los sistemas biométricos actuales, especialmente su dependencia de internet y su baja interoperabilidad con ERP. Ante esta problemática, se propone un sistema IoT basado en ESP32-CAM con autenticación por rostro y huella, capaz de operar offline y sincronizar datos posteriormente. Además, se establecen los objetivos, alcances y lineamientos generales del proyecto.

2. Marco teórico

A continuación se presentan los fundamentos teóricos y tecnológico que facilitan la comprensión del proyecto

2.1 Conceptos básicos

En esta sección se definen los términos fundamentales que sustentan el desarrollo del sistema, abarcando tanto el contexto normativo y general del problema, como las herramientas específicas utilizadas para su resolución.

2.1.1 Conceptos globales

A continuación se detallan los conceptos generales vinculados a la gestión corporativa, los estándares biométricos y los paradigmas tecnológicos que enmarcan la propuesta.

- Control de asistencia: Proceso por el cual una empresa u organización registra las horas de entrada y salida de sus trabajadores.
- Sistema ERP: Enterprise Resource Planning, sistema de gestión empresarial que permite procesos de una empresa u organización, tales como recursos humanos, contabilidad, finanzas, inventario, asistencia.
- Internet de las cosas (IoT): Paradigma tecnológico que permite la conexión entre dispositivos físicos a través de redes de comunicación.
- Resolución exenta N°38: Norma promulgada por la Dirección del Trabajo (DT) en 2024 (actualizada en 2025), que establece los requisitos obligatorios para los sistemas electrónicos de registro y control de asistencia.
- Biometría: La biometría corresponde al conjunto de técnicas utilizadas para identificar personas mediante características físicas o conductuales únicas, tales como el rostro o la huella digital. Este tipo de autenticación es ampliamente

utilizada en sistemas de control de asistencia por su alta precisión y dificultad de suplantación [31].

- Sincronización offline-online: Proceso mediante el cual un dispositivo opera sin conexión utilizando almacenamiento local y, una vez restablecida la conectividad, transmite los datos acumulados hacia un servidor o ERP. Es necesario en entornos con conectividad inestable.

2.1.2 Conceptos de hardware y software

En este apartado se describen los componentes físicos, protocolos de red, arquitecturas y herramientas de programación que conforman la infraestructura técnica del proyecto.

- ESP32-CAM: Microcontrolador basado en el esp32, con la inclusión de una cámara OV2640, conectividad WiFi y Bluetooth, además de contar con la capacidad de procesamiento local. [45]
- API (Aplicacion Programming Interface): Conjunto de métodos o puntos de conexión que permiten la conexión entre diferentes aplicaciones mediante protocolos de redes. [4]
- Backend: Parte del sistema encargada de la lógica del negocio, manejo de datos y comunicación con bases de datos. [32]
- Frontend: Conjunto de interfaces y funcionalidades con las que interactúa el usuario final. [30]
- Mqtt: Protocolo de comunicación ligero basado en el modelo publicador/suscriptor, diseñado para intercambiar mensajes de manera eficiente, utilizado entre dispositivos que cuenten con recursos limitados. [3]
- Javascript Object Notation (JSON): Formato de datos que se suele utilizar en entornos de desarrollo web para el intercambio de datos de manera legible tanto por las personas como las maquinas. [20]
- SPIFFS (SPI Flash File System): Sistema de archivos diseñado para microcontroladores que permite almacenar información en memoria flash mediante archivos. En este proyecto se utiliza para guardar usuarios, turnos, asignaciones y asistencias en formato JSON, permitiendo el funcionamiento offline [21].
- API RESTful: Estilo de arquitectura de servicios web basado en el protocolo HTTP, que permite la comunicación entre aplicaciones usando operaciones estándar como GET, POST, PUT o DELETE. Es utilizado para integrar el dispositivo IoT con sistemas ERP externos mediante intercambio de datos en formato JSON [2].

- Microservicios: Arquitectura de software basada en dividir un sistema en servicios independientes que se comunican entre sí. Esta estructura permite escalabilidad y facilita la separación entre el procesamiento de imágenes, backend y comunicación con ERP [57].
- Broker MQTT: Servidor encargado de recibir, gestionar y distribuir mensajes enviados por dispositivos IoT utilizando el protocolo MQTT. Actúa como intermediario entre el publicador (el ESP32-CAM) y los suscriptores (backend u otros servicios) [19].
- Sensor PIR (Passive Infrared Sensor): Sensor que detecta cambios en la radiación infrarroja del entorno para identificar la presencia de personas por su calor corporal. En este proyecto actúa como “portero” del reconocimiento facial: la cámara solo se usa para buscar un rostro cuando el PIR detecta movimiento frente al dispositivo, evitando que ese procesamiento, más costoso, se ejecute todo el tiempo sin necesidad. La lectura de huella, en cambio, se revisa de forma continua e independiente del PIR. A diferencia de los sensores IR activos, el PIR no emite luz infrarroja sino que detecta pasivamente la que irradian los cuerpos [34].
- Anti-spoofing biométrico: Técnica de seguridad que permite detectar intentos de suplantación de identidad mediante fotografías, pantallas o máscaras frente a un sistema de reconocimiento biométrico. DeepFace, la librería utilizada en este proyecto, incorpora un detector de suplantación basado en redes neuronales convolucionales que analiza micro-texturas, reflectancia de la piel y profundidad en la imagen para distinguir entre un rostro real y una superficie plana [61].
- JWT (JSON Web Token): Estándar abierto (RFC 7519) que define un formato compacto para transmitir información de autenticación y autorización entre partes como un objeto JSON firmado digitalmente. En este proyecto se utiliza para proteger los endpoints del backend, transportando el identificador del usuario, su empresa y su rol, con firma HMAC-SHA256 y expiración de 24 horas, mediante la librería PyJWT [54].
- Arquitectura multi-tenant: Modelo de software donde una única instancia de la aplicación atiende a múltiples organizaciones (empresas), manteniendo los datos de cada una lógicamente aislados [38]. En este proyecto se implementa mediante una columna empresa_id en las tablas principales del sistema, garantizando que los usuarios de una empresa no puedan acceder a los datos de otra. Adicionalmente, incluye un endpoint público de auto-registro que permite a nuevas empresas incorporarse al sistema sin intervención del administrador central, creando la

organización, el usuario administrador y la asociación entre ambos en una única transacción atómica.

- Cifrado Fernet: Esquema de cifrado simétrico que utiliza AES-128 en modo CBC con autenticación HMAC-SHA256, proporcionando confidencialidad e integridad. En este proyecto se emplea, mediante la librería cryptography de Python [56], para cifrar los embeddings faciales (vectores de características biométricas) antes de almacenarlos en la base de datos, protegiendo los datos sensibles incluso ante accesos no autorizados a la base de datos.
- DeepFace: Framework de reconocimiento facial de código abierto que integra múltiples modelos de deep learning (Facenet, VGG-Face, ArcFace, entre otros) y detectores faciales (RetinaFace, MTCNN, OpenCV). En este proyecto se utiliza con el modelo Facenet para generar embeddings faciales de 128 dimensiones y con un detector configurable mediante variable de entorno, utilizando MTCNN por defecto por su velocidad (tres veces más rápido que RetinaFace con precisión comparable) para localizar rostros en las imágenes capturadas por el ESP32-CAM.

2.2 Tecnologías utilizadas

A continuación se presentarán todas las tecnologías que harán presente durante el desarrollo del proyecto. Dentro de estas

2.2.1 Microcontrolador ESP32-CAM

El ESP32-CAM es un dispositivo de desarrollo que integra un microcontrolador ESP32-CAM con conectividad Wi-Fi y Bluetooth, además de una cámara OV2640. La elección de este dispositivo se debe a su bajo costo, capacidad de procesamiento local, compatibilidad con el entorno de Arduino y su amplia comunidad de soporte. De las características que se pueden destacar encontramos la captura de imágenes para el reconocimiento facial, gestionar sensores biométricos y mantener conectividad directa con el servidor o la red local. Otras alternativas como la Raspberry Pi Zero W o Arduino MKR1000 fueron descartadas por su mayor consumo y costo implementación. [33]

2.2.2 Lector de huella digital AS608

El lector de huellas digital usa el sensor AS608, un lector de huella con la capacidad de registrar y verificar la identidad de los usuarios mediante patrones dactilares. La elección de este dispositivo fue por la compatibilidad con el ESP32-CAM, su bajo

consumo y su resistencia a condiciones de luz ambiental en comparación a otros sensores ópticos tradicionales como el R307. El AS608 se comunica con el microcontrolador a través de una interfaz UART, y su integración se realiza mediante la biblioteca Adafruit FingERPrint Sensor Library [1], lo que facilita el registro, búsqueda y verificación de huellas directamente desde el dispositivo IoT. [69]

2.2.3 Protocolos de comunicación HTTP

El protocolo de comunicación HTTP (Hypertext Transfer Protocol) es un protocolo cliente-servidor utilizado para transmitir información mediante solicitudes y respuestas que se transmite a través de los métodos GET, POST, PUT Y DELETE, generalmente utilizando formatos estructurados como JSON (JavaScript Object Notation). Permite la comunicación directa entre dispositivos y servidores, por lo que para este proyecto se le puede dar un buen uso para realizar la petición de información. [37]

2.2.4 Protocolo de comunicación MQTT

El protocolo de comunicación MQTT o Message Queuing Telemetry Transport es un protocolo ligero de mensajería diseñado específicamente para entornos IoT. Se base en la arquitectura de publicador-suscriptor, donde los dispositivos envían mensajes a un broker central que los distribuye entre los clientes suscritos a los distintos tópicos que pueden existir. Este modelo reduce el uso de ancho de banda, lo que lo hace ideal para redes con conectividad limitada o intermitente. [46]

2.2.5 Contenedores Docker

Docker es una plataforma de virtualización ligera que permite empaquetar aplicaciones junto las dependencias dentro de servidores virtuales aislados denominados contenedores. Cada contenedor ejecuta un entorno idéntico al que se utilizaría en producción, lo que facilita el despliegue, la portabilidad de los sistemas. Para el área de IoT y web, Docker es útil para ejecutar servicios como bases de datos, servidores o brokers MQTT de forma independiente y controlada, lo que lo hace útil para el trabajo al momento de levantar los servicios en distintos servidores. [16]

2.2.6 Sistemas de bases de datos

Un sistema de gestor de bases de datos es un conjunto de programas que permiten administrar bases de datos de manera estructurada. La principal función es proporcionar una interfaz entre los usuarios y los datos almacenados, garantizando integridad consistencia y seguridad de la información. Los SGBD relacionales, basado en el

modelo propuesto por Edgar F. Codd (1970), organizan los datos en tablas vinculadas mediante relaciones, lo que facilita su consulta a través del lenguaje SQL. Los SGBD más utilizados que se destacan son MySQL, MariaDb, Oracle Database y PostgreSQL. Estos sistemas ofrecen las características fundamentales, tales como el control de concurrencia, mecanismos de respaldo y recuperación, gestión de usuarios y soporte para transacciones atómicas (ACID). PostgreSQL es un sistema de base de datos relacional y orientado a objetos de código abierto, desarrollado inicialmente en la Universidad de California en Berkeley, caracterizada principalmente por ofrece un motor avanzado con soporte para consultas complejas, procedimientos almacenados, tipos de datos personalizados, índices avanzados y compatibilidad con JSON y XML, lo que permite la fácil integración con aplicaciones web e IoT. [64] Para el contexto del proyecto, PostgreSQL [50] se utiliza como base de datos, ya que tiene compatibilidad con entornos de desarrollo como Python.

2.2.7 Sensor PIR (HC-SR501)

El sensor infrarrojo pasivo HC-SR501 es un módulo de bajo costo diseñado para detectar movimiento mediante la medición de cambios en la radiación infrarroja del entorno [34]. Cuando una persona se sitúa dentro de su campo de visión (aproximadamente 6 metros de alcance y 120° de apertura), el sensor emite una señal digital HIGH. En este proyecto se utiliza como detector de presencia para activar los mecanismos de captura biométrica solo cuando hay una persona frente al dispositivo, optimizando así el consumo energético del lector de huellas AS608 y la cámara OV2640 durante los períodos de inactividad.

La elección del sensor PIR, en lugar del sensor IR activo originalmente planificado, respondió a dos factores: Primero, la extrema limitación de pines GPIO disponibles en el ESP32-CAM tras la asignación del bus paralelo de la cámara, que hubiera requerido sacrificar otra funcionalidad para añadir el sensor IR. Por otra parte, la disponibilidad del detector anti-spoofing integrado en DeepFace, que proporciona una capa de seguridad contra suplantación mediante software sin necesidad de hardware adicional [61].

2.2.8 Framework DeepFace

DeepFace es un framework de reconocimiento facial para Python desarrollado por Sefik Ilkin Serengil [61], que envuelve múltiples modelos de deep learning en una API unificada. Soporta los modelos Facenet [59], VGG-Face, ArcFace, DeepFace, DeepID y Dlib, así como los detectores faciales RetinaFace [15], MTCNN [72], OpenCV [44], SSD y Dlib. Para este proyecto se seleccionó el modelo Facenet por su equilibrio entre precisión y tamaño de embedding (128 dimensiones). El detector facial es configurable mediante

variable de entorno, utilizando MTCNN por defecto por su velocidad superior (tres veces más rápido que RetinaFace), lo que resulta crítico para un dispositivo embebido como el ESP32-CAM.

DeepFace incluye además un módulo de anti-spoofing capaz de distinguir entre rostros reales y fotografías o pantallas, analizando micro-texturas y reflectancia. Como capa adicional de seguridad antes de invocar DeepFace, el sistema aplica un filtro de calidad de imagen basado en la varianza del Laplaciano [48], que rechaza imágenes excesivamente borrosas o planas (posibles fotos de pantalla o papel) antes de llegar al modelo. Para optimizar el rendimiento, el modelo Facenet se precarga al iniciar el servidor y los embeddings de todos los trabajadores se mantienen en una caché en memoria con tiempo de vida configurable, eliminando la latencia de consultas repetitivas a la base de datos. El sistema soporta además múltiples fotos de referencia por persona (multi-encoding), almacenadas en una tabla dedicada `encodings_faciales`, lo que permite comparar cada captura en vivo contra todos los embeddings de referencia y seleccionar la menor distancia, mejorando la precisión ante variaciones de iluminación y ángulo.

2.2.9 Broker Mosquitto en contenedor Docker

Eclipse Mosquitto [18] es un broker MQTT de código abierto y de bajo consumo de recursos, lo que lo vuelve una opción habitual para arquitecturas IoT como la de este proyecto, donde el broker debe convivir con otros servicios sin demandar demasiada memoria ni procesamiento. Se distribuye también como imagen oficial de Docker, lo que permite desplegarlo junto al resto del backend sin depender de una instalación nativa en el sistema operativo.

MQTT puede transportarse de dos formas: mediante una conexión TCP directa al broker, o encapsulando los mismos mensajes sobre una conexión WebSocket. En este proyecto el ESP32-CAM se conecta al broker exclusivamente por esta segunda vía, por dos razones. La primera es práctica: la versión del cliente MQTT del entorno ESP-IDF utilizada en el firmware solo soporta ese modo de transporte. La segunda es de diseño: al viajar sobre el mismo protocolo que usa cualquier navegador web, atraviesa firewalls y proxies corporativos con mayor facilidad que una conexión MQTT cruda sobre un puerto no estándar [19], lo cual es relevante para un dispositivo pensado para operar en redes de pequeñas y medianas empresas, donde no siempre se puede garantizar que ese puerto esté abierto.

2.3 Estado del arte

A continuación se revisan los sistemas y tecnologías existentes en materia de control de asistencia biométrica y su integración con plataformas empresariales, con el propósito de identificar el estado actual del mercado, los avances más relevantes y las limitaciones que motivan el desarrollo de la propuesta de este proyecto.

2.3.1 Soluciones basadas en hardware

Los dispositivos biométricos de control de asistencia permiten registrar las marcaciones de los trabajadores mediante la verificación de características físicas únicas, como huellas digitales o reconocimiento facial. Estos equipos incorporan funcionalidades como almacenamiento interno y conectividad por red (LAN/Wi-Fi), lo que los hace adecuados para entornos con requisitos de operación autónoma. A continuación se describen los principales dispositivos presentes en el mercado chileno.

1. GeoVictoria, relojes de control: GeoVictoria ofrece hardware biométrico integrado con una plataforma en la nube para el control de asistencia. Sus dispositivos pueden operar mediante conexión LAN o 3G en zonas remotas. No obstante, su API está sujeta a restricciones comerciales y acceso por suscripción, lo que limita la personalización y la integración independiente. El ecosistema cerrado dificulta cualquier modificación fuera del entorno del fabricante [26].
2. ZKTeco, terminales biométricas de asistencia: ZKTeco es una marca internacional con presencia en Chile y Latinoamérica. Ofrece terminales de control “stand-alone” que combinan huella digital, reconocimiento facial, PIN y tarjeta. Su principal limitante es la ausencia de integración directa con sistemas ERP; la exportación de datos se realiza mediante archivos CSV, lo que dificulta su uso en entornos empresariales más complejos [73].
3. Dispositivos genéricos: corresponden a relojes control adquiribles en portales de venta en línea que incluyen marcaje por huella o rostro. Aunque incorporan estas funciones a bajo costo, la información queda almacenada únicamente en el dispositivo y solo puede exportarse por USB en formatos de difícil integración con sistemas externos.

2.3.2 Soluciones basadas en software

Las plataformas de gestión de asistencia basadas en software o servicios SaaS permiten administrar las jornadas laborales de forma remota, sin requerir un dispositivo biométrico físico. Si bien facilitan la gestión centralizada, presentan dependencias de conectividad

y restricciones de personalización que las hacen poco adecuadas para entornos con infraestructura limitada.

1. Talana/Buk: son plataformas chilenas que ofrecen módulos de gestión de asistencia y turnos mediante aplicaciones web. Ambas dependen de conectividad a internet permanente, carecen de hardware biométrico propio y requieren suscripciones comerciales para su uso [65] [10].
2. Defontana/Softland: son sistemas ERP que integran módulos de control de asistencia, permitiendo registrar jornadas, calcular horas y conectar con el módulo de remuneraciones. Su principal limitante es que el control de asistencia solo está disponible como parte del ERP completo, lo que implica una dependencia total de la plataforma del proveedor [13] [62].

Del análisis anterior se concluye que el mercado chileno dispone de una oferta variada en soluciones de control de asistencia, tanto en hardware biométrico como en plataformas SaaS. Sin embargo, ninguna de las alternativas revisadas combina autenticación biométrica local con integración flexible hacia sistemas ERP sin depender de infraestructura o suscripciones externas. Esta brecha es la que motiva el desarrollo de la propuesta presentada en este proyecto.

2.4 Metodologías

En este apartado se presentan las metodologías de desarrollo y evaluación que serán utilizadas para este proyecto.

2.4.1 Metodología de desarrollo

Considerando que el proyecto está relacionado con el desarrollo de un prototipo IoT que integra hardware, firmware y software, y pensando en su futura implementación en un entorno real, se identificaron y evaluaron distintos modelos del ciclo de vida de desarrollo. Para el contexto específico de este trabajo, se contrastó el enfoque tradicional del Modelo en Cascada (Lineal Secuencial) frente al modelo de Prototipado Evolutivo Incremental.

El Modelo en Cascada exige definir la totalidad de los requerimientos y diseñar el sistema completo de forma estricta antes de comenzar cualquier implementación. En proyectos IoT, este enfoque presenta un alto riesgo: las restricciones técnicas del hardware (como las limitaciones de memoria del ESP32-CAM o la disponibilidad de pines GPIO para los sensores) a menudo se descubren durante la etapa de construcción. Esperar hasta las fases finales del proyecto para integrar y probar la comunicación entre

los componentes físicos, el backend y la base de datos podría resultar en fallas críticas y la necesidad de rediseñar el sistema completo de forma tardía [51].

Por esta razón, se descartó el enfoque secuencial y se optó por emplear una metodología de prototipado evolutivo incremental, adaptándola a la dinámica de un proyecto en solitario. En este esquema, el profesor guía asume el rol de contraparte o cliente, entregando retroalimentación técnica al final de cada ciclo. Esta metodología es la más indicada para el proyecto, ya que permite mitigar riesgos técnicos construyendo, probando y validando módulos independientes de manera progresiva [66] [25].

La metodología de prototipado evolutivo con un enfoque incremental propone un desarrollo basado en versiones sucesivas del prototipo, en la que cada iteración incorpora nuevas funcionalidades y mejoras sobre el anterior. De esta forma se puede probar, evaluar y ajustar continuamente el sistema a medida que va evolucionando. La estructura de la metodología, representada más adelante en la Figura 3.1 del capítulo 3, se compone de cuatro fases principales:

- **Diseño y planificación:** Se definen los objetivos específicos de cada iteración, se seleccionan componentes y se diseña la arquitectura.
- **Implementación y prueba:** Se desarrolla la funcionalidad planificada, se integra con los módulos que ya existen y se verifica su correcto funcionamiento
- **Evaluación y mejora:** Una vez desarrollados los procesos anteriores se analizan los resultados obtenidos, identificando errores y planificando ajustes para la siguiente iteración.

[51]

Cada una de estas fases termina con un incremento funcional validado, lo que permite mantener un progreso controlado y documentado del prototipo.

2.4.2 Metodología de evaluación

Para la evaluación del proyecto se deben considerar 5 aspectos claves para determinar la viabilidad del proyecto, los cuales permiten evaluar y verificar el cumplimiento de los objetivos planteados. Las pruebas evaluarán el prototipo, enfocándose en cómo funciona con todas las piezas y/o software en conjunto, en qué tan útil puede llegar a ser, en validar la lógica del backend desde sus endpoints sin conocer la implementación interna, en verificar cada módulo de forma aislada y automatizada, y finalmente, dado que se tiene como objetivo su uso en un ERP, en conocer el nivel de satisfacción que se tiene del prototipo final.

Pruebas de integración

Las pruebas de integración permiten evaluar diferentes componentes de un sistema ya sea hardware software o base de datos, asegurando que todos estos componentes funcionen correctamente en conjunto asegurando la comunicación e interoperabilidad entre ellos. Para este caso se realizarán pruebas de integración ascendentes de manera que se vayan realizando pruebas de los módulos más pequeños hasta llegar al módulo más grande [28]. La selección de esta prueba está relacionada con la metodología de desarrollo que consiste en el prototipado evolutivo, en el cual se integran progresivamente los módulos validados en etapas previas.

Pruebas de usabilidad

Las pruebas de usabilidad tienen como objetivo evaluar la facilidad de uso, comprensión y satisfacción del usuario con el sistema, conforme a los estándares de calidad de software [29] [42]. Para llevar a cabo las pruebas de usabilidad se utilizará el System Usability Scale (SUS) [9]. El SUS es un método reconocido por su simplicidad de aplicación y por permitir una evaluación rápida y fiable de la usabilidad de un sistema [6]. La metodología del SUS consiste en obtener una puntuación global de usabilidad mediante la aplicación de un cuestionario basado en una escala de Likert de 5 puntos, compuesto por 10 preguntas que abordan diferentes aspectos de la experiencia de usuario, como la facilidad de uso, la claridad de la interfaz y la eficacia del sistema [9].

La razón del uso de esta prueba se debe a que el prototipo propuesto "prototipo de Control de asistencia Biométrico", debe ser fácil de usar para que los trabajadores puedan realizar marcaciones sin complicaciones, como así para quienes integran este sistema a su ERP.

Pruebas de caja negra

Las pruebas de caja negra son una técnica de validación de software en la cual se evalúa el comportamiento del sistema exclusivamente a partir de sus entradas y salidas, sin conocer ni inspeccionar su implementación interna [52]. Este enfoque se seleccionó para el presente proyecto debido a que el backend implementa una API REST con múltiples endpoints que deben responder correctamente ante combinaciones válidas e inválidas de parámetros, encabezados de autenticación y distintos roles de usuario.

Dado que el sistema incorpora lógica de negocio compleja (modelo multi-tenant por empresa, control de acceso basado en roles jerárquicos, enrolamiento biométrico mediante PIN de un solo uso y sincronización diferida de datos offline), las pruebas de caja negra permiten verificar que cada endpoint cumpla su función sin necesidad de examinar el código fuente, cubriendo así la mayor parte de la aplicación desde la

perspectiva del consumidor de la API. Esta técnica se complementa con las pruebas unitarias automatizadas para lograr una cobertura integral del backend.

Pruebas unitarias automatizadas

Las pruebas unitarias automatizadas permiten verificar de forma aislada y repetible el comportamiento de funciones, métodos y módulos individuales del sistema. Para este proyecto se implementó una suite de pruebas utilizando el framework pytest sobre Python 3.12, complementada con herramientas de mocking (unittest.mock, pytest-mock) para simular dependencias externas críticas como los modelos de reconocimiento facial DeepFace, las operaciones de visión por computadora con OpenCV, la comunicación MQTT con dispositivos ESP32-CAM y el envío de correos electrónicos vía SMTP [43].

Esta estrategia de simulación permite ejecutar las pruebas de forma determinista en cualquier entorno (incluyendo pipelines de integración continua) sin depender de hardware físico ni de servicios externos reales. Las pruebas unitarias se enfocan específicamente en validar la corrección de la lógica de negocio, la integridad del cifrado de datos biométricos, la robustez ante condiciones de error como fallos de conexión a la base de datos o webhooks ERP inaccesibles, y la precisión de los mecanismos de control de acceso por roles.

Pruebas de confiabilidad y disponibilidad

Las pruebas de confiabilidad y disponibilidad son cruciales en sistemas que operan en entornos con conectividad intermitente o hardware limitado, como los dispositivos IoT [39]. La finalidad de la prueba de confiabilidad es medir la capacidad del sistema para funcionar correctamente bajo condiciones normales durante un periodo prolongado sin errores ni pérdidas de datos, mientras que las pruebas de disponibilidad hacen referencia a la capacidad del sistema para mantenerse operativo incluso cuando se presentan condiciones extremas, tales como pérdida de conexión o desconexión temporal de energía [5], etc. Según Perry, estas pruebas son esenciales para garantizar que el sistema mantenga la integridad de los datos y su disponibilidad en todo momento [49].

2.5 Resumen del capítulo

El capítulo 2 presenta el marco teórico que sustenta el proyecto, describiendo los conceptos esenciales relacionados con control de asistencia, IoT, ERP, y los requisitos normativos definidos por la Resolución Exenta N.º38. Luego se detallan las tecnologías utilizadas, incluyendo el microcontrolador ESP32-CAM, el lector de huellas AS608, los protocolos HTTP y MQTT, el uso de contenedores Docker y la base de datos

PostgreSQL.

También se revisa el estado del arte, analizando las limitaciones de dispositivos biométricos comerciales y plataformas SaaS como Talana, Buk, Defontana y Softland, evidenciando la falta de soluciones híbridas, abiertas y con funcionamiento offline. Finalmente, el capítulo expone las metodologías de desarrollo y evaluación seleccionadas, donde se justifica el uso del prototipado evolutivo incremental y se describen las pruebas de integración, usabilidad, caja negra, unitarias automatizadas, y confiabilidad y disponibilidad que se aplicarán al sistema.

3. Metodologías

El presente capítulo está destinado a describir la manera en la que se aplicarán las metodologías propuestas en el capítulo anterior, además de presentar los resultados esperados.

3.1 Metodología de desarrollo

Como se menciona el capítulo anterior en la sección 2.4.1 el proyecto propuesto se lleva a cabo mediante una metodología de tipo prototipado evolutivo con enfoque incremental. Esta metodología permite crear versiones funcionales del prototipo de manera progresiva, mejorando en cada iteración en base a pruebas funcionales del prototipo.

A continuación se procederá a explicar las distintas fases del proceso y como se aplicará la metodología.

3.1.1 Prototipado Evolutivo Incremental

La metodología de prototipado evolutivo con un enfoque incremental propone un desarrollo basado en versiones sucesivas del prototipo, en la que cada iteración incorpora nuevas funcionalidades y mejoras sobre el anterior. De esta forma se puede probar, evaluar y ajustar continuamente el sistema a medida que va evolucionando. Como se ilustra en la Figura 3.1, la estructura de la metodología se compone de cuatro fases principales:

- **Requerimientos Iniciales:** Se identifica la problemática a resolver, se definen los objetivos específicos del prototipo para la iteración actual y se establecen las restricciones operativas y el alcance.
- **Diseño del Prototipo:** Se define la arquitectura preliminar y la funcionalidad mínima esperada, seleccionando las tecnologías y componentes de hardware o software necesarios.

- **Implementación Incremental:** Se construyen las versiones funcionales, programando e integrando progresivamente los componentes hasta obtener un prototipo ejecutable.
- **Pruebas y Validación:** El prototipo ejecutable se somete a pruebas funcionales. Se realizan los ajustes y correcciones pertinentes, y la retroalimentación obtenida retroalimenta directamente a la etapa de diseño para iniciar la siguiente iteración.

Cada uno de estos ciclos termina con un incremento funcional validado, lo que permite mantener un progreso controlado y documentado del prototipo a lo largo del tiempo. A continuación, se presenta el diagrama que representa este flujo de trabajo.

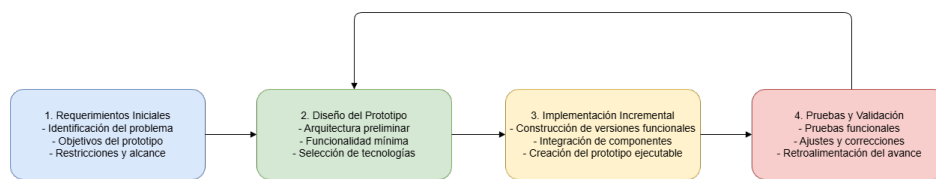


Figura 3.1: Metodología prototipado evolutivo con enfoque incremental

1. Requerimientos Iniciales

En esta fase de arranque se definen los objetivos específicos de la iteración correspondiente. Para cada ciclo se identifican las necesidades puntuales, como las limitaciones de memoria del ESP32-CAM o la necesidad de aislar datos por empresa. Esta etapa garantiza que cada iteración tenga un alcance claro y responda a una problemática técnica o funcional bien delimitada, estableciendo las bases para el trabajo posterior.

2. Diseño del Prototipo

A partir de los requerimientos, se definen los módulos que se incorporarán al prototipo. Se desarrollan diagramas preliminares, modelos de datos, mockups de interfaz y definiciones de comunicación entre módulos (HTTP o MQTT). Asimismo, se determinan las dependencias entre componentes, lo que permite asegurar que el incremento planificado se pueda integrar de manera fluida y coherente con la arquitectura de las iteraciones anteriores.

3. Implementación Incremental

En esta fase se procede a construir la funcionalidad definida en el diseño. La implementación incluye tareas prácticas como el desarrollo del firmware del ESP32-CAM, la programación del lector de huellas AS608, la gestión del almacenamiento en

LittleFS, el desarrollo del backend en Flask y la integración de modelos biométricos. El objetivo es materializar el diseño en un bloque de código y hardware real, logrando un componente ejecutable.

4. Pruebas y Validación

Finalizada la implementación, el incremento es sometido a pruebas integradas para verificar su correcto funcionamiento. Se evalúa tanto el comportamiento individual del módulo como su interacción con los componentes ya existentes. Si se detectan errores, se documentan y se corrigen antes de cerrar la iteración. Esta fase es fundamental para asegurar la calidad del prototipo y validar que cumple con los requerimientos definidos al inicio del ciclo antes de pasar al siguiente incremento.

3.2 Plan de trabajo

La planificación del proyecto se estructura mediante la metodología de prototipado evolutivo con enfoque incremental descrita en la sección 3.1. Cada iteración incorpora nuevas funcionalidades sobre el prototipo anterior, permitiendo evaluar, corregir y fortalecer el sistema progresivamente. Si bien a cada iteración se le asignó un plazo fijo de 3 semanas, el avance acumulado de la Tabla 3.1 se distribuye según la complejidad relativa estimada de cada una, considerando la cantidad de subsistemas involucrados, el riesgo técnico y el volumen de pruebas asociado. Iteraciones como la construcción del backend completo, el reconocimiento facial o el panel web con integración ERP concentran un mayor peso porque integran más componentes y mayor riesgo técnico, mientras que iteraciones de alcance más acotado, como el módulo antifraude, aportan un peso menor al avance total.

Iter.	Objetivo	Duración	Peso	Avance
1	Integración HW + servidor embebido	3 semanas	8%	8%
2	Almacenamiento local + modo offline	3 semanas	8%	16%
3	Backend + BD + comunicación HTTP/MQTT	3 semanas	16%	32%
4	Reconocimiento facial + anti-spoofing + cifrado	3 semanas	18%	50%
5	Autenticación JWT + multi-tenant + enrolamiento	3 semanas	14%	64%
6	Módulo antifraude (PIR + flash + firma de movimiento)	3 semanas	10%	74%
7	Panel web gestión dispositivo + integración ERP	3 semanas	16%	90%
8	Sincronización diferida + logs + cierre	3 semanas	10%	100%

Cuadro 3.1: Plan de trabajo del proyecto: 8 iteraciones de 3 semanas cada una (24 semanas total). El “Peso” refleja la complejidad relativa estimada de cada iteración y el “Avance” es la suma acumulada de los pesos.

El avance de cada iteración se evalúa mediante la superación de las pruebas funcionales definidas en la fase de implementación de cada ciclo. La metodología de evaluación detallada (pruebas de integración, usabilidad SUS, confiabilidad y disponibilidad) se describe en la sección 3.3. El desarrollo detallado de cada iteración, incluyendo análisis, diseño, implementación y resultados de pruebas, se presenta en el capítulo 4.

3.3 Metodología de evaluación

En esta sección se explica de qué manera se aplicaron, dentro del desarrollo del prototipo, las metodologías de evaluación descritas en el apartado 2.4.2 del marco teórico. Cada técnica de evaluación fue seleccionada en función de los objetivos del proyecto y del enfoque metodológico utilizado prototipado evolutivo con incrementos funcionales, lo que permitió validar el funcionamiento conjunto del hardware, software, backend y mecanismos de autenticación biométrica en entornos reales o simulados. Las evaluaciones se aplicaron de manera iterativa al finalizar cada incremento del prototipo, garantizando que las funcionalidades incorporadas funcionaran correctamente antes de avanzar al siguiente nivel de complejidad.

3.3.1 Pruebas de integración

Las pruebas de integración, como por su nombre dice son pruebas que permiten integrar componentes de un sistema verificando que su interacción sea correcta cuando se implementa un nuevo módulo o dispositivo como es en el contexto de este proyecto. Dado que el proyecto está compuesto por múltiples módulos independientes, lector de huellas, cámara del ESP32-CAM, el almacenamiento local, backend, reconocimiento facial, sistema antifraude (PIR + anti-spoofing), panel web y módulo ERP, es necesario verificar su interoperabilidad a medida que se ensamblan.

Con el objetivo de asegurar que todos los componentes mencionados anteriormente interactúen correctamente entre sí se realizarán pruebas de integración de manera ascendente. El procedimiento que irá realizando el siguiente procedimiento.

1. Integración del hardware: Se evaluará la comunicación entre el ESP32-CAM y el módulo de huella digital. Además de comprobar la lectura y envío de datos de sensores IR.
2. Integración Almacenamiento: Se realizarán pruebas entorno al almacenamiento de los identificadores de huella, imágenes capturadas además de los registro de asistencia, donde se verificará la consistencia del almacenamiento.

3. Integración con el backend: Se enviarán distintos tipos de datos al backend, imágenes en base64, registros de logs, entre otros para verificar la comunicación entre el ESP32 y el backend.
4. Integración Backend con la base de datos: Se realizarán inserciones y búsqueda de elementos para verificar la conectividad constante con la base de datos.
5. Integración backend con interfaz web: Se comprobará si se rescatan los registros enviados por el prototipo y actualización en tiempo real con los datos.

Estas pruebas permiten evaluar el sistema propuesto de manera sistemática, de manera que cada elemento más pequeño se va integrando a medida que se avanza en el proyecto.

3.3.2 Pruebas de usabilidad

Las pruebas de usabilidad, como se mencionó en el punto 2.4.2, permiten evaluar la facilidad de uso, la eficacia del sistema y la claridad de la interfaz. Dado que los usuarios esperados del prototipo incluyen tanto trabajadores (usuarios no técnicos) como desarrolladores e integradores ERP (usuarios técnicos), la evaluación debe abarcar ambos perfiles.

Para llevar a cabo esta evaluación se utilizará el cuestionario System Usability Scale (SUS) desarrollado por Brooke [9], compuesto por 10 preguntas que se responden en una escala Likert de 5 puntos (1 = “Totalmente en desacuerdo”, 5 = “Totalmente de acuerdo”). Las preguntas impares son afirmaciones positivas y las pares son negativas (invertidas). Se aplica el mismo cuestionario a ambos grupos de usuarios, para poder comparar los puntajes entre sí.

Cuestionario SUS estándar

A continuación se presentan, en la Tabla 3.2, las 10 preguntas del instrumento SUS en su traducción al español, utilizadas para ambos perfiles de usuario:

N.º	Pregunta	Tipo
1	Creo que me gustaría usar este sistema con frecuencia.	Positiva
2	Encontré el sistema innecesariamente complejo.	Invertida
3	Pensé que el sistema era fácil de usar.	Positiva
4	Creo que necesitaría apoyo de una persona técnica para poder usar este sistema.	Invertida
5	Consideré que las funciones del sistema estaban bien integradas.	Positiva
6	Pensé que había demasiada inconsistencia en el sistema.	Invertida
7	Imagino que la mayoría de las personas aprenderían a usar este sistema rápidamente.	Positiva
8	Encontré el sistema muy engorroso de usar.	Invertida
9	Me sentí seguro al usar el sistema.	Positiva
10	Necesité aprender muchas cosas antes de poder empezar a usar este sistema.	Invertida

Cuadro 3.2: Cuestionario SUS estándar (Brooke, 1996), traducción al español.

La puntuación SUS se calcula según el método descrito por Brooke [9]: para cada pregunta impar se resta 1 a la respuesta, y para cada pregunta par se resta la respuesta de 5. La suma de los 10 valores se multiplica por 2.5, obteniendo una puntuación entre 0 y 100. Según Bangor et al. [6], una puntuación ≥ 70 se considera “aceptable”, entre 50 y 70 “marginal” y < 50 “no aceptable”.

Adicionalmente, se aplicarán preguntas complementarias de contexto específicas para cada grupo (ver Anexo A), las cuales no forman parte del cuestionario SUS estándar y se analizan de forma descriptiva separada.

3.3.3 Pruebas de confiabilidad y disponibilidad

Para un prototipo IoT con funcionamiento offline, la confiabilidad del almacenamiento local y la disponibilidad del sistema durante fallas de red son esenciales. Estas pruebas evaluarán qué tan robusto puede llegar a ser el prototipo frente a escenarios reales de operaciones y fallos. Para el procedimiento se realizará lo siguiente.

- Escenario de conectividad: Para realizar esta prueba se consideran distintos escenarios que permita evaluar el funcionamiento del prototipo en estos escenarios de los cuales consideramos escenarios de desconexión total, conectividad intermitente, reconexión luego de una desconexión prolongada.
- Métricas a evaluar: Dentro de las métricas que se evaluarán corresponden a los porcentajes de registros almacenados sin errores, cantidad de re intentos

automáticos, consistencia del log de eventos, tiempo de recuperación tras re conectar (sincronizar datos).

- Verificación con base de datos: Con la finalidad de comprobar la confiabilidad del sistema se debe comparar el número de registros capturados offline con los almacenados en la base de datos o que fueron sincronizados anteriormente.

3.3.4 Pruebas de caja negra

Las pruebas de caja negra evalúan el comportamiento del sistema a partir de sus entradas y salidas, sin examinar su estructura interna [52]. Se diseñan casos de prueba basados en la especificación de requisitos, explorando valores válidos, condiciones de borde y entradas erróneas [41]. Como no requieren conocer el código fuente, son útiles para validar que el comportamiento externo del sistema coincida con lo documentado.

En este proyecto, las pruebas de caja negra verifican el funcionamiento de la API REST desde la perspectiva de sus consumidores (dispositivos ESP32-CAM, panel web y sistemas ERP). Para cada endpoint se revisa que el código de estado HTTP sea el correcto, que la estructura de los cuerpos JSON coincida con lo esperado, que los mensajes de error sean claros sin exponer detalles internos, y que los mecanismos de autenticación y autorización (JWT con roles jerárquicos) restrinjan el acceso según el perfil del usuario.

La pertinencia de esta técnica para el proyecto se justifica por tres razones. Primero, la API REST es la interfaz externa única del backend, por lo que cualquier defecto en un endpoint afecta a todos los consumidores del sistema. Segundo, la lógica de negocio (modelo multi-tenant, roles jerárquicos, enrolamiento biométrico con PIN, sincronización offline) genera una cantidad de escenarios funcionales que no puede verificarse solo inspeccionando el código. Tercero, en un proceso de prototipado evolutivo incremental, estas pruebas permiten revalidar el comportamiento de todos los endpoints al final de cada iteración, detectando regresiones sin modificar los casos de prueba existentes.

La cobertura abarcó todos los endpoints del backend, agrupados por área funcional: autenticación y control de acceso, registro de usuarios y empresas, operaciones CRUD sobre entidades del modelo de datos, marcaciones y sincronización offline, y reconocimiento facial. Para cada uno se diseñaron casos de flujo normal y condiciones de error (parámetros faltantes, valores fuera de rango, autenticación insuficiente).

3.3.5 Pruebas unitarias automatizadas

Las pruebas unitarias verifican funciones o módulos de forma aislada, reemplazando dependencias externas con mocks para que el resultado dependa solo de la lógica bajo

prueba [43]. A diferencia de las pruebas de integración, se enfocan en la corrección algorítmica de cada pieza de código, permitiendo localizar defectos con precisión y ejecutar la suite rápidamente.

En este proyecto, las pruebas unitarias cubren cuatro aspectos del backend:

- **Lógica de negocio:** validación de RUT, herencia de roles jerárquicos, asignación de turnos, aislamiento multi-tenant, detección de marcaciones duplicadas y generación de PIN de enrolamiento.
- **Datos sensibles:** cifrado y descifrado de embeddings faciales con Fernet, persistencia de huellas dactilares y gestión de consentimiento biométrico.
- **Robustez ante errores:** manejo de fallos de conexión a la base de datos, timeouts en webhooks ERP, interrupción del servidor SMTP y mensajes MQTT malformados.
- **Control de acceso:** verificación de que los roles jerárquicos (administrador, empleador, trabajador) denieguen el acceso a recursos no autorizados en cada endpoint.

El mocking es necesario porque el sistema depende de componentes externos (DeepFace, OpenCV, broker MQTT, servidor SMTP, webhooks ERP) que requerían hardware físico o servicios activos para ejecutar las pruebas. Al simular estas dependencias, la suite se ejecuta de forma determinista en cualquier entorno (incluyendo CI/CD) sin necesidad del hardware real, lo que permite revalidar todo el código en cada iteración del prototipado evolutivo.

La suite se organizó por módulos del backend: base de datos, cifrado, correo electrónico, MQTT, autenticación, reconocimiento facial, CRUD de entidades, sincronización offline, integraciones ERP y un emulador del ESP32-CAM. Esta organización permite ejecutar solo las pruebas del módulo modificado, acelerando la retroalimentación durante el desarrollo.

3.3.6 Resultados esperados de las pruebas

Este apartado define los valores objetivos y criterios de aceptación que se esperan alcanzar al ejecutar el conjunto de pruebas descritas en la sección 3.3 (Metodología de evaluación). Estos valores sirven como referencia para comparar los resultados reales que se presentarán en el capítulo 5.

Pruebas de integración

Las pruebas de integración ascendente definidas en la sección 3.3.1 apuntan a verificar la correcta comunicación entre los módulos del sistema. Según Pressman [51] e IBM [28], una prueba de integración ascendente debe garantizar el 100% de interoperabilidad entre los módulos verificados, un valor inferior indicaría una falla crítica de acoplamiento que invalidaría el incremento. Se espera alcanzar los resultados de la Tabla 3.3.

Criterio	Valor esperado
Comunicación ESP32-CAM ↔ AS608 (UART, 57600 baud)	100%
Captura de imagen facial (OV2640) → envío al backend	100%
Persistencia de marcaciones en LittleFS tras 50 marcaciones	100%
Sincronización diferida con backend (reintento tras reconexión)	100%
Insertión/consulta en PostgreSQL sin errores	100%
Renderizado de vistas web con datos actualizados	100%

Cuadro 3.3: Resultados esperados de las pruebas de integración.

Pruebas de usabilidad (SUS)

La encuesta System Usability Scale (SUS) descrita en la sección 3.3.2 se aplicará a dos grupos de usuarios. Según Bangor et al. [6], un puntaje $SUS \geq 70$ se considera “aceptable”, entre 50 y 70 “marginal” y < 50 “no aceptable”. Sistemas con flujos de embedding facial y configuración de red suelen situarse en el rango 65 a 75, por lo que se fija como objetivo mínimo un 70 global. Se esperan los puntajes de la Tabla 3.4.

Métrica	Valor esperado
Puntuación SUS - usuarios no técnicos	≥ 68
Puntuación SUS - usuarios técnicos	≥ 70
Puntuación SUS global del sistema	≥ 70
Tasa de completitud de tareas (no técnicos)	$\geq 85\%$
Tasa de completitud de tareas (técnicos)	$\geq 90\%$
Tiempo medio de configuración inicial (no técnicos)	≤ 10 min
Tiempo medio de configuración inicial (técnicos)	≤ 5 min

Cuadro 3.4: Resultados esperados de las pruebas de usabilidad (SUS).

Pruebas de confiabilidad y disponibilidad

Las pruebas de confiabilidad y disponibilidad descritas en la sección 3.3.3 someten al sistema a tres escenarios de conectividad. Según Perry [49] y Atlassian [5], un sistema IoT con operación offline debe garantizar cero pérdida de datos y una disponibilidad acumulada $\geq 90\%$ en escenarios reales de uso para ser considerado operacionalmente viable. Se esperan los resultados de la Tabla 3.5.

Escenario	Métrica	Valor esperado
Desconexión total	Registros almacenados sin error	100%
Desconexión total	Reintentos automáticos de reconexión Wi-Fi	5 antes de AP
Conectividad intermitente	Reintentos exitosos de sincronización	$\geq 95\%$
Conectividad intermitente	Consistencia del log de eventos	0 inconsistencias
Reconexión prolongada	Tiempo de sincronización de 50 marcaciones	< 60 s
Operación continua	Disponibilidad acumulada (uptime)	$\geq 90\%$
Persistencia	Datos conservados tras corte de energía	100%

Cuadro 3.5: Resultados esperados de las pruebas de confiabilidad y disponibilidad.

Pruebas de caja negra

Las pruebas de caja negra descritas en la sección 3.3.4 evaluaron los 28 endpoints de la API REST del backend. Dado que estos endpoints son el punto de entrada principal al backend desde los dispositivos ESP32, el panel web y los sistemas ERP externos, deben verificar exhaustivamente tanto entradas válidas como inválidas. Un endpoint que no responda correctamente ante un error compromete la experiencia de usuario y la integridad de los datos. Se esperan los resultados de la Tabla 3.6.

Criterio	Valor esperado
Endpoints de autenticación (login, me, register, register-company)	100% funcionales
Endpoints de CRUD (personas, turnos, asignaciones, dispositivos, ERP)	100% funcionales
Endpoints de asistencias y sincronización	100% funcionales
Endpoints de reconocimiento facial (identificar, verificar, registrar)	100% funcionales
Endpoints de enrolamiento de dispositivos (PIN generar/enrolar)	100% funcionales
Códigos de error HTTP (400, 401, 403, 404, 409, 500)	100% correctos
Aislamiento multi-tenant por empresa	100%

Cuadro 3.6: Resultados esperados de las pruebas de caja negra.

Pruebas unitarias automatizadas

Las pruebas unitarias automatizadas descritas en la sección 3.3.5 se implementaron con pytest sobre una base de datos PostgreSQL en Docker. En proyectos con componentes biométricos y comunicación IoT, una cobertura de código $\geq 90\%$ es considerada un estándar de calidad robusto para el backend, ya que garantiza que la lógica de negocio, la gestión de errores y los mecanismos de seguridad han sido verificados de forma exhaustiva [52]. Los módulos críticos de seguridad como el cifrado de datos biométricos deben mantener 100% de cobertura para asegurar que no existan caminos de código no probados.

3.4 Resultados Esperados de los Objetivos Específicos

A continuación en esta sección se presentarán los resultados esperados para cada uno de los objetivos específicos planteados en el capítulo 1.

- Objetivo específico 1: Diseñar la arquitectura física y lógica del prototipo IoT, seleccionando componentes que garanticen el funcionamiento autónomo y demostrando su viabilidad económica.
 - Resultado esperado: Se espera obtener una arquitectura física y lógica funcional basada en el ESP32-CAM, con los componentes seleccionados integrados y operando de forma autónoma, alcanzando un presupuesto de hardware total que no supere los \$45.000 CLP.

- Objetivo específico 2: Implementar el módulo de autenticación biométrica híbrida.
 - Resultado esperado: Se espera que el prototipo permita registrar la asistencia de un trabajador mediante autenticación biométrica, logrando un tiempo de procesamiento inferior a 5 segundos por marcación facial y una precisión superior al 90% en la identificación correcta durante las pruebas de laboratorio.
- Objetivo específico 3: Integrar el sistema de archivos local (LittleFS) para la operación offline.
 - Resultado esperado: Se espera que el prototipo opere en modo offline almacenando localmente al menos 1.000 registros de asistencia en formato JSON, sin pérdida de datos ante simulaciones de corte de energía o desconexión de red, para su posterior sincronización con el servidor.
- Objetivo específico 4: Construir una interfaz web embebida y un panel de administración para la configuración del sistema.
 - Resultado esperado: Se espera disponer de una interfaz web embebida y un panel de administración funcionales para la configuración del sistema, validando su facilidad de uso mediante la obtención de un puntaje igual o superior a 68 puntos en la evaluación de usabilidad SUS.
- Objetivo específico 5: Desarrollar una API RESTful que permita la integración automatizada con sistemas ERP externos.
 - Resultado esperado: Se espera contar con una API RESTful que permita la integración automatizada con sistemas ERP externos, logrando sincronizar y transmitir el 100% de los paquetes de datos generados en modo offline una vez restablecida la conectividad.
- Objetivo específico 6: Evaluar la robustez y disponibilidad del prototipo integrado en escenarios de conectividad intermitente.
 - Resultado esperado: Se espera que el hardware y el software funcionen correctamente en conjunto, comprobando la correcta ejecución de los mecanismos de reintento automático en escenarios de conectividad intermitente y obteniendo una disponibilidad de al menos un 90% cuando el dispositivo se encuentre offline o con conexión inestable.

3.5 Resumen del capítulo

El capítulo 3 presenta la metodología de prototipado evolutivo incremental adoptada para el desarrollo del sistema, describiendo sus tres fases (diseño y planificación, implementación y prueba, evaluación y mejora) aplicadas al contexto del proyecto. Se incluye el plan de trabajo con la carta Gantt de las 8 iteraciones, detallando los objetivos, duración estimada y avance acumulado de cada una.

Asimismo, se explica la metodología de evaluación, que incluye pruebas de integración, usabilidad mediante el método SUS, y pruebas de confiabilidad y disponibilidad para medir el desempeño del sistema en escenarios reales. El capítulo concluye con los resultados esperados, tanto de los objetivos específicos del proyecto como de las pruebas a realizar, los cuales serán contrastados con los resultados reales en el capítulo 5.

4. Desarrollo e implementación

En el siguiente capítulo se presentan en detalle las 8 iteraciones del proyecto, desarrolladas según la metodología de prototipado evolutivo incremental descrita en el capítulo 3. Para cada iteración se describen las fases de análisis, diseño, implementación y pruebas, reflejando la construcción progresiva del prototipo desde la integración inicial de hardware hasta la sincronización diferida y el cierre del proyecto.

4.1 Diseño de software

Esta sección está destinada a presentar las arquitecturas desarrolladas para este proyecto como así diagramas y mockups que permitan dar entender la lógica del proyecto como así futuras vistas.

4.1.1 Arquitectura física

La arquitectura física se organiza como un conjunto de servicios independientes que se comunican entre sí, siguiendo un enfoque de microservicios. La Fig. 4.1 muestra los componentes físicos involucrados y el camino que recorre una marcación desde que se genera hasta que queda disponible para la empresa: el ESP32-CAM, como dispositivo de campo, captura la imagen o la huella y la envía hacia el servidor mediante el broker MQTT (Mosquitto) o vía HTTP, según el tipo de operación. El backend (Flask) recibe esa información, invoca al servicio de reconocimiento facial (DeepFace) cuando corresponde, y persiste el resultado en la base de datos PostgreSQL. Desde ahí, el mismo backend se encarga de dos salidas independientes: reenviar la marcación al sistema ERP de la empresa mediante un webhook, y exponerla en el panel web (Next.js) para que el administrador o empleador la consulte en tiempo real.

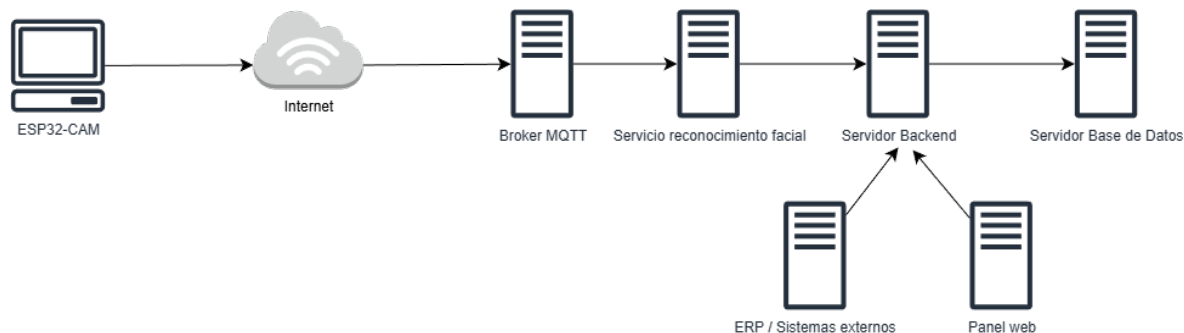


Figura 4.1: Arquitectura física.

4.1.2 Arquitectura lógica

Mientras que la arquitectura física muestra dónde corre cada servicio, la arquitectura lógica de la Fig. 4.2 muestra cómo se organizan los módulos de software dentro de esos servicios y cómo se relacionan entre sí. En el lado del dispositivo, el ESP32-CAM agrupa cuatro módulos: la captura de imágenes (cámara OV2640), la lectura de huella (lector AS608 vía UART), el almacenamiento local en LittleFS (usuarios, turnos, asignaciones y asistencias en formato JSON) y el servidor web embebido que sirve la interfaz de configuración. Estos cuatro módulos no son independientes entre sí: por ejemplo, una marcación capturada por la cámara o el lector se guarda primero en LittleFS y solo después se intenta enviar al backend, lo que permite que el dispositivo siga funcionando aunque no haya conexión.

Del lado del servidor, el backend se organiza en blueprints de Flask (uno por dominio: personas, turnos, asistencias, dispositivos, ERP, etc.), cada uno con acceso a la base de datos PostgreSQL y, cuando corresponde, al servicio de reconocimiento facial. La comunicación entre el dispositivo y el backend ocurre por dos canales según el tipo de mensaje: MQTT para eventos de bajo volumen (heartbeats, comandos, resultados de enrolamiento) y HTTP para el envío de imágenes y la sincronización de datos. Esta separación de canales es la que permite que el dispositivo opere de forma autónoma cuando no hay red, almacene localmente lo que no pudo enviar, y sincronice todo una vez que la conexión se restablece.

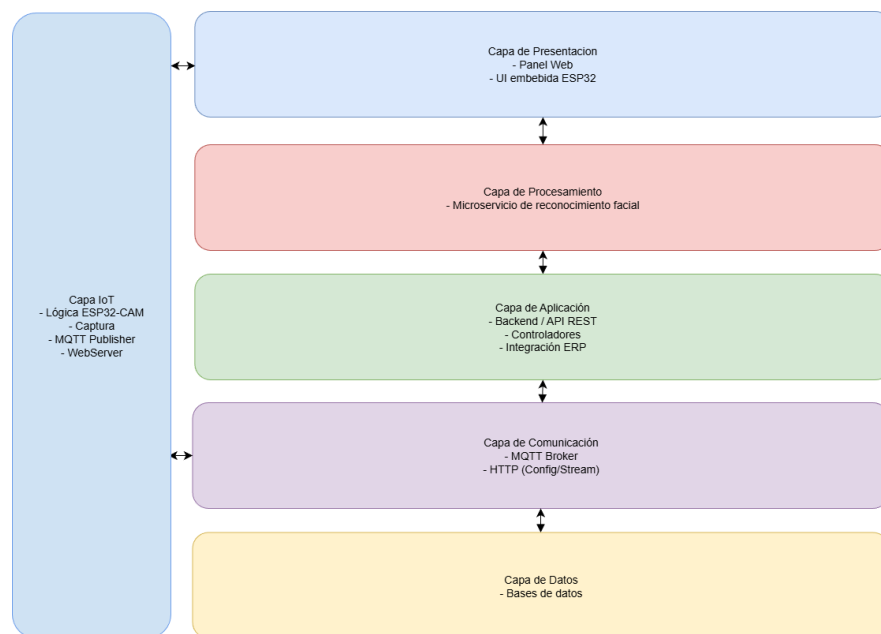


Figura 4.2: Arquitectura lógica.

La Fig. 4.3 detalla, a nivel de hardware y firmware, los mismos cuatro módulos del ESP32-CAM descritos en la arquitectura lógica. En la capa de hardware se distinguen el microcontrolador ESP32, la cámara OV2640 (conectada por el bus paralelo DVP), el lector de huellas AS608 (conectado por UART) y el sensor PIR que activa la captura solo ante presencia detectada. En la capa de firmware, el servidor web embebido y el cliente MQTT comparten el acceso al sistema de archivos LittleFS, mientras que la máquina de estados central coordina cuándo se invoca cada módulo de captura biométrica según si el dispositivo está en modo de registro o de marcaje. Esta vista complementa a la Fig. 4.2, mostrando con qué pines y protocolos concretos se implementa cada bloque lógico.

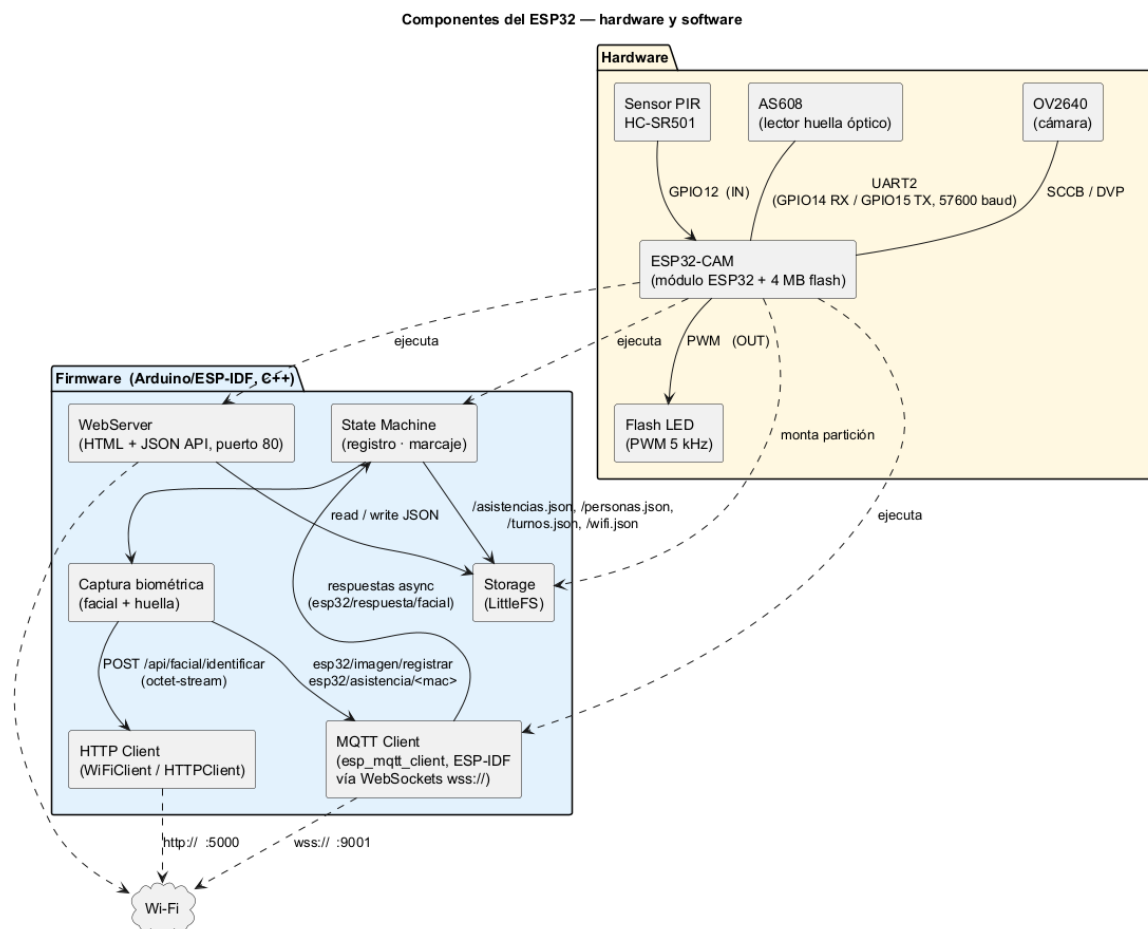


Figura 4.3: Componentes del ESP32-CAM: hardware y firmware.

4.2 Implementación

En este apartado se procederá a explicar cómo se llevaron a cabo las iteraciones planteadas en el capítulo 3.1.

4.2.1 Iteración 1: Integración de hardware y servidor embebido

La primera iteración corresponde al diseño conceptual del sistema y a la preparación del entorno de desarrollo, incluyendo la integración inicial del hardware biométrico seleccionado. Esta etapa establece la base funcional sobre la cual se construirán las iteraciones posteriores.

Análisis

En esta iteración se definieron los requisitos mínimos para validar la viabilidad técnica del sistema, analizando los componentes del hardware y las funcionalidades necesarias

para las iteraciones siguientes.

El análisis se centró en tres ejes principales:

- **Definición de funcionalidades mínimas (MVP):** Se identificaron las capacidades básicas que el dispositivo debía cumplir desde el inicio: capturar imágenes desde la cámara integrada, leer huellas dactilares mediante el sensor AS608, montar un servidor HTTP local con páginas HTML embebidas y operar de forma autónoma mediante Wi-Fi. Estas funciones representan la base operativa sobre la cual se desarrollarán características más avanzadas como turnos, sincronización y gestión de usuarios.
- **Identificación de limitaciones técnicas:** Se revisaron las restricciones tecnológicas del ESP32-CAM, como la limitada memoria RAM disponible (520 KB), el uso compartido del bus de cámara con los pines GPIO, la ausencia de pines GPIO libres -lo que forzó el uso de los pines GPIO14 (RX) y GPIO15 (TX) para la comunicación UART con el lector de huellas- y la necesidad de un manejo eficiente de cadenas HTML grandes en memoria. También se identificó que el flash LED integrado consume corriente significativa (250 mA), requiriendo protección mediante PWM para evitar caídas de tensión durante la captura.
- **Estrategia de operación autónoma:** Dado que uno de los requisitos fundamentales del sistema es la autonomía, se analizó la necesidad de montar un servidor web local. En caso de no contar con Wi-Fi configurada, el dispositivo debe activar automáticamente un modo Access Point (AP) con SSID ESP32-ASISTENCIA, permitiendo al administrador conectarse directamente para configurar la red y registrar usuarios sin dependencia de internet. Esto reduce riesgos y asegura la continuidad operacional del dispositivo.

Como resultado del análisis, se establecieron los criterios de diseño que guiarían el desarrollo de esta iteración: simplicidad, bajo consumo de recursos, modularidad de componentes y autonomía operativa.

Diseño

En esta iteración se elaboraron los primeros elementos arquitectónicos del sistema:

- **Arquitectura física,** representando los componentes utilizados y su interconexión, la cual se puede revisar en la Fig. 4.1 de la sección 4.1.1.
- **Arquitectura lógica,** definiendo los módulos internos del firmware tales como captura de imágenes, lectura de huella, almacenamiento y conectividad, la cual se puede revisar en la Fig. 4.2 de la sección 4.1.2.

- **Mockups iniciales** de la interfaz embebida del ESP32-CAM, incluyendo:
 - Menú principal.

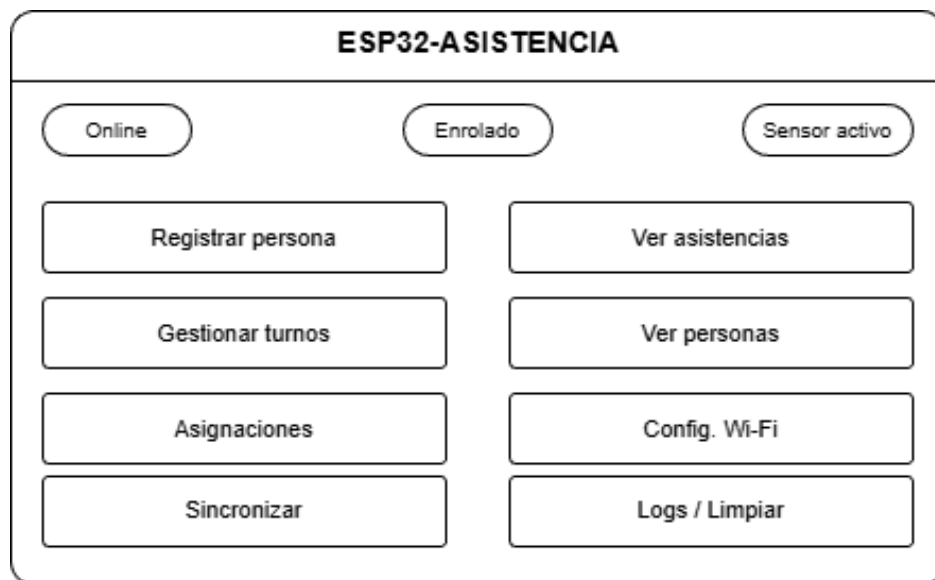


Figura 4.4: Mockup menú principal en el ESP32-CAM.

Wireframe inicial de la vista /: agrupa los indicadores de estado del dispositivo (conectividad, enrolamiento, sensor PIR) en la parte superior, y los accesos a cada módulo del sistema (registro, asistencias, turnos, personas, asignaciones, configuración Wi-Fi, sincronización y logs) como botones de navegación.

- Panel de configuración de Wi-Fi.

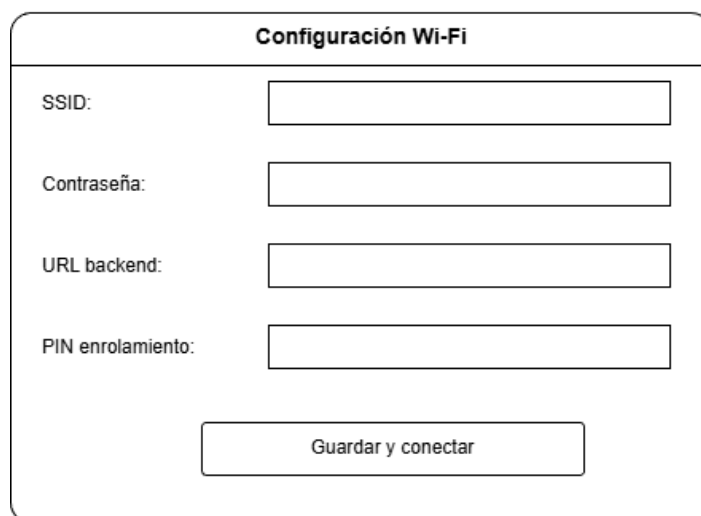


Figura 4.5: Mockup panel de configuración Wi-Fi.

Wireframe de la vista /wifi-setup: formulario con los cuatro campos que se persisten en wifi.json (SSID, contraseña de red, URL del backend y PIN de enrolamiento), accesible también desde el modo Access Point cuando el dispositivo no tiene credenciales configuradas.

- Vista de registro de usuario.

Registrar persona

Nombre:

RUT:

Correo:

Figura 4.6: Mockup registro de usuario en el ESP32-CAM.

Wireframe de la vista /register: formulario de datos básicos (nombre, RUT, correo) seguido de los dos botones de captura biométrica, con un indicador de estado del proceso de enrolamiento (huella y rostro).

- Vista de usuarios almacenados localmente.

Personas registradas		
Nombre	RUT	Estado
Juan Pérez	12.345.678-9	OK
María Soto	9.876.543-2	OK
...	...	...
Editar / Eliminar		

Figura 4.7: Mockup de usuarios almacenados en el ESP32-CAM.

Wireframe de la vista /personas: listado tabular de las personas registradas leídas desde personas.json, con acciones de edición y eliminación por fila.

- Vista de turnos almacenados localmente.

The mockup shows a container titled "Turnos". Inside, there is a table with three columns: "Nombre", "Inicio", and "Fin". The table contains three rows: "Turno mañana" with times "08:00" and "14:00", "Turno tarde" with times "14:00" and "20:00", and an ellipsis row. Below the table is a button labeled "Crear nuevo turno".

Nombre	Inicio	Fin
Turno mañana	08:00	14:00
Turno tarde	14:00	20:00
...	...	...

Crear nuevo turno

Figura 4.8: Mockup de visualización de turnos en el ESP32-CAM.

Wireframe de la vista /turnos: listado de los turnos almacenados en turnos.json, con el botón de acceso a la creación de un nuevo turno.

- Vista de crear turnos.

The mockup shows a container titled "Crear turno". It contains a form with the following elements: a "Nombre:" label followed by a text input field; "Hora inicio:" and "Hora fin:" labels each followed by a time input field; a "Colación:" label followed by a checkbox; and a "Guardar turno" button at the bottom.

Nombre:

Hora inicio: Hora fin:

Colación: ☐

Guardar turno

Figura 4.9: Mockup de creación de turnos en el ESP32-CAM.

Wireframe de la vista /gestion: formulario para definir nombre, horario de inicio y fin, y la casilla opcional de colación que habilita los campos colacion_inicio/colacion_fin descritos en el Capítulo 4.

- Vista de asignación de turnos.

Asignaciones	
Persona:	(seleccionar) ▼
Turno:	(seleccionar) ▼
<button>Asignar</button>	
Persona	Turno asignado
Juan Pérez	Turno mañana

Figura 4.10: Mockup de asignación de turnos en el ESP32-CAM.

Wireframe de la vista /asignaciones: selección de persona y turno mediante listas desplegables, seguida del listado de asignaciones vigentes ya registradas.

- **Diseño del flujo operacional:** a nivel conceptual, se planteó que el dispositivo debía capturar la huella o el rostro y luego evaluar si tenía conectividad disponible. Con conexión, la marcación se enviaría de inmediato al servidor; sin ella, se guardaría localmente para sincronizarse más adelante, cuando la red se restableciera. En ambos casos, la validación final (si la huella o el rostro corresponden a una persona registrada) ocurriría en el servidor, que respondería con un resultado de éxito o de error. El detalle de cómo se implementó cada uno de estos pasos, una vez que todos los componentes involucrados (enrolamiento, reconocimiento facial, sincronización diferida e integración ERP) estuvieron contruidos, se presenta de forma consolidada al cierre del desarrollo, en la Sección 4.2.8.

Estos elementos permiten visualizar el flujo básico del dispositivo antes de incorporar funcionalidades adicionales como turnos, sincronización o panel web backend, los cuales se desarrollarán en iteraciones posteriores.

Implementación

En esta etapa se integró tanto el hardware principal del prototipo como el desarrollo de las vistas web embebidas que permiten la interacción del usuario con el dispositivo. Para ello se utilizaron las siguientes librerías:

- `esp_camera`, utilizada para inicializar y configurar la cámara OV2640 integrada en el ESP32-CAM.

- `HardwareSerial` y `Adafruit_Fingerprint`, empleadas para la comunicación UART con el lector biométrico AS608.
- `WebServer`, utilizada para montar un servidor web HTTP en el puerto 80 y atender rutas asociadas a las vistas.
- `LittleFS` y `ArduinoJson` [8], empleadas para almacenamiento persistente en memoria flash y manipulación estructurada de datos JSON.

Integración del lector de huellas AS608

El lector biométrico AS608 fue conectado directamente al ESP32-CAM mediante la interfaz UART secundaria (`HardwareSerial 2`), utilizando los pines GPIO14 (RX) y GPIO15 (TX). Esta conexión, que emplea pines distintos a los documentados inicialmente, fue forzada por la escasez de GPIO libres tras la asignación del bus de cámara. La velocidad de comunicación se estableció en 57600 baudios, y la librería `Adafruit_Fingerprint` se vincula a dicho puerto serial. La Figura 4.11 presenta la conexión física entre ambos dispositivos.

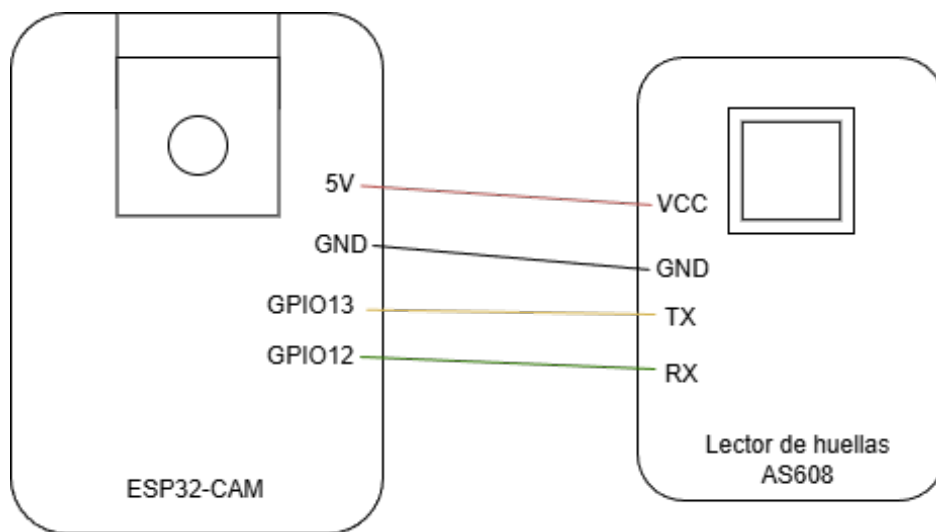


Figura 4.11: Conexiones entre el ESP32-CAM y el lector de huellas AS608.

Una vez establecida la comunicación, el firmware puede ejecutar operaciones como:

- Captura de imagen dactilar.
- Conversión de la imagen en una plantilla biométrica.
- Almacenamiento del template en un slot interno del sensor (1-127).
- Identificación mediante búsqueda de coincidencias en memoria.

La inicialización del lector se realiza con `finger.begin(57600)` sobre el puerto serie secundario, seguida de `finger.verifyPassword()` para confirmar que el sensor responde correctamente antes de habilitar las operaciones de captura, conversión y búsqueda descritas anteriormente.

Configuración de la cámara y el flash PWM

La cámara OV2640 se configuró con resolución VGA (640×480) y compresión JPEG con calidad 8 (escala 0-63, donde valores bajos indican mayor calidad). La frecuencia del reloj externo (XCLK) se fijó en 20 MHz. Para evitar el parpadeo visible y las caídas de tensión provocadas por el flash LED durante la captura, se implementó un control PWM a 5 kHz con resolución de 8 bits, aplicando un ciclo de trabajo del 50% (duty 128 de 255) durante la iluminación. Esto permite encender el flash con brillo suficiente sin sobrecargar la fuente de alimentación del ESP32-CAM.

Calibración del sensor PIR

Se integró un sensor PIR (Passive Infrared) conectado al GPIO12 configurado con resistencia de pull-down interna. El sensor PIR actúa como detector de presencia: cuando una persona se sitúa frente al dispositivo, el PIR detecta el cambio de radiación infrarroja y activa el modo de captura. El setup incluye una fase de calibración de 3 segundos para estabilizar la lectura del sensor antes de iniciar el bucle principal. La lógica de uso del PIR como control de activación se detalla en la iteración 6.

Montaje del servidor web embebido

Para habilitar la interacción local con el dispositivo, se implementó un servidor web mediante la librería `WebServer` en el puerto 80. Este servidor permite entregar páginas HTML, datos JSON y recursos estáticos. Cuando no hay conectividad a una red externa, el ESP32-CAM activa automáticamente un Access Point con SSID ESP32-ASISTENCIA y contraseña Asistencia2026, sirviendo las mismas páginas sin depender de internet.

Las rutas principales del sistema incluyen:

- `/`: Menú principal con indicadores de estado.
- `/wifi-setup`: Configuración de red Wi-Fi y parámetros del backend.
- `/register`: Registro de personas con captura biométrica.
- `/asistencias`: Lista de asistencias almacenadas localmente.
- `/gestion`: Gestión de turnos (creación y administración).
- `/personas`: Visualización de personas registradas.

- /asignaciones: Vista para realizar asignaciones de turnos.
- /logs: Visualización de logs del sistema.
- /turnos: Visualización de turnos creados.

Para la implementación del servidor HTTP embebido, se utilizó el módulo WebServer del ESP32. Cada operación del sistema (registro de usuarios, captura de huellas, creación de turnos, asignación de turnos, lectura de asistencias, configuración Wi-Fi, entre otras) fue mapeada a un endpoint específico mediante funciones manejadoras (handler functions), registradas mediante "server.on(ruta, metodo, handler)" para cada una de las rutas listadas anteriormente. Este mecanismo permite encapsular la lógica de cada componente del sistema y mantener una estructura modular del firmware.

Creación de vistas HTML embebidas

Las vistas fueron desarrolladas como archivos .html independientes almacenados en el sistema de archivos LittleFS del ESP32-CAM, conteniendo HTML, CSS y JavaScript autónomo, sin dependencias de CDN externas. El servidor web embebido las sirve mediante la función servirArchivo(), que lee el archivo solicitado desde LittleFS y lo envía al cliente. Esto garantiza que el ESP32-CAM sirva cada página incluso en modo Access Point sin conexión a internet.

Menú principal

La Figura 4.12 presenta la vista principal accesible en la ruta /. Desde esta interfaz el usuario puede visualizar el estado del dispositivo (online/offline, sensor activo, enrolado), registrar personas, ver asistencias, gestionar turnos, ver asignaciones, acceder a la configuración de red, sincronizar con el backend y limpiar datos del dispositivo. Con el avance de las iteraciones se agregaron a esta vista los indicadores de estado que se actualizan cada pocos segundos consultando al dispositivo, un bloque para mostrar el PIN de registro que permanece oculto hasta que el usuario decide revelarlo, una zona separada para las acciones de reinicio y borrado total de datos, y la opción de proteger la página con contraseña de administrador cuando se requiere restringir el acceso a esas acciones.

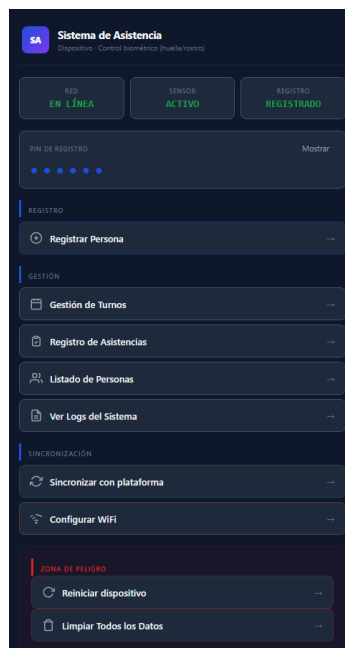


Figura 4.12: Menú principal del sistema web embebido.

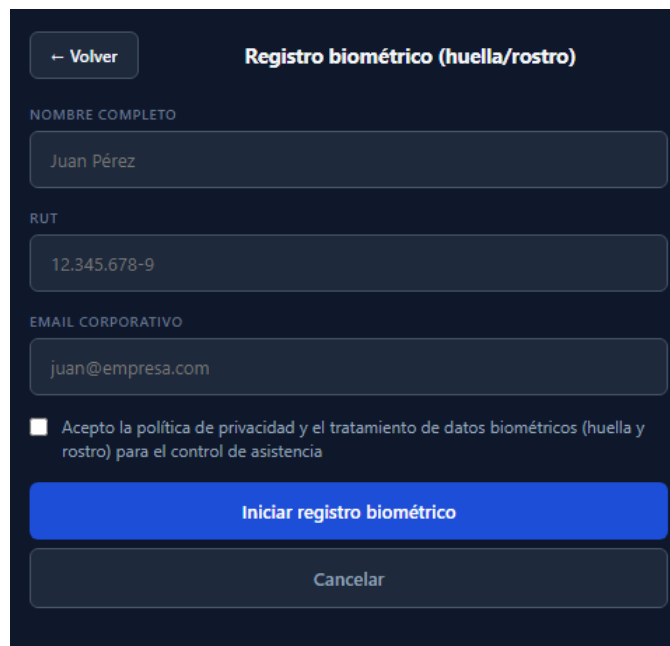
Vista de configuración Wi-Fi

Mediante la ruta `/wifi-setup`, el dispositivo proporciona un formulario para actualizar las credenciales de la red local (SSID, contraseña), la URL del backend, la dirección del broker MQTT y el PIN de enrolamiento. Estos valores se persisten en SPIFFS mediante el archivo `wifi.json`. La Figura 4.13 muestra esta interfaz. Por requerimientos de seguridad identificados en iteraciones posteriores, se agregó a esta vista la opción de forzar conexiones cifradas hacia el backend y el broker MQTT, además de un apartado donde se puede definir o cambiar la contraseña de administrador pidiendo la contraseña actual como confirmación, de modo que no cualquier persona con acceso a la red local pueda modificar la configuración del dispositivo.

Figura 4.13: Interfaz de configuración Wi-Fi.

Vista de registro de personas

La ruta `/register` despliega una vista que combina un formulario para ingresar el nombre del usuario, RUT y correo electrónico, y muestra indicadores visuales del estado del proceso de enrolamiento (huella, rostro). La Figura 4.14 lo muestra. Se incorporó a esta vista una casilla de consentimiento explícito para el tratamiento de datos biométricos, exigida por las consideraciones éticas y de privacidad descritas en el marco metodológico; mientras no se marque, el botón para iniciar el registro biométrico queda deshabilitado. El flujo se complementó además con un panel que informa el avance del proceso y con una pantalla de captura facial guiada, que muestra cuántas de las tres fotografías necesarias para el reconocimiento facial ya se han tomado.



← Volver **Registro biométrico (huella/rostro)**

NOMBRE COMPLETO

Juan Pérez

RUT

12.345.678-9

EMAIL CORPORATIVO

juan@empresa.com

☐ Acepto la política de privacidad y el tratamiento de datos biométricos (huella y rostro) para el control de asistencia

Iniciar registro biométrico

Cancelar

Figura 4.14: Vista para el registro de personas en el dispositivo.

Vista de asistencias

Para visualizar registros de asistencia tanto locales como sincronizados, se implementó la vista en la ruta `/asistencias`. Esta consulta archivos JSON almacenados en LittleFS. La Figura 4.15 lo ejemplifica. Cada registro se presenta como una tarjeta diferenciada por color según sea entrada o salida, con el nombre de la persona, la marca de tiempo y su identificador asociado. A la versión inicial, que solo listaba los registros, se le agregaron las opciones de editar o eliminar cada fila, pensadas para que el encargado de turno pueda corregir una marcación errónea, como una doble lectura biométrica, sin necesidad de acceder directamente al backend. También se sumó un contador de registros totales en la parte superior, útil para revisar de un vistazo cuántos datos hay almacenados localmente antes de sincronizar.

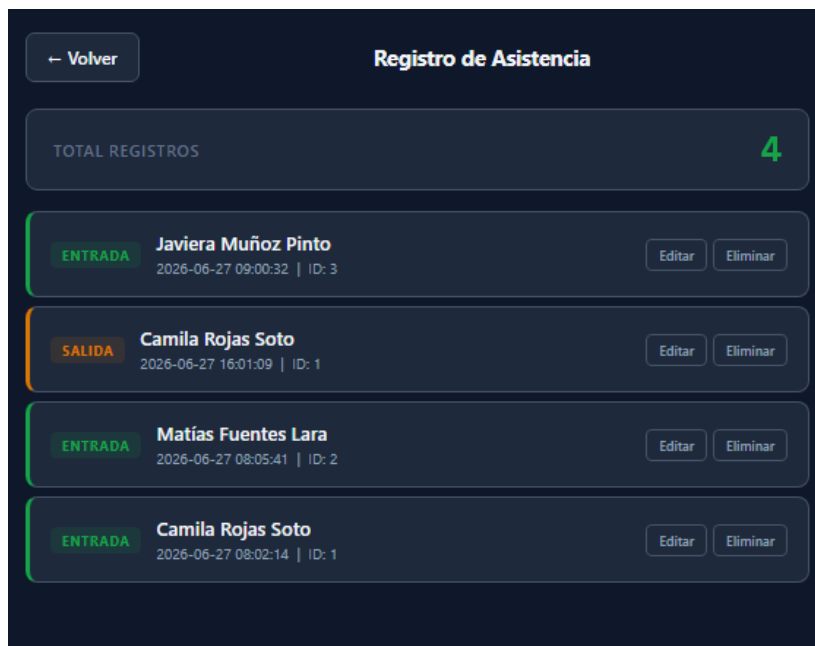


Figura 4.15: Vista de asistencias locales almacenadas en el dispositivo.

Vista de Personas registradas

Para visualizar las personas registradas, se implementó la vista en la ruta `/personas`. Esta consulta archivos JSON almacenados en LittleFS y permite editar nombre/correo, actualizar huella o rostro, y eliminar personas. La Figura 4.16 lo ejemplifica. Las opciones para actualizar la huella o el rostro, y para agregar fotos adicionales, se incorporaron después de notar en las pruebas de campo que volver a registrar a una persona desde cero, borrando y repitiendo todo el proceso, tomaba demasiado tiempo. Con estas opciones se puede corregir solo la huella o el rostro de alguien ya registrado, o sumar fotos para mejorar el reconocimiento facial, sin perder el resto de sus datos ni su historial de asistencia.

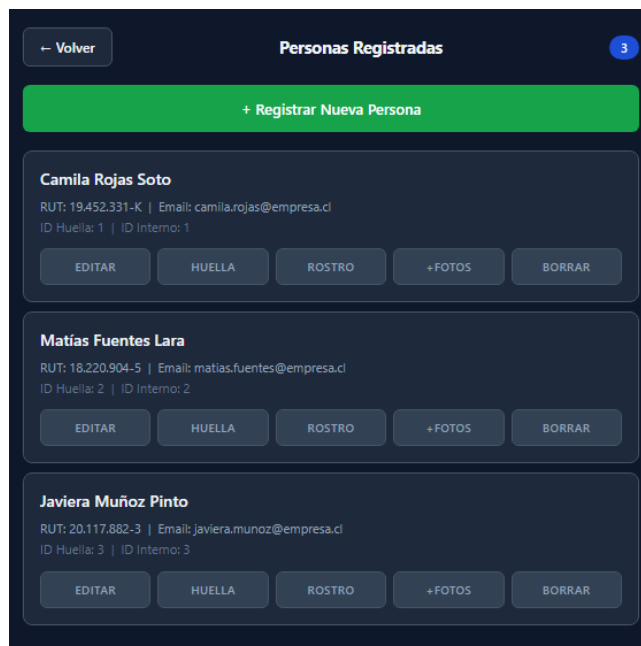


Figura 4.16: Vista de personas registradas en el dispositivo.

Vista de Crear Turnos

Para la creación de turnos y visualización de los existentes, se implementó la vista en la ruta `/gestion`, complementada con `/turnos`. Estas consultan archivos JSON almacenados en LittleFS. La Figura 4.17 lo ejemplifica. La vista `/gestion` funciona como panel resumen, con accesos directos a la gestión de turnos, a las asignaciones y a la sincronización manual con la plataforma. La vista `/turnos` concentra el formulario para crear un turno nuevo, con nombre, hora de inicio y fin, y un selector de días de la semana, junto con el listado editable de los turnos ya creados. Esta separación entre resumen y edición se introdujo para que las modificaciones no ocurran por accidente desde la pantalla que se usa con más frecuencia.

— Volver

Gestión de Turnos

CREAR NUEVO TURNO

NOMBRE DEL TURNO

Ej: Turno Mañana

INICIO 08:00 FIN 16:00

DÍAS DE LA SEMANA

L M X J V S D

Días: L, M, X, J, V

Crear Turno

TURNOS EXISTENTES

Turno Mañana
08:00 – 16:00 | ID: 11

Turno Tarde
16:00 – 00:00 | ID: 12

Turno Fin de Semana
09:00 – 18:00 | ID: 13

— Volver

Figura 4.17: Vista de gestión y creación de turnos en el dispositivo.

Vista de asignación de turnos

Para visualizar las asignaciones de turnos y realizar la acción de asignar un turno a una persona, se implementó la vista en la ruta `/asignaciones`. La Figura 4.18 lo ejemplifica. El formulario superior permite elegir una persona y un turno ya creados desde listas desplegables, evitando que el usuario tenga que recordar o escribir identificadores internos. El listado de asignaciones actuales muestra el nombre de la persona y del turno en lugar de solo sus identificadores, e incluye una opción para quitar una asignación, agregada después de notar que reasignar turnos por cambios de horario del personal era algo que ocurría seguido durante el uso del prototipo.

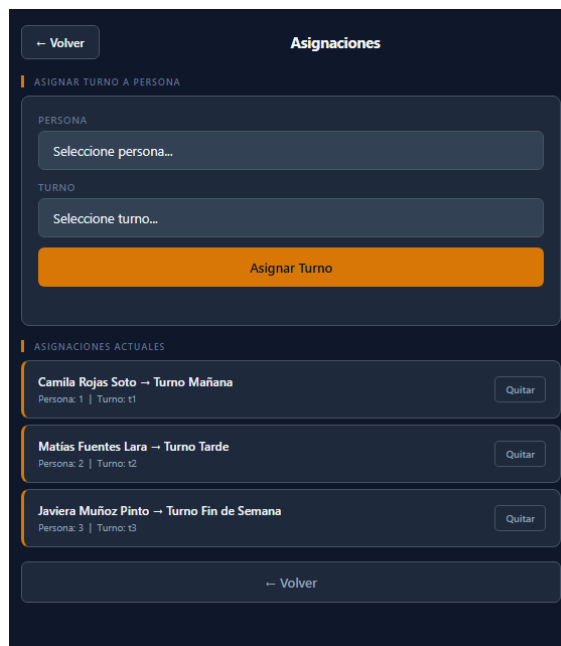


Figura 4.18: Vista de gestión de asignaciones en el dispositivo.

Pruebas

Para validar el funcionamiento del prototipo y comprobar la correcta integración entre los distintos módulos del sistema, se diseñaron pruebas de integración enfocadas exclusivamente en los componentes implementados durante esta iteración:

- **Prueba de captura de imágenes:** Verificar que la cámara OV2640 capture imágenes JPEG de forma continua y estable, sin corrupción de frames ni errores de inicialización del sensor. Se espera que la función `esp_camera_fb_get()` retorne buffers consistentes tras cada llamada.
- **Prueba de comunicación UART con AS608:** Verificar que el lector biométrico responda correctamente al comando de verificación de contraseña (`finger.verifyPassword()`) y que la comunicación serie a 57600 baudios se establezca sin pérdida de paquetes. Se espera confirmación de conexión en menos de 1 segundo tras el inicio.
- **Prueba de lectura biométrica:** Verificar que el AS608 pueda capturar una imagen dactilar (`finger.getImage()`), convertirla a plantilla (`finger.image2Tz()`) y almacenarla en un slot del sensor (`finger.storeModel()`). Se espera que el proceso completo tome menos de 2 segundos.
- **Prueba de acceso vía AP:** Verificar que al iniciar sin credenciales Wi-Fi configuradas, el ESP32-CAM active el Access Point ESP32-ASISTENCIA y

que las páginas HTML sean accesibles desde un navegador conectado a la IP 192.168.4.1. Se espera que todas las rutas definidas respondan con código HTTP 200.

- **Prueba de carga de vistas vía Wi-Fi externa:** Verificar que al conectar el dispositivo a una red Wi-Fi configurada, las mismas vistas se carguen correctamente a través de la IP asignada por el router, manteniendo estilos, scripts y funcionalidad.

Refinamientos posteriores

Durante el desarrollo de las iteraciones siguientes surgieron dos problemas asociados a la base construida en esta etapa, los cuales fueron corregidos durante el mismo ciclo de implementación.

El primero se presentó al combinar las funcionalidades de conectividad y enrolamiento: un dispositivo podía encontrarse conectado a internet sin contar aún con una persona enrolada, o bien tener una persona enrolada pero haber perdido la conexión de red. En ambos casos, el firmware intentaba ejecutar la sincronización o el reconocimiento facial de igual forma, lo que provocaba fallos sin generar un mensaje de error claro para el usuario. Para resolver esta situación, se implementó una función de validación que verifica simultáneamente ambas condiciones antes de permitir cualquier operación dependiente del backend, evitando así que el dispositivo ejecute procesos que se sabe resultarán en error.

El segundo ajuste estuvo relacionado con la calidad de las imágenes utilizadas para el reconocimiento facial. Las capturas estándar de la cámara se almacenaban con un nivel de compresión orientado a vistas previas rápidas, el cual resultaba insuficiente para generar un embedding facial de calidad adecuada, debido a la pérdida de detalle en los rasgos del rostro. Por este motivo, se diferenció el proceso de captura correspondiente al registro biométrico, incrementando temporalmente la resolución del sensor exclusivamente para la fotografía de referencia, y restableciéndola a su configuración estándar una vez completada la captura, sin afectar el resto de las imágenes procesadas por el sistema.

4.2.2 Iteración 2: Almacenamiento local y modo offline

La segunda iteración se centra en habilitar el funcionamiento autónomo del dispositivo sin conexión a internet, integrando un sistema de almacenamiento persistente basado en LittleFS (sucesor moderno de SPIFFS para ESP32) y los mecanismos de captura biométrica offline. Esta etapa materializa lo definido en el apartado 3.1.2 del capítulo anterior.

Análisis

Para el correcto funcionamiento offline del prototipo se identificaron los siguientes requerimientos:

- Almacenar localmente los datos de usuarios, turnos, asignaciones y asistencias en formato JSON.
- Permitir el registro de marcaciones aun cuando no exista conectividad externa, tanto por huella (offline total) como con validación de turnos locales.
- Garantizar persistencia de la información tras reinicios del ESP32-CAM o cortes de energía.
- Habilitar la gestión completa del sistema mediante una interfaz web embebida accesible vía Access Point.
- Mantener compatibilidad con la futura sincronización automática en modo online, marcando cada registro con un campo sincronizado (booleano) para su posterior envío al backend.

Para cumplir estos requisitos se seleccionó el sistema de archivos LittleFS y el formato JSON debido a su bajo peso, facilidad de manipulación mediante la librería ArduinoJson y compatibilidad con las capacidades del ESP32-CAM (4 MB de flash, de los cuales aproximadamente 1.5 MB están disponibles para LittleFS). Adicionalmente, se definió un archivo de configuración, `wifi.json`, destinado a almacenar credenciales Wi-Fi, URL del backend, dirección del broker MQTT y PIN de enrolamiento.

Diseño

Considerando la arquitectura establecida, se diseñaron las siguientes estructuras de almacenamiento en formato JSON:

- **personas.json**: Arreglo de objetos con campos `id`, `nombre`, `rut`, `email`, `huella_id`, `fecha_registro`, `sincronizado`.
- **turnos.json**: Arreglo de objetos con campos `id`, `backend_id`, `nombre`, `inicio`, `fin`, `dias`, `sincronizado`.
- **asignaciones.json**: Arreglo de objetos con campos `persona_id`, `turno_id`, `backend_id`, `fecha_asignacion`, `sincronizado`.
- **asistencias.json**: Arreglo de objetos con campos `persona_id`, `nombre`, `tipo` (entrada/salida), `metodo` (huella_offline, facial, etc.), `timestamp`, `sincronizado`.

- **wifi.json**: Objeto con campos ssid, pass, backend, mqtt, pin (para enrolamiento del dispositivo).
- **erp-config.json**: Arreglo con configuraciones de integraciones ERP, sincronizado desde el backend cuando hay conectividad.

El campo sincronizado (booleano) en cada entidad actúa como bandera para el proceso de sincronización diferida: cuando el dispositivo recupera conectividad, itera sobre los registros con sincronizado = false y los envía al backend. Una vez confirmada la recepción, se marca sincronizado = true.

Implementación

En esta iteración se integraron las librerías LittleFS, ArduinoJson, HardwareSerial y Adafruit_Fingerprint. Las dos primeras permiten gestionar archivos JSON alojados en la memoria flash del ESP32-CAM, mientras que las dos últimas habilitan la comunicación UART con el lector biométrico AS608 para el registro y verificación de huellas digitales.

Inicialización del sistema de archivos y persistencia

El primer componente de la implementación consiste en montar LittleFS mediante `LittleFS.begin(true)` y verificar la existencia de los archivos necesarios. En caso de no existir, el sistema los crea con una estructura inicial vacía (arreglo [] para las colecciones, objeto con valores por defecto para `wifi.json`), asegurando que el dispositivo pueda operar incluso tras un primer arranque sin configuración previa.

Se implementaron dos funciones genéricas para la manipulación de archivos JSON:

- `loadArray(const char* path, DynamicJsonDocument& doc)`: Abre un archivo, deserializa su contenido como `JsonArray` y lo retorna. Si el archivo no existe o está corrupto, retorna un arreglo vacío.
- `saveArray(const char* path, DynamicJsonDocument& doc)`: Serializa el documento JSON y lo escribe en el archivo correspondiente, sobrescribiendo el contenido anterior.

Estas funciones encapsulan toda la lógica de persistencia del sistema y son reutilizadas por todos los módulos que necesiten leer o escribir datos.

El tamaño de cada `DynamicJsonDocument` se dimensionó según el volumen esperado de datos de cada archivo, aunque varía según la función que lo utiliza: `asignaciones.json` utiliza típicamente 1024 bytes, `personas.json` y `turnos.json` se manejan con buffers de entre 2048 y 8192 bytes dependiendo del contexto (lectura local, sincronización o registro), y `asistencias.json` utiliza 32768 bytes. Adicionalmente,

en la función `procesarAsistencia()` se incorporó una validación sobre el retorno de `createNestedObject()`: si el objeto retornado es nulo (indicando desbordamiento del documento), se registra una advertencia en el log del sistema, permitiendo detectar tempranamente cuellos de botella de memoria sin comprometer la integridad del archivo JSON. El dimensionamiento de este buffer y la estrategia adoptada para sostener el volumen de registros exigido en los objetivos del proyecto se detallan en el apartado de refinamientos posteriores de esta iteración.

Registro de usuarios con huella digital

El registro de usuarios se implementó mediante un flujo que combina la captura biométrica gestionada por el lector AS608 con el almacenamiento en `personas.json`. El lector administra internamente modelos de huellas mediante *slots* numerados del 1 al 127. Por ello, antes del enrolamiento es necesario identificar un slot disponible dentro del sensor. La función `encontrarSlotLibre()` recorre el arreglo de personas, determina el ID de huella más alto registrado y retorna el siguiente disponible.

Una vez encontrado un slot, se ejecuta el proceso de enrolamiento en dos fases:

1. **Primera captura** (`ESTADO_ESPERANDO_HUELLA_REGISTRO`): El sistema espera que el usuario coloque el dedo. Al detectar una imagen válida, la convierte a plantilla en el buffer 1 y transiciona a `ESTADO_ESPERANDO_SOLTAR_DEDO`.
2. **Espera y segunda captura** (`ESTADO_REGISTRO_SEGUNDA_HUELLA`): Una vez que el sensor detecta que el dedo fue retirado (`FINGERPRINT_NOFINGER`), el sistema solicita una segunda captura del mismo dedo. Al obtenerla, la convierte a plantilla en el buffer 2, crea el modelo combinado (`finger.createModel()`) y lo almacena en el slot correspondiente (`finger.storeModel()`).

Luego del enrolamiento exitoso, se almacena un nuevo objeto dentro de `personas.json` con un identificador único (generado localmente con prefijo `local-` o provisto por el backend si hay conectividad), el nombre, RUT, correo electrónico y el slot asignado. La etapa de captura de huella aquí descrita corresponde a los primeros estados de la máquina de estados del registro biométrico completo, presentada más adelante en la Figura 4.21 (sección 4.3.4), una vez introducido el envío de la foto de referencia al backend por HTTP.

Registro de asistencia en modo offline

El registro de asistencia se realiza completamente en el dispositivo, sin necesidad de conexión externa. El flujo implementado consiste en:

1. Capturar una huella mediante el lector AS608 (`finger.getImage()`).
2. Convertir la imagen a plantilla (`finger.image2Tz()`) y ejecutar una búsqueda biométrica (`finger.fingerSearch()`) que retorna el `fingerID` del slot coincidente.
3. Identificar al usuario asociado mediante el campo `huella_id` en `personas.json` con la función `buscarPersonaPorHuella()`.
4. Verificar que el usuario cuente con un turno asignado consultando `asignaciones.json` mediante `turnoActivo()`. Si no tiene turno, el sistema rechaza la marcación.
5. Determinar el tipo de evento (entrada/salida) alternando automáticamente respecto al último registro de la persona en `asistencias.json`.
6. Generar un nuevo registro con `sincronizado = false` (salvo que se logre enviar al backend en ese momento).

El manejador `handleMarcarAsistencia()` expone esta funcionalidad a través de la interfaz web, permitiendo también marcaciones manuales. El escaneo automático continuo se controla mediante intervalos configurables: 2 segundos entre verificaciones de huella y 6 segundos entre verificaciones faciales.

Gestión de turnos y asignaciones

Además del registro de asistencias, el sistema permite crear turnos (`handleCreateTurn()`) y asignarlos a personas (`handleAssignTurn()`) desde la interfaz web. Cada operación persiste los datos en sus archivos JSON correspondientes y, si hay conectividad, intenta replicar la operación en el backend mediante HTTP POST.

Operación offline del dispositivo

Con el objetivo de asegurar autonomía total, el ESP32-CAM fue configurado para operar en dos modos:

- **Modo Wi-Fi STA:** Si hay credenciales configuradas en `wifi.json`, el dispositivo intenta conectarse a la red externa. Si la conexión falla reiteradamente (5 intentos), activa el modo AP como fallback sin deshabilitar el intento de reconexión.
- **Modo Access Point:** El punto de acceso utiliza el SSID ESP32-ASISTENCIA y contraseña `Asistencia2026`, permitiendo que un administrador se conecte directamente al dispositivo en la IP 192.168.4.1 para realizar todas las operaciones mediante la interfaz web embebida.

Pruebas

Se diseñaron las siguientes pruebas para validar el correcto funcionamiento del almacenamiento local y el modo offline:

- **Prueba de persistencia tras reinicio:** Registrar usuarios, turnos, asignaciones y asistencias, luego reiniciar el ESP32-CAM (por corte de energía o comando software). Verificar que todos los archivos JSON conserven su contenido íntegro y que el sistema pueda leerlos correctamente tras el reinicio.
- **Prueba de lectura y deserialización:** Abrir cada archivo JSON generado y verificar que ArduinoJson pueda deserializar su contenido sin errores ni pérdida de campos. El tamaño de cada DynamicJsonDocument se configuró según la capacidad esperada del archivo: 2048 bytes para personas.json y turnos.json, 1024 bytes para asignaciones.json y 32768 bytes para asistencias.json. Se incluyó además una validación post-inserción (mediante `isNull()`) en la función `procesarAsistencia()` para detectar y registrar en logs cualquier desbordamiento del documento al crear nuevos registros de asistencia.
- **Prueba de escritura concurrente:** Realizar múltiples operaciones de escritura (registro de persona + asignación de turno + marcación de asistencia) en rápida sucesión sin reiniciar el sistema. Verificar que no se produzcan corrupciones de archivos ni pérdida de datos.
- **Prueba de marcación offline por huella:** Desconectar el dispositivo de toda red externa, registrar un usuario con huella y turno, y efectuar una marcación. Verificar que el registro aparezca en asistencias.json con `sincronizado = false`.
- **Prueba de alternancia entrada/salida:** Realizar dos marcaciones consecutivas para un mismo usuario. Verificar que el sistema registre correctamente “entrada” seguido de “salida”.
- **Prueba de validación de turno offline:** Intentar marcar asistencia para un usuario sin turno asignado. Verificar que el sistema rechace la marcación con el mensaje “Sin turno asignado”.
- **Prueba de funcionamiento del AP:** Desconectar el ESP32-CAM de cualquier red, verificar que el AP ESP32-ASISTENCIA se active automáticamente y que todas las operaciones de gestión (registro, turnos, asignaciones, asistencias) funcionen correctamente desde un navegador conectado a la IP 192.168.4.1.

Refinamientos posteriores

Durante la revisión de esta iteración se identificó una limitación que comprometía directamente el objetivo específico de sostener al menos 1.000 registros de asistencia almacenados de forma segura en LittleFS. El diseño original cargaba el archivo `asistencias.json` completo en un único `DynamicJsonDocument` cada vez que se necesitaba leer o escribir un registro, de modo que la capacidad práctica del sistema no estaba acotada por el espacio disponible en la memoria flash —de varios megabytes y suficiente para almacenar miles de registros en formato JSON— sino por el tamaño del buffer reservado en RAM. Con el buffer original de 8192 bytes la capacidad estimada era de apenas unas decenas de registros, muy por debajo de lo exigido en los objetivos del proyecto.

Adicionalmente, se observó que los registros ya sincronizados con el backend no se eliminaban del archivo local: la función `sincronizarAsistencias()` únicamente actualizaba el campo `sincronizado` a `true`, por lo que el arreglo en memoria crecía de forma indefinida durante toda la vida útil del dispositivo, incluso operando con conectividad constante.

Para corregir esta situación se aplicaron dos ajustes complementarios. Primero, se incrementó el tamaño del `DynamicJsonDocument` utilizado para `asistencias.json` de 8192 a 32768 bytes, ampliando el margen disponible para los registros pendientes de sincronización. Segundo, y de mayor relevancia, se modificó `sincronizarAsistencias()` para que, una vez confirmada la recepción exitosa por parte del backend, los registros marcados como sincronizados sean eliminados del archivo local en lugar de conservarse indefinidamente. De esta forma, el buffer en RAM solo necesita reservar espacio para los registros generados durante un período sin conectividad, mientras que el historial completo de marcaciones queda respaldado de manera permanente en el backend. Esta estrategia permite que el dispositivo sostenga miles de marcaciones acumuladas a lo largo de su operación sin acercarse al límite de memoria, cumpliendo así el volumen exigido por el objetivo específico correspondiente, condicionado a que el dispositivo sincronice periódicamente con el backend.

4.2.3 Iteración 3: Backend, base de datos y comunicación

El propósito de esta iteración consiste en implementar un backend centralizado que permita procesar la información enviada por el ESP32-CAM, administrar la base de datos, gestionar registros de trabajadores y turnos, y habilitar la comunicación bidireccional mediante MQTT y HTTP. Esta etapa corresponde a la construcción del servidor del sistema, incorporando funcionalidades que serán extendidas en iteraciones posteriores.

Análisis

En esta iteración se realiza el estudio de los mecanismos de comunicación necesarios para establecer el flujo de datos entre el dispositivo ESP32-CAM y el backend del sistema, considerando tanto la operación en línea como escenarios donde la conectividad es limitada o intermitente.

En primer lugar, se evaluaron los protocolos HTTP y MQTT como alternativas principales para el envío de datos. La decisión final fue asignar a cada protocolo el rol donde ofrece mayores ventajas:

- **HTTP** se utiliza tanto para el registro facial durante el enrolamiento como para la identificación facial durante la marcación, enviando el JPEG crudo como application/octet-stream (más liviano que codificarlo en Base64) a los endpoints correspondientes. También se usa para todas las operaciones REST (CRUD de personas, turnos, asignaciones, asistencias), la sincronización diferida, la consulta de configuración ERP y el enrolamiento del dispositivo.
- **MQTT** (con transporte WebSockets seguros wss://) se reserva para los eventos de bajo volumen que no necesitan transportar una imagen: el resultado del enrolamiento de huella, los heartbeats periódicos, los pings de verificación de conectividad y el aviso de desconexión abrupta (Last Will Testament).

En una primera versión del proyecto se había evaluado enviar también las fotos de registro facial por MQTT, aprovechando que el broker entrega confirmación de recepción y permite no bloquear al dispositivo mientras el backend procesa la imagen. En la práctica esto resultó innecesario: terminaba usando dos protocolos distintos para resolver el mismo intercambio entre dispositivo y servidor, una foto que sube y una respuesta que confirma si fue aceptada. Por eso el registro facial se simplificó a una petición HTTP corriente, igual a la que ya se usaba para la identificación: el dispositivo sube la foto, espera la respuesta y sabe de inmediato si quedó aceptada o si debe repetir la captura.

Asimismo, se analizó la conectividad Wi-Fi como medio de enlace principal del dispositivo. En escenarios con conexión estable, el ESP32-CAM envía imágenes y consultas por HTTP al backend, y eventos de estado por MQTT al broker; mientras que, en situaciones de baja o nula conectividad, el dispositivo almacena registros localmente (Iteración 2) y los reenvía cuando la red se restablece. Se definieron mecanismos de reconexión progresiva con backoff exponencial (3, 6, 9, 12 y 15 segundos entre intentos), manejo de desconexiones mediante el evento ARDUINO_EVENT_WIFI_STA_DISCONNECTED y un watchdog que intenta restaurar la conectividad periódicamente.

Finalmente, se evaluó la compatibilidad del flujo de datos con la arquitectura general definida en el apartado 4.1.1, determinando que esta iteración corresponde a la habilitación de la capa de comunicación que enlaza el dispositivo con el backend.

Diseño

El diseño de esta iteración se centra en la definición detallada de los mecanismos de comunicación y conectividad, así como en la arquitectura de la base de datos que soportará todas las funcionalidades del sistema.

El backend se organiza de forma modular mediante blueprints de Flask, cada uno responsable de un dominio específico del sistema: personas, turnos, asignaciones, asistencias, reconocimiento facial, dispositivos, logs, integraciones ERP y autenticación. Esta separación facilita el mantenimiento y la escalabilidad del código.

La API REST se diseña para soportar las funcionalidades de gestión operativa del sistema. Los endpoints definidos para esta iteración incluyen:

- GET /api/personas: Obtiene el listado de personas registradas (con filtro por roles y empresa en iteraciones futuras).
- POST /api/personas: Registra una nueva persona (nombre, RUT, email, huella_id).
- GET /api/turnos: Devuelve todos los turnos almacenados.
- POST /api/turnos: Permite crear un nuevo turno indicando nombre, hora de inicio, hora de término y días.
- DELETE /api/turnos/<id>: Elimina un turno específico.
- GET /api/asignaciones: Lista todas las asignaciones activas entre personas y turnos.
- POST /api/asignaciones: Asigna un turno a una persona.
- DELETE /api/asignaciones/<id>: Elimina una asignación.
- POST /api/asistencias: Registra una asistencia enviada desde el dispositivo.
- POST /api/asistencias/sync: Recibe un lote de asistencias acumuladas en modo offline y las inserta con deduplicación.

Todos estos endpoints operan sobre una base de datos relacional PostgreSQL. El diseño del esquema incluye las siguientes tablas:

- empresas: Entidad multi-tenant con nombre, RUT empresa, email de contacto, teléfono y dirección.
- dispositivos: Registro de cada ESP32-CAM enrolado, con MAC address, IP local, estado (activo/inactivo), último heartbeat y código de enrolamiento (PIN de 8 caracteres).
- usuarios_web: Usuarios del panel web backend con autenticación por email y contraseña (hash bcrypt).
- usuario_empresa: Tabla de relación muchos-a-muchos entre usuarios y empresas, con rol (admin, empleador, trabajador).
- personas: Trabajadores registrados con nombre, RUT (único), email, huella_id, encoding facial (cifrado) y empresa asociada.
- turnos: Jornadas laborales con nombre, hora de inicio, hora de fin, días y empresa asociada.
- asignaciones: Relación entre personas y turnos, con vigencia.
- asistencias: Registros de marcación con persona, dispositivo, tipo (entrada/salida), método, timestamp, origen y estado de sincronización.
- sincronizacion_log: Bitácora de procesos de sincronización con conteo de registros enviados y procesados correctamente.
- integraciones_erp: Configuración de webhooks para integración con sistemas ERP externos (nombre, tipo, URL, headers, field mapping, envío automático).
- consentimientos: Registro de aceptación de política de privacidad biométrica por persona.
- logs_biometricos: Auditoría de operaciones biométricas (registro, verificación, identificación) con resultado, timestamp e IP de origen.
- eliminaciones_biometricas: Registro histórico de datos biométricos eliminados (embeddings anteriores, fotos) para cumplimiento normativo y trazabilidad.
- encodings_faciales: Almacena múltiples embeddings faciales cifrados por persona (FOREIGN KEY con eliminación en cascada), permitiendo que un trabajador tenga varias fotos de referencia con distintos ángulos e iluminación para mejorar la precisión del reconocimiento.

La Figura 4.19 presenta el diagrama entidad-relación correspondiente. En él se visualizan las 14 tablas que componen el esquema, organizadas en torno a tres dominios principales: gestión de personas y turnos (personas, turnos, asignaciones), registro biométrico y de asistencia (asistencias, encodings_faciales, logs_biometricos, consentimientos, eliminaciones_biometricas), y administración del sistema (empresas, dispositivos, usuarios_web, usuario_empresa, integraciones_erp, sincronizacion_log). Las flechas indican las relaciones de clave foránea entre tablas, destacando cómo la entidad empresas actúa como raíz del modelo multi-tenant, la tabla personas como eje central que conecta la información laboral con los datos biométricos, y las tablas asistencias y sincronizacion_log como destino final del flujo de datos operacional.

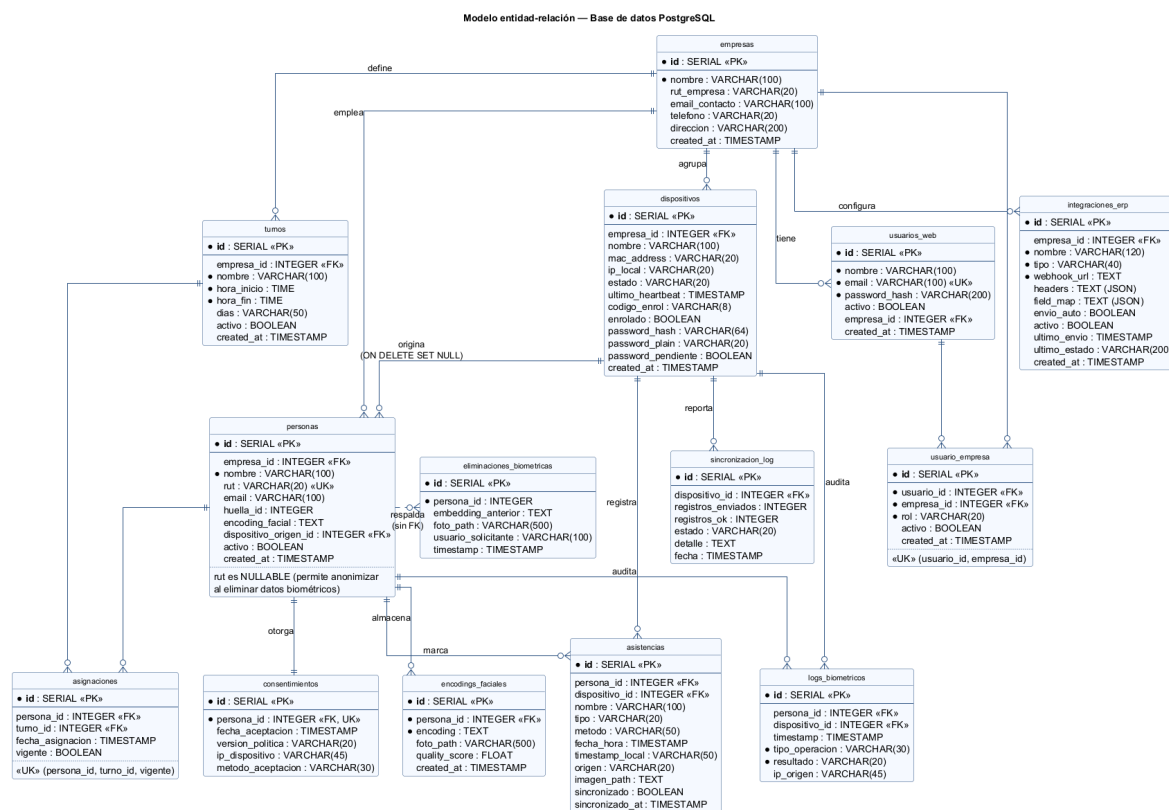


Figura 4.19: Diagrama entidad-relación de la base de datos (14 tablas).

Las imágenes faciales, tanto las de registro como las de identificación durante la marcación, viajan del ESP32-CAM al backend por HTTP, como un JPEG crudo en formato application/octet-stream dirigido al endpoint correspondiente (POST /api/facial/registrar o POST /api/facial/identificar). El backend responde en la misma petición indicando si la foto fue aceptada, de modo que el dispositivo no necesita esperar un mensaje aparte para saber el resultado.

MQTT queda reservado para los eventos de estado del dispositivo, que son livianos y no requieren transportar una imagen. Se definen los tópicos esp32/heartbeat/<MAC>

para monitorizar la actividad de los dispositivos y `esp32/lwt/<MAC>` para detectar desconexiones mediante el mecanismo de Last Will Testament (LWT). Para la detección activa de desconexión, se implementó el tópico `esp32/ping/<MAC>`: el backend publica un ping cada 30 segundos; si el ESP32-CAM no responde con un heartbeat dentro de 60 segundos, se marca como inactivo. La Figura 4.20 resume la topología de publicación y suscripción.

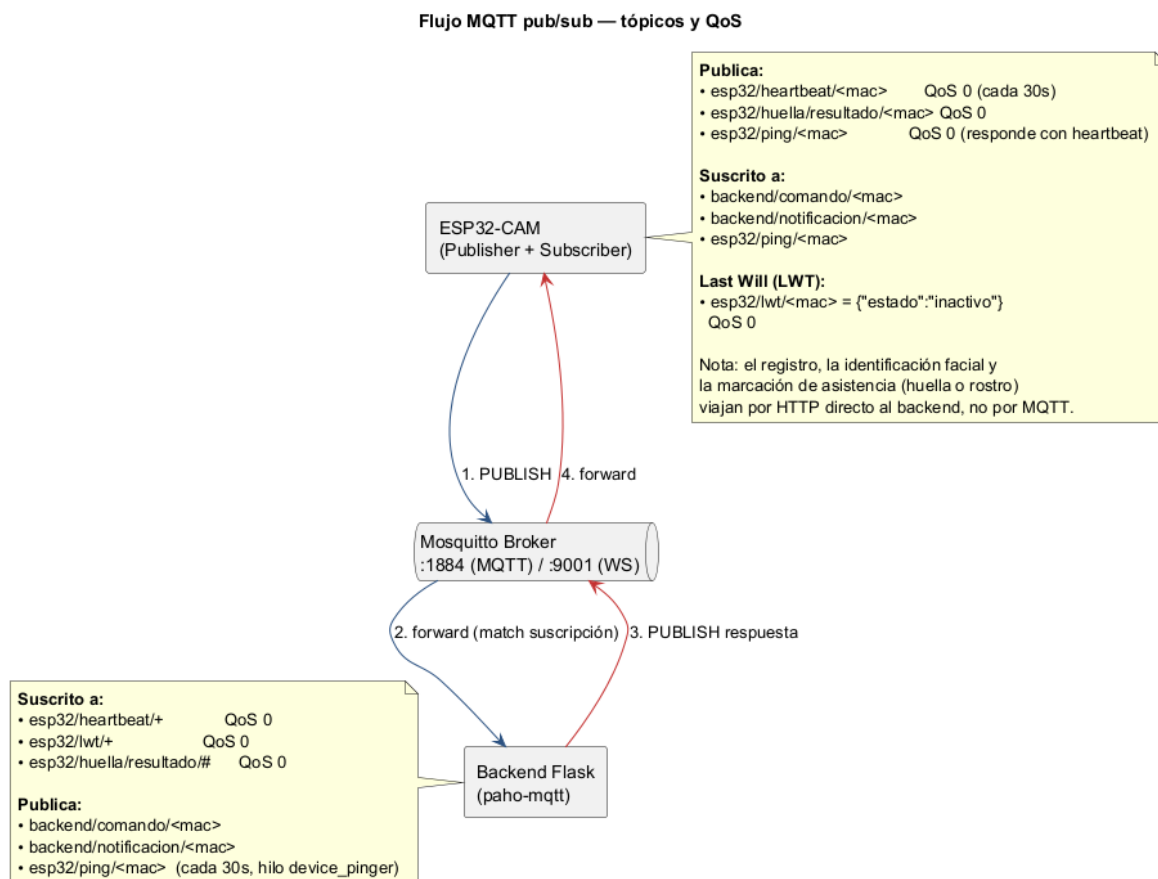


Figura 4.20: Flujo MQTT pub/sub: tópicos, direcciones y QoS entre el ESP32-CAM, el broker Mosquitto y el backend.

Implementación

La implementación de la Iteración 3 se centró en habilitar la comunicación entre el dispositivo ESP32-CAM y el backend desarrollado en Python con Flask [47], así como en construir la base de datos PostgreSQL, accedida mediante el adaptador `psycopg2` [53], que soporta todo el sistema.

Inicialización de la base de datos

Se implementó el módulo `database.py` con una función `init_db()` que crea todas las tablas del sistema utilizando sentencias `CREATE TABLE IF NOT EXISTS`, asegurando

idempotencia en los despliegues. Adicionalmente, se aplican sentencias ALTER TABLE ADD COLUMN IF NOT EXISTS para garantizar migraciones no destructivas. La función incluye la creación de índices sobre columnas frecuentemente consultadas (persona_id, dispositivo_id, fecha_hora, empresa_id) y la inserción de datos semilla (empresa por defecto, usuario administrador con contraseña hasheada).

Conexión Wi-Fi del ESP32-CAM y gestión de conectividad

Para permitir la operación en red, el dispositivo implementa un mecanismo de conexión automática a Wi-Fi mediante la función tryConnectWiFi(). Esta función lee las credenciales desde wifi.json, configura el modo STA y deshabilita el ahorro de energía Wi-Fi (WIFI_PS_NONE) para mantener una conexión estable. La verificación continua de conectividad se realiza en verificarConexionWiFi(), que implementa un mecanismo de reintentos con backoff progresivo: tras detectar una desconexión, espera un tiempo creciente antes de reintentar (3s, 6s, 9s, 12s, 15s). Si tras 5 intentos fallidos no se logra reconectar, el dispositivo activa el Access Point como fallback.

El evento ARDUINO_EVENT_WIFI_STA_DISCONNECTED queda registrado con su código de razón y un contador de desconexiones, información expuesta a través del endpoint /wifi-diag para diagnóstico remoto.

Envío de imágenes faciales por HTTP: registro e identificación

Tanto el registro como la identificación facial envían la imagen al backend de la misma manera: como JPEG crudo, sin codificarla en Base64, lo que evita el costo de procesamiento y el aumento de tamaño que implica esa codificación. Durante el registro, cuando el usuario presiona el botón de captura en la interfaz del dispositivo, el firmware toma la foto con la cámara en su calidad más alta y la sube de inmediato al backend mediante una petición POST, identificando a la persona con su RUT en una cabecera de la solicitud. El backend procesa la imagen y responde en esa misma petición si quedó aceptada o si hay que repetir la captura, lo que el dispositivo refleja en el contador de fotos de la pantalla de registro.

Durante la identificación (la consulta que ocurre al momento de marcar asistencia), el flujo es prácticamente el mismo a nivel de transporte: la imagen se captura y se envía por HTTP al endpoint de identificación, y el backend responde de forma síncrona indicando a qué persona corresponde el rostro, o un error si no encuentra coincidencia. Usar el mismo mecanismo síncrono para ambos casos simplificó el firmware, que ya no necesita dos rutas distintas para llegar al backend según el tipo de operación.

La Figura 4.21 ilustra la máquina de estados completa del proceso de enrolamiento biométrico en el ESP32-CAM, integrando la captura de huella digital (introducida en la Iteración 2) con el envío de la foto de referencia al backend por HTTP descrito en

esta sección.

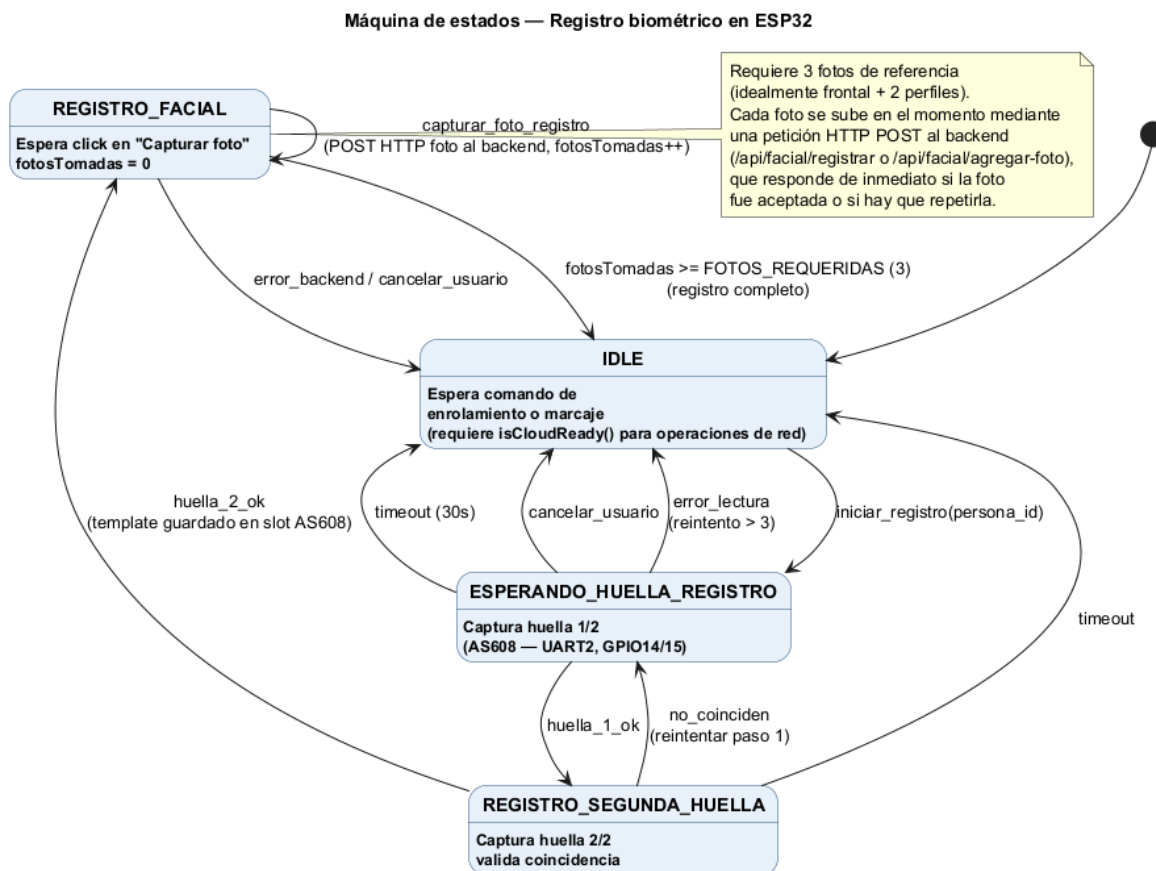


Figura 4.21: Máquina de estados del registro biométrico en el ESP32-CAM.

Recepción de imágenes en el backend

El backend recibe las fotos de registro e identificación a través de los endpoints REST del módulo `routes/facial.py`, descritos en el diseño de esta sección (ver Anexo B para el detalle completo de endpoints REST y tópicos MQTT). Al llegar una foto de registro, el backend verifica que exista consentimiento biométrico para esa persona, guarda la imagen, extrae el embedding facial y lo cifra antes de almacenarlo; si algo falla en el camino (consentimiento faltante, imagen borrosa, rostro duplicado), responde con el error correspondiente en lugar de dejar la solicitud sin respuesta.

Aparte de las imágenes, el backend mantiene un cliente MQTT para los eventos de estado del dispositivo: procesa los heartbeats, actualizando la IP y la actividad del dispositivo en la base de datos, y los mensajes de desconexión abrupta (LWT), marcándolo como inactivo. También corre en segundo plano una tarea que envía un ping cada 30 segundos a cada dispositivo enrolado, como verificación activa de que sigue en línea. Cada cambio de estado se transmite a los clientes web en tiempo real mediante streaming SSE, para que el panel se actualice sin que el administrador tenga

que recargar la página.

Mantenimiento de la conexión MQTT en el ESP32-CAM

El dispositivo mantiene, en paralelo a sus conexiones HTTP, un cliente MQTT que se conecta al broker si hay uno configurado y el dispositivo tiene red. Soporta URLs con esquema mqtt://, ws:// o wss:// (WebSockets seguros con TLS). Al conectarse, registra un mensaje de último aviso (LWT) para que el backend se entere si el dispositivo se cae de forma abrupta, y queda suscrito al tópico de ping para responder de inmediato cuando el backend pregunta si sigue activo. Este canal MQTT se usa únicamente para esta señalización de estado: ni el registro ni la identificación facial dependen de él.

Envío de datos mediante HTTP desde el ESP32-CAM

Se implementó la capacidad del ESP32-CAM para comunicarse con la API REST del backend mediante solicitudes HTTP usando la librería HTTPClient. La función auxiliar beginHttp() configura cada petición con timeout de 10 segundos y el header X-Device-MAC para que el backend pueda identificar al dispositivo emisor. Esta función es utilizada por todos los handlers que requieren comunicación con el backend:

- sincronizarPersonasDesdeBackend(): Descarga la lista completa de personas desde GET /api/personas y la persiste en personas.json.
- sincronizarTurnosDesdeBackend(): Descarga turnos desde GET /api/turnos.
- sincronizarAsignacionesDesdeBackend(): Descarga asignaciones desde GET /api/asignaciones.
- crearTurnoEnBackend(): Crea un turno en el backend vía POST /api/turnos.
- postAsistenciaEnBackend(): Envía una asistencia individual vía POST /api/asistencias.
- sincronizarAsistencias(): Envía un lote de asistencias pendientes vía POST /api/asistencias/sync.

Implementación de la API REST del backend

El backend se implementó con Flask, organizando las rutas en blueprints independientes por dominio:

- personas_bp (routes/personas.py): CRUD de personas con validación de email, unicidad de RUT y manejo de empresa.
- turnos_bp (routes/turnos.py): CRUD de turnos con filtro por empresa.

- `asignaciones_bp` (`routes/asignaciones.py`): CRUD de asignaciones con verificación de pertenencia a empresa.
- `asistencias_bp` (`routes/asistencias.py`): Registro de asistencias individuales y sincronización por lotes.
- `facial_bp` (`routes/facial.py`): Endpoints de reconocimiento facial (registro, verificación, identificación 1:N).
- `dispositivos_bp` (`routes/dispositivos.py`): Gestión y verificación de dispositivos.
- `logs_bp` (`routes/logs.py`): Consulta de logs de sincronización.
- `erp_bp` (`routes/erp.py`): CRUD de integraciones ERP y envío de datos.
- `auth_bp` (`routes/auth.py`): Autenticación de usuarios del panel web.

Cada blueprint se registra en la aplicación Flask principal (`app.py`), junto con la configuración de CORS para permitir peticiones desde cualquier origen. El backend se ejecuta en el puerto 5000 con modo debug activado y recarga automática deshabilitada para evitar conflictos con el cliente MQTT.

Infraestructura con Docker Compose

Para garantizar un entorno reproducible, se definió un archivo `docker-compose.yml` que levanta el broker Mosquitto (imagen `eclipse-mosquitto:latest`) con el puerto 1884 (externo, mapeado al 1883 interno) para MQTT nativo y el puerto 9001 para WebSockets expuestos, volúmenes para configuración, datos y logs persistentes, y una red bridge dedicada. El backend Flask y la base de datos PostgreSQL pueden ejecutarse en el host o en contenedores adicionales según la configuración de despliegue. La Figura 4.22 presenta el diagrama de despliegue completo.

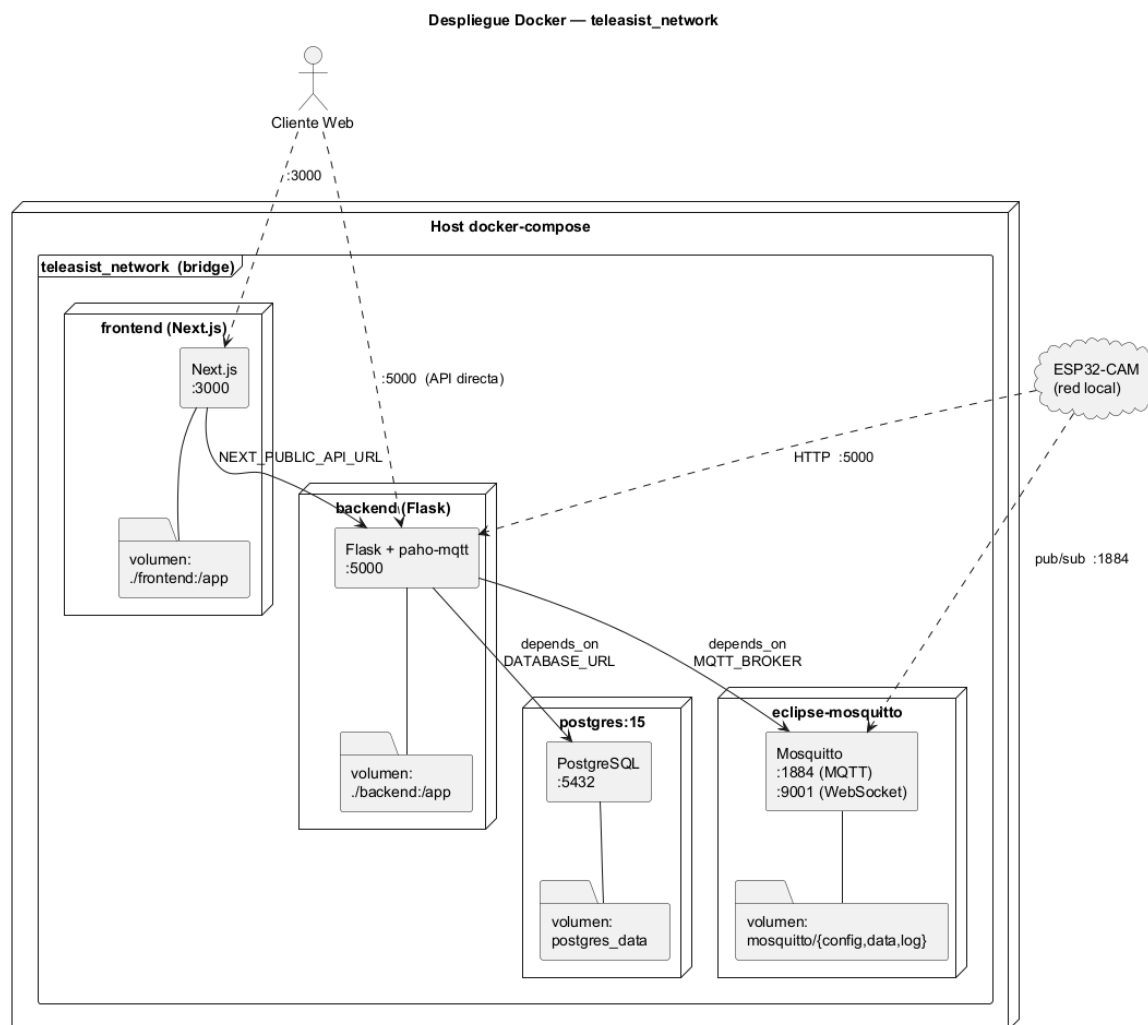


Figura 4.22: Arquitectura de despliegue Docker: servicios, puertos, volúmenes y dependencias en la red.

Pruebas

Se diseñaron las siguientes pruebas para validar la comunicación y el backend:

- **Prueba de conexión Wi-Fi y reconexión:** Configurar credenciales válidas en el ESP32-CAM, verificar que se conecte correctamente y obtenga una IP. Luego, apagar el router, verificar que el dispositivo detecte la desconexión mediante el evento de sistema y active el mecanismo de reintentos. Finalmente, encender el router y verificar que el dispositivo reconecte automáticamente en menos de 30 segundos.
- **Prueba de envío de imagen facial por HTTP:** Capturar una imagen desde el ESP32-CAM y enviarla al endpoint de registro mediante POST. Verificar que

el backend reciba la imagen, la guarde en disco y responda en la misma petición indicando si la foto fue aceptada.

- **Prueba de heartbeat y LWT:** Verificar que, estando el ESP32-CAM conectado, el backend reciba heartbeats periódicos (cada 30 segundos) y actualice el campo `ultimo_heartbeat` en la tabla `dispositivos`. Desconectar el dispositivo abruptamente y verificar que el mensaje LWT se publique y el backend marque el dispositivo como inactivo en menos de 10 segundos.
- **Prueba de ping/pong activo:** Conectar el ESP32-CAM al broker MQTT. Verificar que el backend publique `esp32/ping/<MAC>` periódicamente (cada 30 segundos) y que el ESP32-CAM responda con un heartbeat que contenga el campo `"pong": true`. Detener la conectividad del ESP32-CAM y verificar que, tras 60 segundos sin respuesta, el backend marque el dispositivo como inactivo.
- **Prueba de recepción de datos en backend vía HTTP:** Enviar solicitudes POST a `/api/personas`, `/api/turnos`, `/api/asignaciones` y `/api/asistencias` desde el ESP32-CAM. Verificar que cada solicitud sea procesada correctamente por el backend (código HTTP 200/201) y que los datos queden persistidos en PostgreSQL.
- **Prueba de sincronización por lotes:** Generar 5 marcaciones offline en el ESP32-CAM, conectar el dispositivo a la red y ejecutar la sincronización. Verificar que los 5 registros lleguen al backend vía POST `/api/asistencias/sync`, que se inserten correctamente y que el dispositivo marque cada registro como sincronizado `= true`.
- **Prueba de inicialización de base de datos:** Ejecutar `init_db()` sobre una base de datos vacía. Verificar que se creen las 14 tablas, los índices y los datos semilla (empresa por defecto y usuario administrador) sin errores.
- **Prueba de latencia de transmisión:** Medir el tiempo total desde que el ESP32-CAM envía una imagen por HTTP hasta que recibe la respuesta de confirmación del backend. Se espera un tiempo menor a 5 segundos en condiciones de red local.

Nuevos tópicos MQTT para control y notificación

En iteraciones posteriores se agregaron los siguientes tópicos al ecosistema MQTT:

- `backend/comando/<MAC>`: Permite al backend enviar comandos remotos al ESP32-CAM. La función `enviar_comando_dispositivo()` en `mqtt_handler.py` publica un JSON con el campo `comando` en este tópico.

- `backend/notificacion/<MAC>`: Utilizado por el módulo `eventos_mqtt.py` para notificar a los dispositivos que deben sincronizar sus datos. Cada vez que se crea, actualiza o elimina una persona, turno o asignación, la función `notificar_sincronizacion()` publica un mensaje indicando el tipo de entidad y la acción realizada.
- `esp32/huella/resultado/<huella_id>`: Tópico en el que el ESP32-CAM publica el resultado del enrolamiento de una huella. El backend se suscribe a `esp32/huella/resultado/#` y procesa la respuesta, actualizando la persona correspondiente y notificando a través de SSE.

Módulo `eventos_mqtt.py`

Se implementó el módulo `eventos_mqtt.py` con la función `notificar_sincronizacion(empresa_id, tipo, accion, registro_id)`. Esta función consulta los dispositivos enrolados de la empresa y publica un mensaje en `backend/notificacion/<MAC>` por cada uno, permitiendo que los ESP32-CAM reciban notificaciones en tiempo real sobre cambios en personas, turnos y asignaciones. Es invocada desde `routes/personas.py`, `routes/turnos.py` y `routes/asignaciones.py` tras cada operación de creación, actualización o eliminación.

Streaming SSE de huellas

Para mantener actualizados los clientes web durante el enrolamiento de huellas, se implementó el endpoint `GET /sse/huellas` en `app.py`. La función `broadcast_huella_update()` difunde los resultados del enrolamiento (recibidos por MQTT en `esp32/huella/resultado/#`) a todos los clientes conectados al SSE, permitiendo que el panel web muestre en tiempo real el progreso del registro biométrico.

Refinamientos posteriores

- **Decorador `@requiere_dispositivo_enrolado`**: definido en `routes/auth.py`, valida que el dispositivo asociado a la petición (`request.dispositivo_id`) exista en la tabla `dispositivos` con `enrolado = TRUE`, devolviendo 403 en caso contrario. Se aplica sobre endpoints biométricos sensibles como `/api/facial/registrar`, evitando que un dispositivo no autorizado opere sobre datos biométricos.
- **Integridad referencial con `ON DELETE SET NULL` y `RUT nullable`**: la columna `dispositivo_origen_id` de la tabla `personas` (y `dispositivo_id` de `asistencias`) se definió con `REFERENCES dispositivos(id) ON DELETE SET NULL`, de modo que eliminar un dispositivo no elimina en cascada las personas ni asistencias asociadas, sino que conserva el historial con la referencia en `NULL`. De

forma análoga, la columna rut de personas se migró a nullable (ALTER TABLE personas ALTER COLUMN rut DROP NOT NULL) para permitir el borrado de datos sensibles de una persona sin perder su historial de asistencias.

4.2.4 Iteración 4: Reconocimiento facial, anti-spoofing y cifrado biométrico

Esta iteración incorpora el procesamiento biométrico en el backend: reconocimiento facial con DeepFace, validación anti-spoofing y cifrado de embeddings faciales. El sistema pasa de solo transmitir imágenes a realizar verificación e identificación biométrica.

Análisis

Para incorporar el reconocimiento facial al sistema, se evaluaron diversas librerías y modelos de deep learning. Se seleccionó DeepFace por las siguientes razones:

- Soporta múltiples modelos de reconocimiento facial (Facenet, VGG-Face, ArcFace, etc.) y detectores (RetinaFace, MTCNN, OpenCV, etc.).
- Incluye funcionalidad de anti-spoofing integrada, capaz de distinguir entre un rostro real y una fotografía o pantalla, analizando textura, profundidad y reflectancia de la piel.
- Su API en Python es sencilla de integrar con Flask y compatible con las imágenes JPEG capturadas por la cámara OV2640 del ESP32-CAM.
- Facenet genera embeddings de 128 dimensiones, optimizando el almacenamiento y la velocidad de comparación.

Se identificaron los siguientes requisitos funcionales para esta iteración:

- **Registro facial:** Recibir una imagen del ESP32-CAM, extraer el embedding facial, validar que el rostro no esté duplicado en la base de datos (detección de identidades repetidas) y almacenar el embedding cifrado.
- **Identificación 1:N:** Dada una imagen capturada en vivo, comparar su embedding contra todos los embeddings almacenados y retornar la identidad con mayor coincidencia, si supera el umbral de similitud.
- **Verificación 1:1:** Dada una imagen y un rut, verificar si la imagen corresponde a la persona declarada.

- **Anti-spoofing:** Detectar intentos de suplantación mediante fotografías impresas, pantallas de dispositivos o máscaras, rechazando la captura si no se detecta un rostro real.
- **Cifrado de embeddings:** Los vectores de características faciales (embeddings) son datos biométricos sensibles. Para protegerlos, se deben cifrar antes de persistirlos en la base de datos, utilizando una clave derivada de una variable de entorno.
- **Consentimiento biométrico:** Todo registro facial debe estar precedido por la aceptación explícita de una política de privacidad por parte del trabajador, en cumplimiento de la normativa de protección de datos.

Diseño

El flujo de procesamiento facial se divide en tres operaciones principales:

1. **Registro** (POST /api/facial/registrar):

- Recibe la imagen como JPEG crudo (application/octet-stream) con el RUT en un header HTTP, que es como la envía el dispositivo, o como JSON con la imagen en Base64, opción que se mantiene para clientes web.
- Verifica que exista consentimiento biométrico en la tabla consentimientos. Si no existe, rechaza con HTTP 403.
- Guarda la imagen como static/previews/<id>.jpg.
- Aplica un **filtro de calidad de imagen** que mide la nitidez mediante la varianza del Laplaciano [48]. Si la imagen es demasiado plana o borrosa (por debajo de un umbral configurable, 50 por defecto), se rechaza como posible intento de suplantación (foto de pantalla o papel) antes de llegar a DeepFace, ahorrando procesamiento innecesario del modelo.
- Extrae el embedding con DeepFace usando el modelo Facenet y un detector configurable mediante variable de entorno (FACIAL_DETECTOR), utilizando MTCNN por defecto por su velocidad (tres veces más rápido que RetinaFace con precisión comparable) y sin anti-spoofing en esta etapa.
- Cifra el embedding con Fernet y lo almacena en la tabla encodings_faciales, que permite acumular múltiples fotos de referencia por persona con distintos ángulos e iluminación, mejorando la precisión en subsecuentes identificaciones.

El flujo completo de enrolamiento biométrico, desde la creación de la persona en el panel web por parte del administrador, pasando por el registro de huella en dos pasos en el ESP32-CAM, hasta la captura facial enviada por HTTP al backend donde se extrae, cifra y almacena el embedding, se ilustra en el diagrama de secuencia de la Figura 6.1 del Anexo C.

2. Identificación 1:N (POST /api/facial/identificar):

- Recibe la imagen como JPEG crudo (application/octet-stream) desde el ESP32-CAM, o como JSON con Base64 desde la web.
- Aplica el mismo filtro de calidad Laplacian como barrera anti-spoofing previa, rechazando imágenes de baja nitidez antes de invocar DeepFace.
- Extrae el embedding con DeepFace (anti-spoofing activado mediante `anti_spoofing=True`).
- Consulta una caché en memoria de embeddings (con TTL configurable, 60 segundos por defecto). Si la caché expiró, recarga todos los embeddings desde la base de datos. Para cada persona, compara contra todos los embeddings almacenados en `encodings_faciales` (multi-encoding) y selecciona la menor distancia. Esto mejora la precisión porque el sistema puede reconocer a un trabajador aunque la captura en vivo tenga una iluminación o ángulo diferentes a los de su foto de registro.
- Si la menor distancia es inferior al umbral (10.0 para Facenet), retorna el rut correspondiente. En caso contrario, retorna HTTP 404.

El ciclo de marcaje automático por centinela facial, en el que cada 6 segundos el ESP32-CAM evalúa el sensor PIR, captura una imagen, la envía por HTTP al endpoint de identificación, y según la respuesta del backend registra la asistencia y dispara el push asíncrono a los ERP configurados, se presenta en el diagrama de secuencia de la Figura 6.2 del Anexo C, dado su tamaño.

3. Verificación 1:1 (POST /api/facial/verificar):

- Recibe rut e imagen (Base64).
- Descifra todos los embeddings almacenados para esa persona en `encodings_faciales` y extrae el embedding de la imagen capturada (con anti-spoofing activado).
- Calcula la distancia euclidiana contra cada embedding de referencia y selecciona la menor. Si es menor que 10.0, retorna éxito con el nivel de confianza calculado como $\max(0, (1 - \text{distancia}/20) * 100)$.

El modelo seleccionado es Facenet por su equilibrio entre precisión y tamaño de embedding (128 dimensiones). El detector facial es configurable mediante variable de entorno (`FACIAL_DETECTOR`), utilizando MTCNN por defecto. Si bien RetinaFace ofrece mayor robustez ante variaciones extremas de pose e iluminación, MTCNN es tres veces más rápido y mantiene una precisión comparable para las condiciones típicas de una cámara de control de asistencia (rostro frontal a corta distancia), lo que lo hace más adecuado para un dispositivo embebido como el ESP32-CAM. El umbral de 10.0 para la distancia euclidiana fue elegido con base en la documentación de DeepFace y ajustes empíricos realizados durante el desarrollo.

Para el cifrado de embeddings se utiliza el algoritmo Fernet (AES-128 en modo CBC con HMAC-SHA256 para autenticación), parte de la librería `cryptography` de Python. La clave de cifrado se deriva aplicando SHA-256 a una variable de entorno (`BIOMETRIC_KEY`), garantizando que los embeddings nunca se almacenen en texto plano en la base de datos.

Implementación

Procesamiento de imágenes en el backend

El módulo `routes/facial.py` implementa los tres endpoints descritos en el diseño. Las funciones auxiliares incluyen:

- `_validar_calidad_imagen(img_path)`: Aplica el filtro Laplaciano (varianza del Laplaciano) sobre la imagen decodificada para medir su nitidez. Si el valor está por debajo del umbral configurable en `FACIAL_NITIDEZ_UMBRAL` (50 por defecto), la imagen se considera borrosa o plana (posible foto de pantalla o papel) y se rechaza antes de llegar a DeepFace, funcionando como una capa adicional de anti-spoofing.
- `extraer_embedding(img_path, anti_spoofing)`: Invoca `DeepFace.represent()` con el modelo Facenet pre-cargado y el detector configurable vía variable de entorno. Si `anti_spoofing=True`, activa la detección de suplantación.
- `guardar_imagen_de_registro(persona_id, imagen_b64)`: Decodifica la imagen Base64, la convierte a RGB con Pillow y la guarda en `static/previews/<id>.jpg`.
- `_verificar_consentimiento(persona_id)`: Consulta la tabla consentimientos para verificar que exista un registro de aceptación.
- `_log_biometrico(persona_id, dispositivo_id, tipo_operacion, resultado)`: Inserta un registro de auditoría en `logs_biometricos`.

Además, para optimizar el rendimiento, el modelo Facenet se precarga al iniciar el servidor mediante `DeepFace.build_model('Facenet', task='facial_recognition')`, eliminando la latencia de carga de pesos en la primera solicitud. Los embeddings almacenados en la base de datos se mantienen en una caché en memoria con tiempo de vida configurable (por defecto 60 segundos, mediante `FACIAL_CACHE_TTL`), evitando consultas repetitivas y descifrados a la base de datos en cada identificación. La caché se invalida automáticamente al registrar una nueva foto o al eliminar datos biométricos, garantizando consistencia.

Cifrado y descifrado de embeddings

El módulo `encryption.py` proporciona dos funciones:

- `cifrar_embedding(embedding)`: Serializa el embedding como JSON, lo cifra con Fernet y codifica el resultado en Base64 URL-safe.
- `descifrar_embedding(cifrado_str)`: Decodifica el Base64, descifra con Fernet y deserializa el JSON para obtener el vector original.

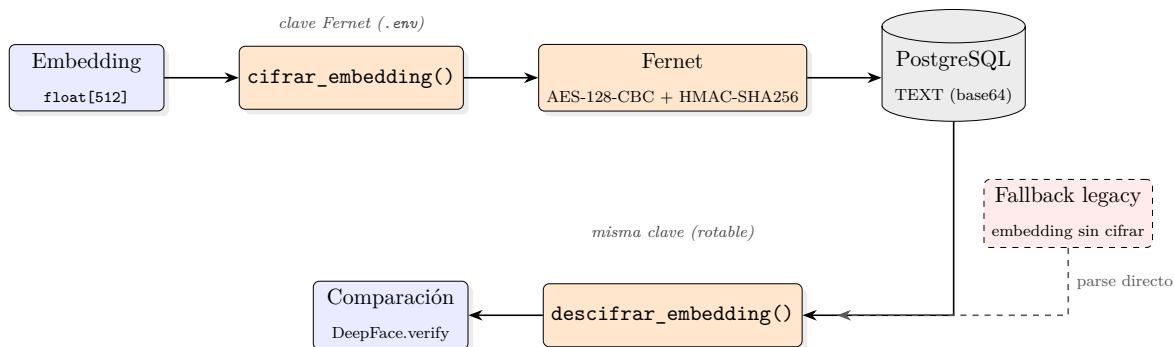


Figura 4.23: Pipeline de cifrado biométrico con Fernet (AES-128-CBC + HMAC-SHA256), con fallback para embeddings legacy almacenados sin cifrar.

Actualización y eliminación de datos faciales

Se implementó el endpoint `PUT /api/facial/actualizar/<persona_id>` que permite reemplazar el embedding y la foto de una persona existente. También se implementó `DELETE /api/personas/<id>/datos-biometricos` que elimina el embedding facial, la huella asociada, la foto en disco y el consentimiento, pero conserva los registros de asistencia históricos. La eliminación queda registrada en la tabla `eliminaciones_biometricas` para trazabilidad.

Se implementó además el endpoint `POST /api/facial/agregar-foto` que permite agregar fotos de referencia adicionales a una persona ya registrada, sin reemplazar las existentes. Cada nueva foto pasa por el mismo filtro de calidad Laplacian y se almacena

como un nuevo registro en `encodings_faciales`. Esta funcionalidad de enrolamiento progresivo permite mejorar la precisión del sistema a lo largo del tiempo: si un trabajador experimenta falsos rechazos, se le pueden agregar más fotos de referencia con distintos ángulos o condiciones de iluminación, aumentando la robustez del reconocimiento sin necesidad de rehacer el registro completo.

Pruebas

Se diseñaron las siguientes pruebas para validar el módulo de reconocimiento facial:

- **Prueba de registro facial exitoso:** Enviar una imagen facial válida junto con un rut que tenga consentimiento registrado. Verificar que el backend extraiga el embedding, lo almacene cifrado y responda con HTTP 200.
- **Prueba de detección de duplicados:** Registrar una segunda imagen del mismo rostro para otra persona. Verificar que el backend detecte la similitud (distancia < 10.0) y rechace el registro con HTTP 409, indicando el nombre de la persona con la que coincide.
- **Prueba de rechazo por falta de consentimiento:** Intentar registrar un rostro para una persona sin consentimiento biométrico. Verificar que el backend rechace la operación con HTTP 403.
- **Prueba de identificación 1:N:** Registrar 3 personas con sus respectivas fotos. Enviar una nueva foto de una de ellas al endpoint de identificación. Verificar que el backend retorne el rut correcto.
- **Prueba de identificación con rostro desconocido:** Enviar una foto de una persona no registrada. Verificar que el backend retorne HTTP 404 y registre el intento fallido en `logs_biometricos`.
- **Prueba de anti-spoofing con fotografía impresa:** Capturar una foto de un rostro registrado desde una pantalla o impresión y enviarla al endpoint de identificación. Verificar que DeepFace con `anti_spoofing=True` rechace la imagen por no detectar un rostro real (excepción `ValueError` con mensaje “Spoof detected”).
- **Prueba de resistencia a condiciones de iluminación:** Capturar imágenes del mismo rostro bajo 3 condiciones de iluminación (luz natural, luz artificial tenue, contraluz). Verificar que los 3 embeddings generados mantengan distancia euclidiana entre sí menor a 10.0 y que el sistema identifique correctamente a la persona.

- **Prueba de cifrado y descifrado:** Almacenar un embedding en la base de datos y verificar que el valor en la columna `encoding_facial` no sea legible (no contenga los números del vector en texto plano). Luego, recuperar el registro y descifrar el embedding; verificar que el vector original se reconstruya idénticamente.
- **Prueba de eliminación de datos biométricos:** Ejecutar el endpoint de eliminación de datos biométricos para una persona con rostro registrado. Verificar que el embedding se elimine de la base de datos, que la foto se borre del disco, que el consentimiento se elimine, que los registros de asistencia se conserven y que quede una entrada en `eliminaciones_biometricas`.
- **Prueba de filtro de calidad de imagen:** Enviar al endpoint de registro una imagen deliberadamente borrosa o una foto de una pantalla (baja varianza de Laplacian). Verificar que el backend rechace la imagen antes de invocar DeepFace, retornando un error de calidad insuficiente.
- **Prueba de múltiples fotos de referencia:** Registrar una persona con una foto frontal. Luego, agregar una segunda foto de la misma persona con distinto ángulo mediante `POST /api/facial/agregar-foto`. Realizar una identificación con una tercera foto y verificar que el sistema reconozca a la persona correctamente, comparando contra ambos embeddings de referencia.
- **Prueba de caché de embeddings:** Realizar una identificación exitosa y verificar los tiempos de respuesta. Realizar una segunda identificación inmediatamente después; verificar que el tiempo sea significativamente menor porque los embeddings se sirvieron desde la caché en memoria sin consultar la base de datos.

Endpoint identificar-o-registrar

Se implementó el endpoint `POST /api/facial/identificar-o-registrar` que combina ambos flujos en una sola llamada: primero intenta identificar a la persona por su rostro; si la identificación es exitosa, retorna los datos de la persona; si falla y se proporciona un RUT con consentimiento biométrico, registra automáticamente la nueva persona con el embedding facial extraído. Este endpoint facilita el enrolamiento progresivo en el dispositivo: un trabajador puede colocarse frente al ESP32-CAM y, si ya está registrado, se marca su asistencia; si no, el sistema lo registra sobre la marcha. El endpoint incluye validación de calidad de imagen, anti-spoofing y log biométrico tanto para identificación como para registro.

Refinamientos posteriores

- **Transmisión de imágenes vía octet-stream:** el envío de la imagen facial migró de un esquema basado en JSON con la foto codificada en Base64 a un envío binario directo (Content-Type: application/octet-stream) con el RUT o la MAC del dispositivo en un header HTTP (X-RUT / X-Device-MAC). El backend mantiene compatibilidad retroactiva con el formato Base64 como *fallback*. El cambio elimina el overhead de codificación Base64 (aproximadamente 33% de tamaño adicional) y simplifica el firmware, que ahora envía directamente el buffer JPEG capturado por la cámara (fb->buf, fb->len).
- **Captura de 3 fotos de referencia por persona:** el flujo de registro se extendió de una sola foto a tres capturas sucesivas (FOTOS_REQUERIDAS = 3 en el firmware), idealmente frontal y de ambos perfiles, generando tres embeddings independientes almacenados en la tabla encodings_faciales. Esto mejora la tasa de identificación ante variaciones de ángulo o iluminación, ya que la comparación 1:N evalúa contra todos los embeddings de referencia disponibles.

4.2.5 Iteración 5: Autenticación, multi-tenant y enrolamiento de dispositivos

La quinta iteración incorpora la capa de seguridad y administración del sistema: autenticación de usuarios del panel web mediante JWT, un modelo multi-tenant que permite a múltiples empresas operar sobre la misma instancia del backend con datos aislados, y un mecanismo seguro de enrolamiento de dispositivos ESP32-CAM mediante PIN de un solo uso.

Análisis

El sistema, al estar orientado a un entorno empresarial, requiere que diferentes organizaciones puedan utilizar la misma plataforma sin que sus datos se mezclen. Asimismo, el panel web administrativo debe restringir el acceso según el rol del usuario. Finalmente, cada dispositivo ESP32-CAM debe ser vinculado de forma segura a una empresa específica, evitando que dispositivos no autorizados inyecten datos en el sistema.

Los requisitos identificados son:

- **Multi-tenant:** Cada entidad (persona, turno, asignación, asistencia, dispositivo) debe pertenecer a una empresa. Las consultas deben filtrarse por empresa según el contexto del usuario autenticado. El administrador global (rol admin) tiene visibilidad sobre todas las empresas.

- **Autenticación JWT:** El panel web backend debe requerir autenticación para operaciones sensibles. Los tokens JWT (JSON Web Tokens), firmados con HMAC-SHA256, transportan el ID de usuario, la empresa seleccionada y su rol. Tienen expiración de 24 horas.
- **Roles de usuario:** Se definen tres roles jerárquicos:
 - admin: Acceso total al sistema, puede crear empresas, gestionar todos los usuarios y dispositivos.
 - empleador: Acceso limitado a los datos de su empresa; puede gestionar trabajadores, turnos y ver asistencias.
 - trabajador: Acceso restringido a sus propios datos de asistencia.
- **Enrolamiento de dispositivos:** Un administrador genera un PIN de 8 caracteres en el panel web. Ese PIN se ingresa en el ESP32-CAM (vía web embebida), que lo envía al backend junto con su MAC address. Si el PIN es válido y no ha sido usado, el dispositivo queda enrolado y asociado a la empresa correspondiente.
- **Auto-registro de empresas:** Se requiere un endpoint público que permita a una nueva organización registrarse de forma autónoma en el sistema, sin intervención del administrador central. La empresa proporciona sus datos (nombre, RUT, email, etc.) y los datos del usuario administrador; el sistema crea la empresa, el usuario, y la asociación entre ambos con rol admin en una sola transacción atómica, retornando un token JWT para que el nuevo administrador quede autenticado inmediatamente. Esto facilita el despliegue del sistema en nuevos clientes sin requerir configuración manual previa.
- **Monitorización de dispositivos:** Una vez enrolado, el ESP32-CAM envía heartbeats periódicos vía MQTT. El backend actualiza el estado del dispositivo (activo) y registra su última actividad. Un proceso watchdog marca como inactivo a los dispositivos que no han enviado heartbeat en más de 90 segundos.

Diseño

Modelo de datos multi-tenant

La estrategia multi-tenant se implementa a nivel de base de datos mediante una columna `empresa_id` en las tablas `personas`, `turnos`, `dispositivos`, `integraciones_erp` y `asistencias` (esta última a través de la relación con `personas`). La tabla `empresas` actúa como entidad raíz del tenant. La tabla asociativa `usuario_empresa` vincula usuarios con empresas y define el rol que cada usuario tiene en cada empresa, permitiendo que un mismo usuario tenga roles distintos en diferentes organizaciones.

Flujo de autenticación

1. El usuario envía email y contraseña al endpoint POST /api/auth/login.
2. El backend valida las credenciales contra la tabla usuarios_web usando bcrypt.
3. Se consultan las empresas asociadas al usuario en usuario_empresa.
4. Si el usuario pertenece a una sola empresa, se selecciona automáticamente. Si pertenece a varias, se retorna la lista para que elija.
5. Se genera un token JWT con payload: user_id, empresa_id, rol, persona_id (si es trabajador) y exp (24 horas).
6. Las peticiones subsecuentes deben incluir el header Authorization: Bearer <token>.

Se implementan dos niveles de protección para los endpoints:

- @token_required: Exige token JWT válido.
- @requiere_rol('admin', 'empleador'): Además del token, exige que el rol del usuario esté en la lista permitida.
- @token_opcional: Intenta extraer el token si está presente; si no, permite el acceso sin autenticación (útil para endpoints consumidos por el ESP32-CAM, que en su lugar envían el header X-Device-MAC para identificar la empresa).

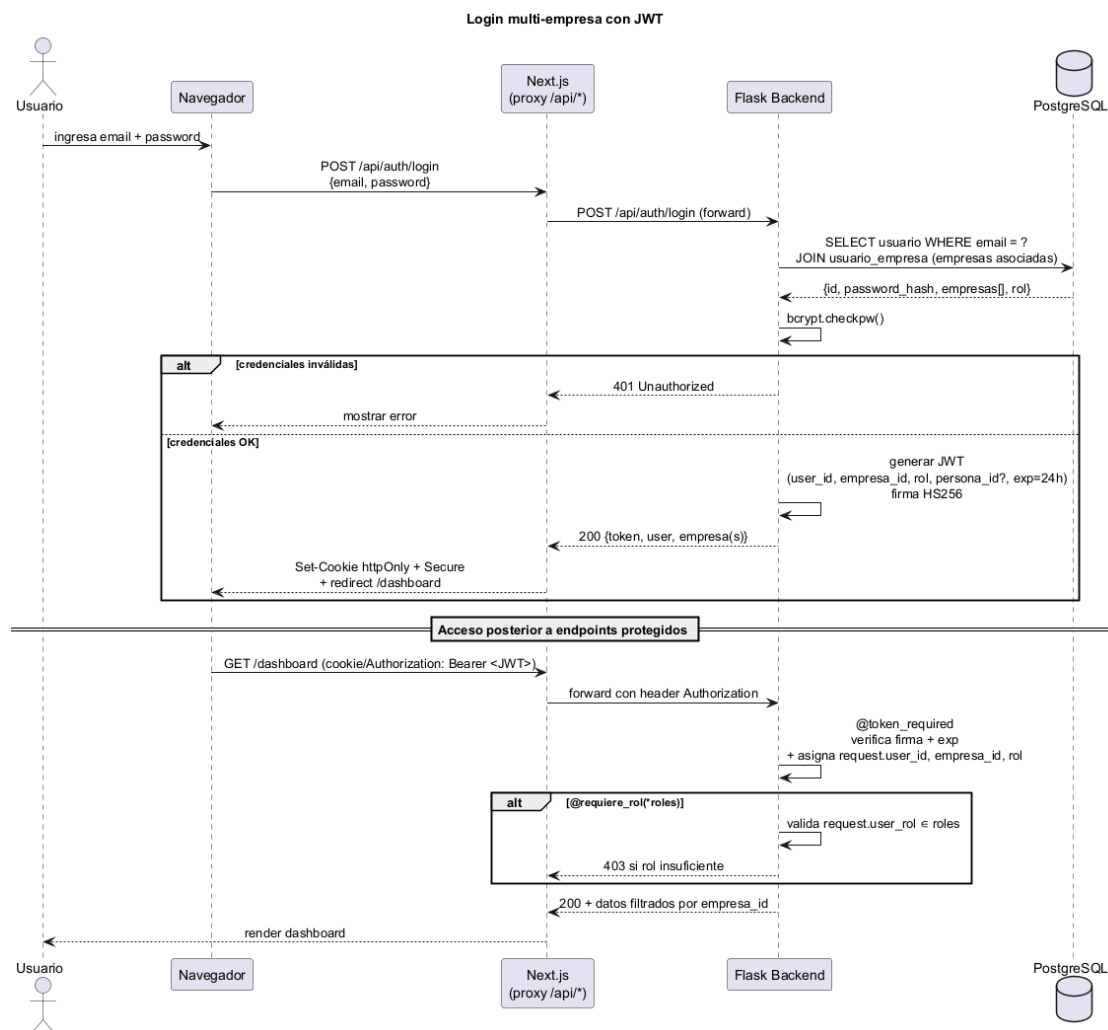


Figura 4.24: Diagrama de secuencia del login multi-empresa con JWT.

Flujo de enrolamiento de dispositivos

1. Un administrador o empleador accede a `POST /api/dispositivos/generar-pin` desde el panel web, especificando un nombre para el dispositivo. El backend genera un PIN alfanumérico de 8 caracteres (ej. A3B9XK2M) usando `secrets.choice()` y lo inserta en la tabla `dispositivos` con `enrolado = FALSE`.
2. El administrador ingresa el PIN en la interfaz web del ESP32-CAM (`/wifi-setup`).
3. Al guardar la configuración, el ESP32-CAM envía `POST /api/dispositivos/enrolar` con el PIN, su MAC address y su IP local.
4. El backend valida el PIN, asocia la MAC y la IP al dispositivo, marca `enrolado = TRUE` y limpia el PIN para que no pueda ser reutilizado.
5. A partir de este momento, el ESP32-CAM envía heartbeats periódicos y el backend lo reconoce como dispositivo autorizado de la empresa.

Heartbeat y Last Will Testament (LWT)

El ESP32-CAM publica periódicamente (cada 30 segundos) un mensaje JSON en el tópico `esp32/heartbeat/<MAC>` con su MAC, timestamp e IP. El backend actualiza el campo `ultimo_heartbeat` y marca el estado como activo. Al conectarse al broker MQTT, el ESP32-CAM configura un mensaje LWT en el tópico `esp32/lwt/<MAC>` con `{"estado": "inactivo"}`. Si el dispositivo se desconecta abruptamente, el broker publica automáticamente el LWT, y el backend marca el dispositivo como inactivo.

Además del LWT y el watchdog pasivo, se implementó un ping activo como tercera capa de detección: un hilo `device_pinger()` publica periódicamente (cada 30 segundos) un mensaje en el tópico `esp32/ping/<MAC>`. El ESP32-CAM, al recibirlo, responde inmediatamente publicando un heartbeat con el campo `"pong": true`. Si el backend no recibe respuesta dentro de 60 segundos, marca el dispositivo como inactivo. Este mecanismo permite detectar desconexiones incluso cuando el mensaje LWT no llega al broker (por ejemplo, por pérdida abrupta de conectividad de red). Adicionalmente, un hilo `device_watchdog` ejecutado cada 60 segundos verifica qué dispositivos no han enviado heartbeat en más de 90 segundos y los marca como inactivos, constituyendo una capa de respaldo adicional.

Implementación

Módulo de autenticación

El módulo `routes/auth.py` implementa:

- `login()`: Valida credenciales con `bcrypt`, determina la empresa (con selección múltiple si aplica), vincula al trabajador con su `persona_id` si el rol es trabajador, y genera el token JWT.
- `register()`: Permite a administradores y empleadores crear nuevos usuarios, asignándoles rol y empresa. Si el email ya existe, se agrega la nueva asociación `usuario_empresa` en lugar de duplicar el usuario.
- `me()`: Retorna los datos del usuario autenticado, incluyendo sus empresas y rol actual.
- `cambiar_password()`: Permite al usuario cambiar su contraseña validando la actual.
- `generar_pin_enrolamiento()`: Crea un PIN para enrolar un nuevo dispositivo.
- `enrolar_dispositivo()`: Valida el PIN y completa el enrolamiento del ESP32-CAM.

- `listar_usuarios()`, `eliminar_usuario()`, `listar_empresas()`, `crear_empresa()`: Operaciones administrativas con control de roles.
- `register_company()`: Endpoint público POST `/api/auth/register-company` que permite el auto-registro de una nueva empresa. Recibe los datos de la organización (nombre, RUT, email, teléfono, dirección) y del usuario administrador (nombre, email, contraseña). En una única transacción atómica crea la empresa, el usuario con hash `bcrypt`, y la asociación `usuario_empresa` con rol `admin`. Retorna un token JWT para autenticar inmediatamente al nuevo administrador.

Los decoradores `@token_required`, `@token_opcional` y `@requiere_rol` se definen en el mismo módulo. El decorador `@token_opcional` incluye lógica para inferir la empresa a partir del header `X-Device-MAC` cuando no hay token JWT presente, buscando la MAC en la tabla de dispositivos. Si la MAC no existe, automáticamente crea un nuevo registro en dispositivos con estado pendiente, lo que permite que los ESP32-CAM comiencen a enviar datos incluso antes de ser formalmente enrolados.

Gestión de dispositivos

El módulo `routes/dispositivos.py` implementa:

- GET `/api/dispositivos`: Lista los dispositivos con su estado, MAC, IP y empresa.
- PUT `/api/dispositivos/<id>`: Actualiza el nombre del dispositivo.
- DELETE `/api/dispositivos/<id>`: Elimina un dispositivo (admin) o lo desvincula de la empresa (empleador).
- POST `/api/dispositivos/verificar`: Prueba conectividad con un dispositivo dada su IP, consultando el endpoint `/estado` del ESP32-CAM.

Monitorización vía MQTT

En el módulo `mqtt_handler.py`, la función `on_message` maneja los tópicos de heartbeat y LWT:

- `esp32/heartbeat/<MAC>`: Actualiza `ultimo_heartbeat`, `estado = 'activo'` y opcionalmente `ip_local` del dispositivo.
- `esp32/lwt/<MAC>`: Marca `estado = 'inactivo'` en la base de datos.

El watchdog (`device_watchdog`) ejecuta un barrido inicial al arrancar (todos los dispositivos se marcan inactivos hasta que envíen su primer heartbeat) y luego verifica cada 60 segundos los heartbeats vencidos.

Pruebas

Se diseñaron las siguientes pruebas para validar la autenticación, el multi-tenant y el enrolamiento:

- **Prueba de login exitoso:** Enviar credenciales válidas del administrador por defecto (admin@empresa.cl / admin123). Verificar que se reciba un token JWT, los datos del usuario y su empresa. Decodificar el token y verificar que contenga user_id, empresa_id y rol = admin.
- **Prueba de login con múltiples empresas:** Crear una segunda empresa y asignar el mismo usuario a ella con rol empleador. Iniciar sesión sin especificar empresa. Verificar que el backend retorne la lista de empresas disponibles. Luego, iniciar sesión especificando una empresa y verificar que el token refleje esa selección.
- **Prueba de rechazo de credenciales inválidas:** Intentar login con contraseña incorrecta. Verificar HTTP 401 y mensaje “Credenciales inválidas”.
- **Prueba de acceso sin token:** Intentar acceder a GET /api/personas sin enviar header Authorization, desde un cliente sin X-Device-MAC. Verificar que el endpoint responda correctamente (acceso público o con datos limitados según el decorador).
- **Prueba de acceso con rol insuficiente:** Autenticarse como trabajador e intentar crear un turno (POST /api/turnos). Verificar HTTP 403 y mensaje “Permisos insuficientes”.
- **Prueba de aislamiento multi-tenant:** Crear dos empresas (A y B), registrar una persona en cada una. Autenticarse como empleador de la empresa A y solicitar GET /api/personas. Verificar que solo se retornen las personas de la empresa A.
- **Prueba de generación de PIN:** Autenticarse como empleador y solicitar POST /api/dispositivos/generar-pin. Verificar que se reciba un PIN de 8 caracteres alfanuméricos y que en la base de datos exista un dispositivo con enrolado = FALSE.
- **Prueba de enrolamiento exitoso:** Con un PIN generado, enviar POST /api/dispositivos/enrolar con el PIN, una MAC y una IP. Verificar que el backend marque el dispositivo como enrolado, asocie la MAC y limpie el PIN. Verificar que el mismo PIN no pueda usarse una segunda vez (HTTP 409).
- **Prueba de heartbeat y estado:** Simular heartbeats desde un dispositivo enrolado. Verificar que el estado en la base de datos sea activo. Detener los

heartbeats por más de 90 segundos y verificar que el watchdog marque el dispositivo como inactivo.

- **Prueba de LWT:** Conectar el ESP32-CAM al broker MQTT, verificar que el backend reciba el heartbeat y lo marque activo. Desconectar el dispositivo abruptamente (corte de energía). Verificar que el broker publique el LWT y el backend actualice el estado a inactivo en menos de 10 segundos.
- **Prueba de auto-registro de empresa:** Enviar una solicitud a POST /api/auth/register-company con los datos de una nueva empresa y su administrador. Verificar que se cree la empresa en la base de datos, que el usuario administrador se cree con contraseña encriptada (bcrypt), que exista la asociación usuario_empresa con rol admin, y que el backend retorne un token JWT válido que permita acceder a endpoints protegidos. Verificar que un segundo intento con el mismo email de administrador retorne HTTP 409 (Conflicto).

Refinamientos posteriores

- **"token_opcional":** el decorador token_opcional, que permite que un mismo endpoint acepte tanto usuarios web autenticados por JWT como dispositivos ESP32 identificados por el header X-Device-MAC, inicialmente creaba un registro nuevo en dispositivos si la MAC no existía en la base de datos. Este comportamiento se eliminó: ahora el decorador únicamente busca y vincula dispositivos ya existentes, dejando la creación exclusivamente al flujo explícito de enrolamiento con PIN. El cambio cierra una vía por la que un dispositivo no autorizado podía aparecer en el sistema sin pasar por el control del administrador o empleador.
- **PIN de enrolamiento por empresa:** la generación de PIN (POST /api/auth/dispositivos/generar-pin) quedó condicionada al rol de quien la solicita: un administrador puede generar el PIN para cualquier empresa indicada en la petición, mientras que un empleador solo puede generarlo para la empresa a la que pertenece (request.empresa_id). El dispositivo creado queda asociado a esa empresa desde el momento de la generación del PIN, reforzando el aislamiento multi-tenant también a nivel de hardware enrolado.

4.2.6 Iteración 6: Módulo antifraude y validación previa a la captura

Esta iteración implementa la prevención de suplantación de identidad en el ESP32-CAM mediante detección física de presencia, control de iluminación y procesamiento

biométrico. Si bien el plan original contemplaba un sensor infrarrojo (IR) dedicado para detectar intentos de suplantación con fotografías, el enfoque implementado evolucionó hacia una solución de múltiples capas que resultó más robusta y prescindió del sensor IR adicional.

Análisis

El análisis de escenarios de suplantación identificó los siguientes vectores de ataque potenciales contra el sistema de reconocimiento facial del ESP32-CAM:

- **Fotografía impresa:** Un atacante presenta una foto del rostro de un trabajador legítimo frente a la cámara.
- **Pantalla de dispositivo:** Un atacante muestra un video o imagen del trabajador en la pantalla de un teléfono o tablet.
- **Ausencia de presencia real:** La cámara captura sin que haya una persona físicamente frente al dispositivo (falsos positivos del sensor de movimiento).

Inicialmente se planificó el uso de un sensor IR dedicado que permitiera detectar la presencia de un rostro real mediante la emisión y recepción de luz infrarroja, cuya reflectancia difiere entre piel humana y superficies planas como papel o pantallas. Sin embargo, durante el desarrollo se identificaron dos factores que motivaron el cambio de estrategia:

1. **Disponibilidad de GPIO:** El ESP32-CAM tiene una cantidad muy limitada de pines GPIO disponibles tras la asignación del bus paralelo de la cámara. Destinar un pin adicional al sensor IR hubiera requerido sacrificar otra funcionalidad (como el sensor PIR de presencia o el control PWM del flash).
2. **Madurez del anti-spoofing por software:** DeepFace, la librería seleccionada para el reconocimiento facial, incorpora un detector de suplantación (**anti-spoofing**) basado en redes neuronales convolucionales que analiza micro-texturas, reflectancia de la piel y profundidad en la imagen capturada. Este detector es capaz de distinguir entre un rostro real tridimensional y una superficie plana (foto, pantalla) con una precisión comparable o superior a la de sensores IR de bajo costo como los inicialmente considerados.

En consecuencia, se adoptó un enfoque de múltiples capas que combina:

1. **Detección física de presencia (PIR):** Un sensor infrarrojo pasivo (PIR) que detecta el calor corporal, activando el sistema de captura solo cuando hay una persona real frente al dispositivo. Esto elimina falsos positivos del software y reduce el consumo energético del sensor de huella y la cámara.

2. **Iluminación controlada (flash PWM):** Un destello breve y de intensidad moderada (50% de duty cycle a 5 kHz) que ilumina el rostro durante la captura sin enceguecer a la persona. La respuesta del rostro real a esta fuente de luz puntual difiere de la de una superficie plana, aportando una capa adicional de discriminación.
3. **Anti-spoofing por software (DeepFace):** La capa más potente, que analiza la imagen capturada con un modelo entrenado para detectar intentos de suplantación. Las imágenes que no superan esta validación son rechazadas con una excepción `ValueError` que el backend traduce en un mensaje de error al ESP32-CAM.

Diseño

Sensor PIR como portero del reconocimiento facial

El lector de huellas se revisa de forma continua, sin depender del PIR: cada 500 milisegundos el sistema consulta al AS608 por si hay un dedo apoyado, de modo que una marcación por huella responda de inmediato sin esperar a que algo más la habilite. El PIR conectado al GPIO12 cumple un rol distinto, acotado al reconocimiento facial: mientras no detecta movimiento, la cámara no se usa para buscar un rostro. Al detectar movimiento (un cambio en la radiación infrarroja del entorno), el sistema enciende el flash a baja intensidad, marca que hay alguien frente al dispositivo y, si en ese momento había un bloqueo temporal del menú activo, lo anula, dando prioridad a la persona que se presentó físicamente. Tras una breve pausa de 800 milisegundos para que la lectura del sensor se estabilice, el sistema queda intentando identificar el rostro cada 6 segundos mientras dure la presencia, siempre que haya conectividad con el backend y hayan pasado al menos 8 segundos desde la última marcación. Si el PIR no vuelve a detectar movimiento durante 15 segundos, se asume que fue una falsa alarma o que la persona se retiró: se apaga el flash y el sistema vuelve a esperar sin haber intentado nada por cámara.

Esta separación evita que el reconocimiento facial, más costoso en procesamiento que la lectura de huella, se ejecute todo el tiempo sin necesidad: solo se activa cuando el PIR confirma que probablemente hay alguien frente al dispositivo.

Control de acceso con cooldown y bloqueo

Para evitar marcaciones múltiples accidentales y garantizar que cada interacción sea deliberada, se implementan tres mecanismos de control:

1. **Cooldown entre marcaciones** (`COOLDOWN_TIEMPO = 8000 ms`): Tras una marcación exitosa, el sistema ignora nuevos intentos durante 8 segundos.

2. **Debounce de huella** (FINGER_DEBOUNCE = 4000 ms): Una misma huella no puede gatillar dos marcaciones en un intervalo menor a 4 segundos, evitando lecturas múltiples del mismo dedo.
3. **Bloqueo por menú** (BLOQUEO_MENU_MS = 30000 ms): Cuando se utiliza la interfaz web para operaciones administrativas (registro, edición, configuración), se activa un bloqueo de 30 segundos que impide la marcación automática, evitando que la persona que está operando el menú sea registrada inadvertidamente.

Firma de movimiento

Para detectar si la escena frente a la cámara cambió entre dos capturas consecutivas (lo que indicaría una persona real moviéndose versus una foto estática), se implementa una firma de movimiento que calcula un hash reducido del buffer JPEG:

- Se muestrean 128 bytes del frame capturado, distribuidos uniformemente a lo largo del buffer.
- Se calcula la suma acumulada de esos bytes, produciendo una firma de 32 bits.
- Si la diferencia entre la firma actual y la anterior supera un umbral (UMBRAL_MOVIMIENTO = 1800), se considera que hubo movimiento real en la escena.

Implementación

Integración del PIR

El sensor PIR se conecta al GPIO12 del ESP32-CAM, configurado con resistencia de pull-down interna para evitar lecturas flotantes. En el setup() se incluye una fase de calibración de 3 segundos (delay(3000)) para estabilizar el sensor antes de comenzar el bucle principal.

En el loop(), la lógica de detección sigue el siguiente flujo:

- Si el PIR detecta HIGH y el flag "hayAlguienFrenteAlSensor" es falso, se activa el modo alerta y se esperan 800 ms para estabilizar la fuente de alimentación (el capacitor de 1000 μ F del ESP32-CAM necesita recuperarse antes de encender la cámara).
- Si el PIR detecta HIGH estando ya en modo alerta, se renueva el timer de actividad.
- Si pasan más de 15 segundos sin detección, se desactiva el modo alerta y se retorna a reposo.

Flash PWM para iluminación facial

El flash LED integrado del ESP32-CAM se controla mediante PWM en el GPIO4, configurado como salida LED PWM con frecuencia de 5 kHz y resolución de 8 bits. Durante la captura de una imagen (tanto para registro como para identificación), el flash se activa con un duty cycle del 50% (`FLASH_PWM_DUTY_50 = 128`) durante 150-200 milisegundos, tiempo suficiente para que el sensor OV2640 ajuste su exposición.

La secuencia de captura es la siguiente:

1. Encender flash al 50% (`ledcWrite(FLASH_PIN, 128)`).
2. Esperar 150 ms para estabilización del sensor.
3. Capturar el frame (`esp_camera_fb_get()`).
4. Apagar el flash inmediatamente (`ledcWrite(FLASH_PIN, 0)`).
5. Procesar o transmitir el frame.

Este enfoque minimiza el tiempo total de encendido del flash y, por tanto, el consumo eléctrico y la molestia para el usuario.

Señalización mediante el LED verde

Además de la iluminación para captura, que usa el flash, el resultado de una operación se señala con el LED verde integrado en el dispositivo:

- **Éxito** (`flashExito()`): el LED parpadea dos veces (150 ms encendido, 150 ms apagado en cada destello).
- **Error** (`flashError()`): no hay señal luminosa propia; la función queda como punto de extensión reservado en el firmware para una señal de error dedicada (por ejemplo, un patrón distinto de parpadeo o un LED rojo adicional).

El parpadeo del LED verde permite al usuario confirmar de un vistazo que su marcación se registró, sin necesidad de mirar una pantalla. La ausencia de una señal equivalente para el caso de error es una limitación reconocida del prototipo (Sección 6.3), ya que actualmente un error solo es visible al consultar la vista web del dispositivo.

Validación anti-spoofing en el backend

La validación anti-spoofing se ejecuta en el backend, no en el ESP32-CAM, debido a las limitaciones de procesamiento del microcontrolador. La función `extraer_embedding()` en `routes/facial.py` recibe el parámetro `anti_spoofing=True` para los endpoints de identificación y verificación, lo que activa el detector de suplantación de DeepFace. Si

la imagen no supera la validación (por ser una foto, pantalla o carecer de características faciales reales), DeepFace lanza una excepción `ValueError` que el backend captura y traduce en una respuesta HTTP 400 con el mensaje “Spoof detected”.

Pruebas

Se diseñaron las siguientes pruebas para validar el módulo antifraude:

- **Prueba de detección PIR:** Situar a una persona a 50 cm del dispositivo. Verificar que el PIR detecte el movimiento en menos de 1 segundo y que el sistema transicione a modo activo. Retirar a la persona y verificar que tras 15 segundos sin detección, el sistema retorne a reposo.
- **Prueba de falsos positivos del PIR:** Dejar el dispositivo en una habitación vacía durante 10 minutos y monitorear los logs. Verificar que no se active el modo de captura sin presencia humana real (tolerancia máxima: 1 falso positivo por hora).
- **Prueba de cooldown:** Realizar una marcación exitosa por huella. Intentar inmediatamente una segunda marcación. Verificar que el sistema rechace el intento por estar en período de cooldown (8 segundos).
- **Prueba de debounce de huella:** Mantener el dedo presionado sobre el sensor AS608 durante 5 segundos. Verificar que solo se registre una marcación, no múltiples.
- **Prueba de bloqueo por menú:** Acceder a la vista /register desde el navegador. Mientras la página está abierta, intentar provocar una marcación automática con huella. Verificar que el sistema esté bloqueado durante 30 segundos.
- **Prueba de anti-spoofing con fotografía:** Imprimir una foto de un rostro registrado y presentarla frente al ESP32-CAM con el PIR activado (para ello, mover la foto simulando una persona). Verificar que DeepFace rechace la imagen con `anti_spoofing=True` y que el backend retorne HTTP 400.
- **Prueba de anti-spoofing con pantalla:** Reproducir un video del rostro de una persona registrada en un teléfono móvil y presentarlo frente al ESP32-CAM. Verificar que el sistema rechace la identificación.
- **Prueba de flash PWM:** Durante una captura, medir con un osciloscopio o cámara de alta velocidad que el flash se encienda al 50% de duty cycle durante no más de 200 ms. Verificar que la iluminación sea suficiente para obtener una imagen nítida del rostro.

- **Prueba de señalización:** Provocar una marcación exitosa y verificar que el LED verde parpadee dos veces. Provocar un error (huella no reconocida) y verificar que no se emita ninguna señal luminosa.

4.2.7 Iteración 7: Panel web backend e integración con ERP

La séptima iteración incorpora el panel de administración web del backend y el módulo de integración con sistemas ERP externos, permitiendo que los datos de asistencia capturados por el dispositivo IoT sean automáticamente transmitidos a plataformas empresariales de recursos humanos o nómina.

Análisis

El sistema, hasta esta iteración, permite registrar asistencias y almacenarlas en la base de datos, pero carece de una interfaz administrativa centralizada y de un mecanismo para integrar los datos con sistemas ERP existentes. Se identificaron los siguientes requisitos:

- **Panel web administrativo:** Una interfaz web que permita a administradores y empleadores gestionar personas, turnos, asignaciones, visualizar asistencias, administrar dispositivos enrolados y configurar integraciones ERP. Debe ser accesible vía navegador sin dependencia del ESP32-CAM.
- **API REST completa y documentada:** Todos los blueprints implementados en iteraciones anteriores deben consolidarse como una API REST coherente, con métodos HTTP estándar, respuestas JSON estructuradas y manejo de autenticación por JWT.
- **Integración con ERP vía webhooks:** Permitir configurar múltiples destinos ERP, cada uno con su propia URL de webhook, headers personalizados (ej. tokens de autenticación) y mapeo de campos (field mapping) para adaptar el formato de los datos de asistencia al esquema esperado por cada ERP.
- **Envío automático:** Las asistencias registradas deben enviarse automáticamente a los ERP configurados, en un hilo separado para no bloquear la respuesta HTTP al dispositivo.
- **Test de conectividad:** Cada integración ERP debe poder probarse manualmente desde el panel web antes de activar el envío automático.

Diseño

Arquitectura del backend

El backend se organiza en 9 blueprints de Flask, cada uno con responsabilidades bien definidas:

1. `auth_bp`: Autenticación JWT, registro de usuarios, gestión de empresas, enrolamiento de dispositivos.
2. `personas_bp`: CRUD de personas con consentimiento biométrico y eliminación de datos sensibles.
3. `turnos_bp`: CRUD de turnos con filtro por empresa.
4. `asignaciones_bp`: CRUD de asignaciones persona-turno.
5. `asistencias_bp`: Registro de asistencias individuales y sincronización por lotes.
6. `facial_bp`: Registro, verificación e identificación facial con anti-spoofing y cifrado.
7. `dispositivos_bp`: Gestión de dispositivos enrolados y verificación de conectividad.
8. `logs_bp`: Consulta de logs de sincronización y logs biométricos.
9. `erp_bp`: CRUD de integraciones ERP, envío de datos y test de conectividad.

Diseño del módulo ERP

La integración con ERP se basa en el patrón webhook saliente: cuando se registra una asistencia, el backend notifica a cada ERP configurado mediante una solicitud HTTP POST al webhook correspondiente, con los datos de la marcación en el cuerpo JSON.

Cada integración ERP se configura con los siguientes parámetros:

- `nombre`: Identificador descriptivo (ej. “Softland RRHH”).
- `tipo`: Clasificación del ERP (`generic`, `softland`, `defontana`, `sap`).
- `webhook_url`: Endpoint HTTP donde el ERP espera recibir los datos.
- `headers`: Objeto JSON con headers HTTP personalizados (ej. `{"Authorization": "Bearer <token>"}`).
- `field_map`: Objeto JSON que mapea los campos estándar del sistema (`rut`, `nombre`, `tipo`, `metodo`, `fecha_hora`) a los nombres de campo esperados por el ERP.

- `envio_auto`: Booleano que indica si el envío se realiza automáticamente al registrar cada asistencia, o solo manualmente.
- `activo`: Booleano para habilitar o deshabilitar temporalmente la integración sin eliminarla.

El field mapping permite adaptar el formato de salida sin modificar el código del backend. Por ejemplo, un ERP que espera los campos `employee_id`, `timestamp` y `event_type` puede configurarse con:

```
{"rut": "employee_id", "fecha_hora": "timestamp", "tipo": "event_type"}
```

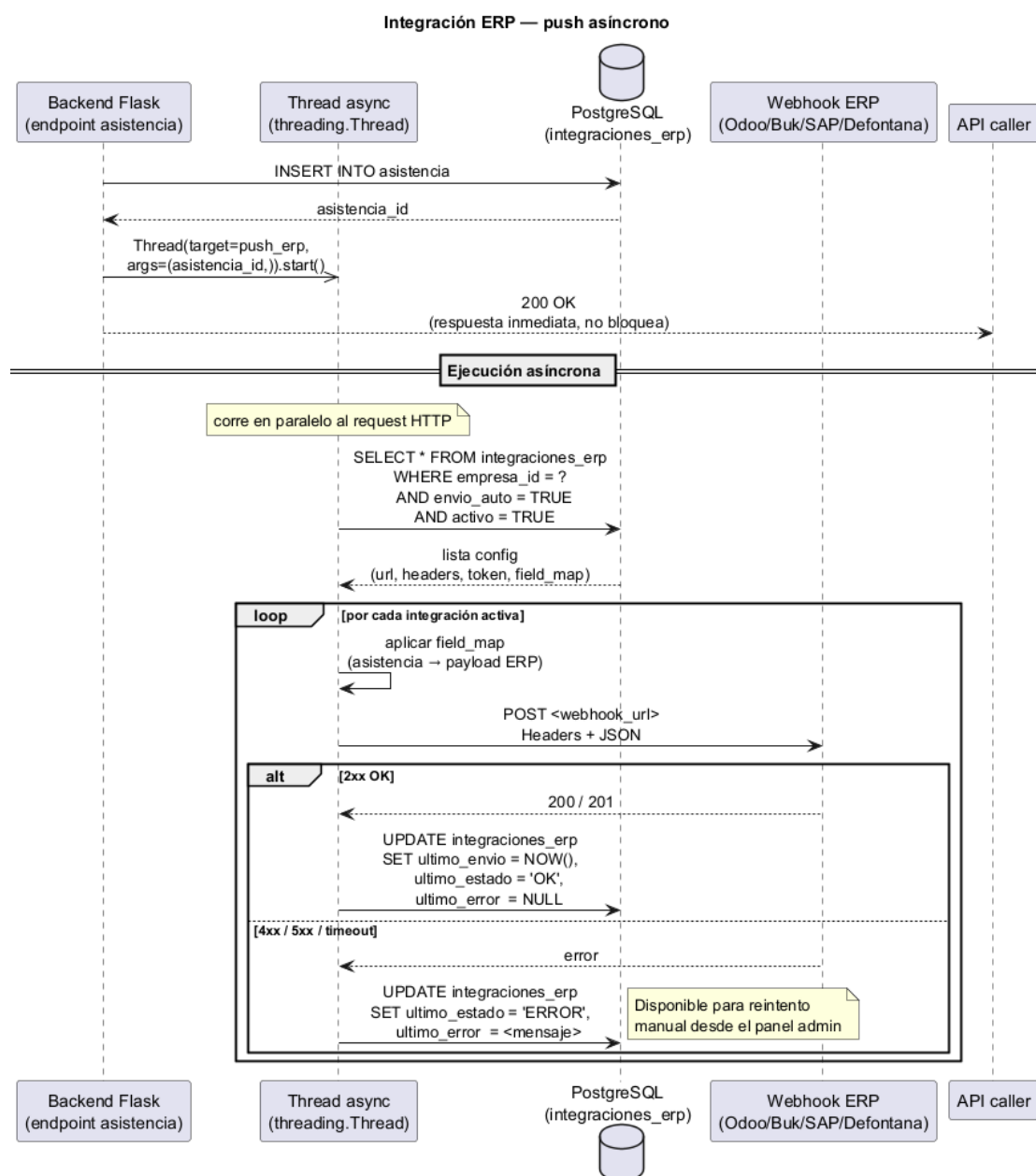


Figura 4.25: Diagrama de secuencia de la integración ERP: push asíncrono con field mapping.

Implementación

Panel web del backend

Para complementar el backend, se desarrolló un panel web independiente (Next.js 16 [70] con React 19 y TypeScript) cuyo módulo principal es la gestión del dispositivo IoT. Este panel, accesible desde cualquier navegador moderno, permite al administrador:

- Visualizar el estado online/offline de cada dispositivo enrolado en tiempo real

mediante tarjetas con código de colores, indicador animado (live-dot) y la IP como enlace clickable con la leyenda “Misma red requerida”.

- Generar el PIN de 8 caracteres necesario para enrolar un nuevo ESP32-CAM.
- Probar la conectividad activa con un dispositivo específico consultando su endpoint /estado (variable NEXT_PUBLIC_DEVICE_BASE_URL, por defecto `http://192.168.4.1`).
- Renombrar y eliminar dispositivos registrados.
- Acceder a los logs de sincronización generados por el dispositivo.
- Acceder a las funcionalidades complementarias de gestión: personas, turnos, asignaciones, asistencias, integración ERP, usuarios y empresas.

El estado en tiempo real se obtiene mediante dos mecanismos complementarios. El principal es un hook `useDeviceWebSocket` que utiliza `EventSource` para conectarse directamente al endpoint SSE del backend (`http://localhost:5000/sse/devices`), recibiendo eventos con el id, nombre, IP, estado y timestamp del último heartbeat de cada dispositivo. Como respaldo, un temporizador ejecuta una consulta REST cada 15 segundos (`GET /api/dispositivos`) para actualizar el estado si la conexión SSE se interrumpe. El estado online se calcula verificando que el campo estado sea 'activo' y que el `ultimo_heartbeat` esté dentro de los últimos 5 minutos; si alguna condición falla, el dispositivo se considera offline.

El panel web se comunica con el backend Flask mediante un proxy interno (`app/api/_proxy.ts`) que reenvía las peticiones REST a la API, evitando problemas de CORS. La conexión SSE, por su parte, se realiza directamente al backend en el puerto 5000 sin pasar por el proxy, ya que requiere una conexión persistente `text/event-stream`. La aplicación está protegida por un middleware de autenticación que valida el token JWT y restringe el acceso según el rol del usuario (admin, empleador, trabajador). El frontend se compila con Docker en un build multi-etapa usando output: 'standalone' para producir una imagen optimizada lista para producción.

Para mantener la consistencia visual entre el panel web y la interfaz embebida del ESP32-CAM, las 9 vistas HTML del dispositivo (`index.html`, `register.html`, `gestion.html`, `personas.html`, `turnos.html`, `asignaciones.html`, `asistencias.html`, `logs.html` y `wifi-setup.html`) fueron actualizadas para adoptar la misma paleta oscura, tipografía y sistema de componentes del panel SAS. Se utilizaron variables CSS para la paleta cromática, diseño basado en tarjetas (cards) para la disposición del contenido, y componentes reutilizables para botones, tablas y formularios, logrando que ambas interfaces presenten una experiencia de usuario unificada.

Adicionalmente, el panel web incorpora las siguientes funcionalidades complementarias:

- **Auto-registro de empresas:** La pantalla de inicio de sesión incluye un modo de registro que permite a nuevas empresas crear su cuenta directamente, invocando al endpoint público POST `/api/auth/register-company`. El formulario solicita nombre de la empresa y datos del administrador de la empresa (nombre, email, contraseña), y al enviarlo crea la organización, el usuario administrador y retorna un token JWT para acceso inmediato.
- **Captura facial por webcam:** El formulario de registro facial en el panel web utiliza la API `navigator.mediaDevices.getUserMedia()` para capturar fotografías directamente desde la cámara del navegador, eliminando la necesidad de subir archivos desde el sistema de archivos local. La interfaz permite visualizar la vista previa en vivo, capturar, previsualizar y confirmar o repetir la toma, con liberación adecuada del stream al cancelar.
- **Edición de usuarios del sistema:** Los administradores y empleadores pueden editar los datos de los usuarios del sistema (nombre, email, rol, contraseña, estado activo/inactivo) mediante un formulario modal que reutiliza la interfaz de registro en modo edición. Las actualizaciones son parciales: solo se envían los campos modificados.

Endpoint de configuración ERP para el ESP32-CAM

Para que el ESP32-CAM pueda notificar asistencias directamente a los ERP configurados (incluso si el backend central no está disponible), se implementó el endpoint GET `/api/dispositivos/erp-config`, que retorna la lista de integraciones ERP activas con envío automático habilitado. El ESP32-CAM consulta este endpoint cada hora y almacena la configuración en `/erp-config.json`, permitiéndole enviar marcaciones a los ERP incluso durante desconexiones parciales.

Contraseñas para autenticación de dispositivos

Para reforzar la seguridad en la comunicación entre el backend y los ESP32-CAM enrolados, se implementó un sistema de contraseñas de dispositivo que permite al administrador generar, gestionar y verificar credenciales específicas para cada terminal IoT. El flujo contempla cuatro operaciones:

- **Generar contraseña** (POST `/api/dispositivos/<id>/generar-password`): El administrador solicita una nueva contraseña de 12 caracteres alfanuméricos, generada mediante `secrets.choice()`. El backend almacena el hash SHA256 de la

contraseña, la versión en texto plano (temporal) y marca la contraseña como pendiente (`password_pendiente = TRUE`). La respuesta incluye la contraseña en texto plano para que el administrador la proporcione al dispositivo.

- **Verificar contraseña pendiente** (GET `/api/dispositivos/check-password`): El ESP32-CAM, al iniciar, consulta este endpoint enviando su MAC address como header `X-Device-MAC`. Si hay una contraseña pendiente, el backend la retorna en texto plano para que el dispositivo la aplique localmente.
- **Confirmar contraseña** (POST `/api/dispositivos/confirmar-password`): Una vez que el dispositivo ha aplicado la contraseña localmente, confirma al backend, que limpia el texto plano (`password_plain = NULL`) y desmarca el estado pendiente. A partir de este momento, solo el hash SHA256 permanece almacenado.
- **Eliminar contraseña** (DELETE `/api/dispositivos/<id>/password`): El administrador puede eliminar la contraseña de un dispositivo, limpiando el hash y el texto plano.

En el panel web, la sección de gestión de dispositivos incorpora botones para generar, regenerar y eliminar contraseñas, así como indicadores visuales del estado de la contraseña (tiene o no tiene, pendiente o confirmada).

Módulo de integración ERP

El módulo `routes/erp.py` implementa:

- **enviar_asistencia_a_erps()**: Función central que itera sobre las integraciones ERP activas de una empresa, transforma los datos de la asistencia según el field mapping configurado, y envía una solicitud POST al webhook con los headers personalizados. Registra el resultado (ok o error) en la tabla `integraciones_erp` (columnas `ultimo_envio` y `ultimo_estado`).
- **__disparar_erp_push()**: Wrapper que ejecuta `enviar_asistencia_a_erps()` en un hilo daemon separado, evitando que la latencia de los ERP externos afecte el tiempo de respuesta al ESP32-CAM. Esta función se invoca automáticamente desde `create_asistencia()` y `sync_asistencias()` cada vez que se registra una asistencia.
- **CRUD de integraciones**: Endpoints REST para crear, listar y eliminar configuraciones ERP.
- **Test de conectividad** (POST `/api/erp/<id>/test`): Envía un payload de prueba con datos ficticios al webhook configurado y retorna el código de respuesta y el cuerpo recibido.

- **Envío manual** (POST /api/erp/<id>/enviar): Envía las últimas 200 asistencias de la empresa al ERP seleccionado, útil para cargas históricas o reprocesamiento.
- **Estado de integración** (GET /api/erp/<id>/estado): Retorna el último envío y su estado.

Control remoto de dispositivos ESP32

Se implementaron tres endpoints en routes/dispositivos.py que permiten controlar remotamente los ESP32-CAM enrolados mediante comandos MQTT:

- **Registrar huella** (POST /api/dispositivos/<id>/registrar-huella): Recibe un persona_id y envía al dispositivo el comando para iniciar el enrolamiento de una huella digital. Verifica que el dispositivo exista, que pertenezca a la empresa del usuario autenticado, y que la persona exista en la misma empresa. Utiliza enviar_comando_dispositivo(mac, 'registrar_huella') para publicar el comando en backend/comando/<MAC>/registrar_huella. El resultado del enrolamiento se recibe luego por el tópico esp32/huella/resultado/#.
- **Reiniciar dispositivo** (POST /api/dispositivos/<id>/reiniciar): Envía el comando reiniciar al ESP32-CAM, provocando un reinicio remoto del dispositivo. Útil para recuperar un dispositivo que presenta fallos de conectividad o para aplicar cambios de configuración. Si el cliente MQTT no está disponible, retorna HTTP 503.
- **Reconectar Wi-Fi** (POST /api/dispositivos/<id>/wifi-reconnect): Envía el comando wifi-reconnect para que el dispositivo reinicie su conexión Wi-Fi. Permite recuperar un dispositivo que ha perdido la conectividad sin necesidad de intervención física.

Los tres endpoints están protegidos con "@requiere_rol('admin', 'empleador')" y verifican que el dispositivo pertenezca a la empresa del usuario autenticado. Los comandos se publican en el tópico backend/comando/<MAC>/<comando> mediante la función enviar_comando_dispositivo() del módulo mqtt_handler.py.

Pruebas

Se diseñaron las siguientes pruebas para validar el panel web para la gestión del dispositivo y la integración ERP:

- **Prueba de streaming SSE:** Conectar el navegador a <http://localhost:5000/sse/devices> y verificar que se reciban eventos text/event-stream con datos JSON de los dispositivos. Simular un heartbeat desde un

ESP32-CAM y verificar que el evento SSE refleje el cambio a estado activo. Desconectar el dispositivo y verificar que el evento SSE refleje el estado inactivo.

- **Prueba de fallback polling:** Interrumpir la conexión SSE (deteniendo el servidor backend momentáneamente). Verificar que el panel web continúe mostrando el estado de los dispositivos mediante las consultas REST periódicas (cada 15 segundos) y que al restaurar la conexión SSE se retome el streaming.
- **Prueba de indicador visual de dispositivo:** Verificar que cada tarjeta de dispositivo en el panel web muestre: la dirección IP como enlace clickable que abre `http://<ip>/` en una nueva pestaña, un indicador animado verde/rojo (live-dot) que refleje el estado online/offline, y la leyenda “Misma red requerida” como sugerencia contextual.
- **Prueba de navegación en panel web:** Acceder a las rutas principales del backend (`/health`, `/api/personas`, `/api/asistencias`) sin autenticación y verificar que respondan correctamente (HTTP 200 con acceso público o HTTP 401 cuando se requiera token).
- **Prueba de creación de integración ERP:** Autenticarse como empleador y enviar POST `/api/erp` con nombre, tipo, webhook URL (usando un servidor de prueba como `https://webhook.site`) y field mapping. Verificar que la integración se cree con `activo = TRUE` y `envio_auto = TRUE`.
- **Prueba de test de webhook:** Ejecutar POST `/api/erp/<id>/test` para la integración creada. Verificar que el webhook de prueba reciba un payload JSON con datos ficticios (`rut = '11.111.111-1'`, `nombre = "Test Usuario"`, `tipo = "entrada"`). Verificar que el estado se actualice en la base de datos.
- **Prueba de envío automático:** Registrar una asistencia real (POST a `/api/asistencias`) para una persona de la empresa configurada. Verificar que el servidor de prueba del webhook reciba automáticamente el payload con los datos de la asistencia.
- **Prueba de field mapping:** Configurar un ERP con field mapping `{"rut": "employee_id", "tipo": "event"}`. Registrar una asistencia y verificar que el webhook reciba el payload con los campos renombrados (`employee_id` en lugar de `rut`, `event` en lugar de `tipo`).
- **Prueba de envío asíncrono:** Medir el tiempo de respuesta del endpoint POST `/api/asistencias` cuando hay un ERP configurado cuyo webhook tarda 5 segundos

en responder. Verificar que la respuesta al cliente sea inmediata (menor a 200 ms) y que el envío al ERP ocurra en segundo plano.

- **Prueba de tolerancia a fallos ERP:** Configurar un webhook que retorne HTTP 500. Registrar una asistencia y verificar que el sistema no falle; el error debe quedar registrado en ultimo_estado de la integración y la asistencia debe persistirse correctamente en la base de datos.
- **Prueba de sincronización de configuración ERP al ESP32-CAM:** Simular la consulta GET /api/dispositivos/erp-config desde el ESP32-CAM (con header X-Device-MAC). Verificar que retorne solo las integraciones activas con envío automático de la empresa correspondiente.
- **Prueba de envío manual por lote:** Ejecutar POST /api/erp/<id>/enviar y verificar que se envíen correctamente los registros de asistencia recientes al webhook del ERP.

Refinamientos posteriores

- **Reasignación de dispositivos entre empresas:** se agregó el endpoint PUT /api/dispositivos/<id>/reasignar, restringido al rol admin, que permite mover un dispositivo ya enrolado de una empresa a otra actualizando su columna empresa_id. Esto resuelve el caso de dispositivos físicos que cambian de destino (por ejemplo, al reasignar hardware entre sucursales o clientes) sin requerir un nuevo proceso de enrolamiento con PIN.
- **Zona horaria de Chile en la integración ERP:** los timestamps enviados a los webhooks ERP se normalizan a America/Santiago mediante el módulo estándar zoneinfo, con conversión explícita antes de formatear cada fecha_hora y timestamp del payload. Esto evita ambigüedades cuando el servidor del backend opera en UTC y el sistema ERP de destino espera horarios locales.
- **Soporte de múltiples fotos en el panel web:** el panel de administración construido con Next.js incorpora en la vista de cada persona la visualización de las fotos de referencia asociadas a sus distintos encodings_faciales, reflejando en la interfaz la capacidad de multi-encoding ya descrita en la Sección 2.3.10. Asimismo, en los despliegues multi-empresa la interfaz incorpora una indicación visual de la empresa a la que pertenece cada registro, relevante para el rol admin, que puede ver datos de más de una organización.

4.2.8 Iteración 8: Sincronización diferida, logs y cierre del proyecto

Esta iteración integra la sincronización diferida entre el ESP32-CAM y el backend, la capa de auditoría y logs del sistema, y las herramientas de diagnóstico.

Análisis

Para que el prototipo sea viable en entornos reales, debe garantizar que ningún dato se pierda durante períodos de desconexión y que todas las operaciones críticas queden registradas para auditoría. Los requisitos identificados son:

- **Sincronización diferida de asistencias:** Los registros de asistencia generados offline (marcados con `sincronizado = false`) deben enviarse automáticamente al backend cuando se restablezca la conectividad. El proceso debe ser idempotente: si un registro ya existe en el backend, no debe duplicarse.
- **Sincronización diferida de entidades:** Las personas, turnos y asignaciones creados localmente durante el modo offline deben sincronizarse con el backend, resolviendo conflictos de identificadores (IDs locales con prefijo local- versus IDs del backend).
- **Sincronización de configuración ERP:** La configuración de integraciones ERP debe poder descargarse desde el backend para que el ESP32-CAM pueda enviar datos de asistencia directamente a los ERP configurados.
- **Logs de sincronización:** Cada operación de sincronización debe registrarse en la base de datos con el conteo de registros enviados y procesados correctamente, permitiendo auditoría posterior.
- **Logs biométricos:** Todas las operaciones de registro, verificación e identificación facial deben quedar registradas con timestamp, resultado e IP de origen, para cumplimiento normativo.
- **Watchdog de dispositivos:** El backend debe monitorear continuamente el estado de los dispositivos enrolados, detectando aquellos que han dejado de comunicarse y marcándolos como inactivos.
- **Herramientas de diagnóstico:** Se requiere un script de prueba que permita simular asistencias faciales usando fotos almacenadas, facilitando las pruebas del sistema sin necesidad del hardware.

Diseño

Proceso de sincronización diferida

El ESP32-CAM ejecuta la sincronización en dos momentos:

1. **Al iniciar:** Si hay conectividad, llama a `sincronizarPendientes()` que ejecuta en secuencia:
 - `sincronizarTurnosPendientes()`: Envía los turnos con `sincronizado = false` al backend vía POST, y actualiza sus `backend_id`.
 - `sincronizarAsignacionesPendientes()`: Similar para asignaciones, dependiendo de que los turnos referenciados ya tengan `backend_id`.
 - `sincronizarAsistencias()`: Envía un lote de asistencias con `sincronizado = false` vía POST `/api/asistencias/sync`.
2. **Periódicamente:** Cada 5 minutos (300,000 ms), se ejecuta `sincronizarPendientes()` en el bucle principal.

Además, el ESP32-CAM consulta GET `/api/dispositivos/erp-config` cada hora (360,000 ms) para mantener actualizada su configuración local de webhooks ERP.

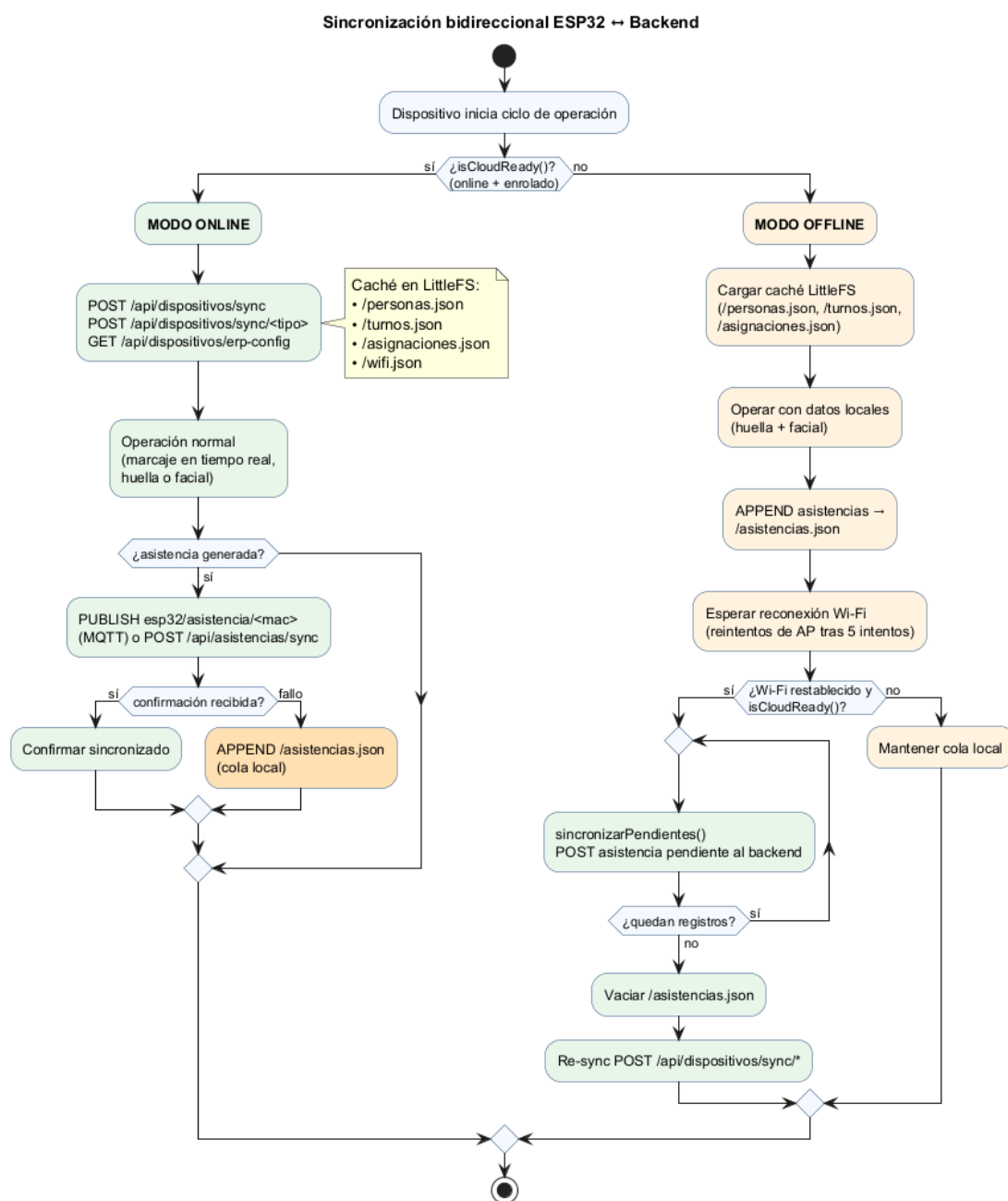


Figura 4.26: Diagrama de actividad: sincronización bidireccional entre los modos online y offline del ESP32-CAM.

Deduplicación en el backend

El endpoint `POST /api/asistencias/sync` implementa una verificación previa a la inserción: para cada registro del lote, consulta si ya existe una asistencia de la misma persona (`persona_id`), mismo tipo (`entrada/salida`), mismo turno (`turno_id`) y misma fecha (`DATE(fecha_hora) = CURRENT_DATE`). Si existe, omite la inserción. Esta verificación por `turno_id` permite que un trabajador registre múltiples asistencias en

un mismo día si pertenece a distintos turnos (por ejemplo, jornada diurna y jornada nocturna), sin generar duplicados dentro del mismo turno. Garantiza idempotencia ante reenvíos.

Tablas de auditoría

- `sincronizacion_log`: Registra cada operación de sincronización con: dispositivo de origen, cantidad de registros enviados, cantidad procesada correctamente, estado y detalle.
- `logs_biometricos`: Registra cada operación biométrica con: persona, dispositivo, timestamp, tipo de operación (registro, verificación, identificación), resultado (éxito, fallo, duplicado, no_encontrado) e IP de origen.
- `eliminaciones_biometricas`: Registra cada eliminación de datos biométricos con: persona, embedding anterior (cifrado), ruta de la foto eliminada y usuario solicitante.

Panel de visualización de logs

El backend expone dos conjuntos de endpoints para consulta de logs:

- `GET /api/logs`: Lista los registros de sincronización, con filtro por empresa para roles no-admin.
- `DELETE /api/logs`: Permite limpiar los logs (admin limpia todo, empleador limpia los de su empresa).
- El ESP32-CAM expone `GET /api/logs` que retorna el buffer de logs en formato HTML, y `/api/logs/clear` para limpiarlo.

Script de simulación de asistencia facial

Se desarrolló `deteccion.py`, un script de línea de comandos que permite probar el flujo de reconocimiento facial sin necesidad del ESP32-CAM. El script escanea las fotos almacenadas localmente para el ambiente de pruebas, presenta un menú interactivo para seleccionar una de ellas, ejecuta el proceso de identificación 1:N contra la base de datos y registra la asistencia correspondiente. Es útil para pruebas de regresión rápida y demostraciones.

Implementación

Sincronización en el ESP32-CAM

Las funciones de sincronización siguen un patrón común:

1. Verificar conectividad (`isOnline` y `WiFi.status() == WL_CONNECTED`).
2. Cargar los datos locales desde `LittleFS`.
3. Iterar sobre los registros con `sincronizado = false`.
4. Para cada uno, construir el payload JSON y enviarlo al backend vía HTTP POST.
5. Si el backend responde con éxito (HTTP 200/201), marcar `sincronizado = true` y actualizar el `backend_id` si corresponde.
6. Persistir los cambios en `LittleFS`.

Para las personas, además del envío de pendientes, se implementa la descarga completa desde el backend (`sincronizarPersonasDesdeBackend()`) que sobrescribe el archivo local si no hay registros locales pendientes, manteniendo el ESP32-CAM actualizado con los datos del servidor.

Campo colación en turnos

Los turnos incorporaron los campos `con_colacion` (booleano), `colacion_inicio` (TIME) y `colacion_fin` (TIME) para modelar el horario de colación de los trabajadores, en cumplimiento del artículo 34 del Código del Trabajo chileno. En la tabla turnos de PostgreSQL se almacenan como TIME (o NULL si `con_colacion = FALSE`). Los endpoints CRUD de `routes/turnos.py` gestionan estos campos: al crear un turno, se envían `con_colacion`, `colacion_inicio` y `colacion_fin`; al actualizar, se usa COALESCE para preservar valores existentes si no se envían. Las consultas GET retornan los horarios como string o null según corresponda. El campo `turno_id` se agregó también a la tabla `asistencias` para asociar cada marcación al turno bajo el cual se registró, permitiendo el cálculo preciso de horas trabajadas descontando la colación.

Sincronización de asistencias por lote

La función `sincronizarAsistencias()` en el ESP32-CAM es especialmente relevante porque maneja el caso más frecuente de datos offline: un trabajador marcó entrada y salida durante el día sin conexión. La función:

1. Carga `asistencias.json` y filtra las que tengan `sincronizado = false`.
2. Construye un JSON con estructura `{"registros": [...]}`.
3. Lo envía a POST `/api/asistencias/sync`.
4. Si el backend retorna HTTP 200, marca todos los registros enviados como sincronizados.

Sincronización en el backend

El endpoint POST `/api/asistencias/sync` en `routes/asistencias.py` recibe el lote e itera sobre cada registro, aplicando la verificación de duplicados y realizando la inserción. También dispara el envío a ERP para cada asistencia nueva.

Watchdog y ping activo de dispositivos

La detección de desconexión de dispositivos se implementó con tres capas complementarias:

1. **LWT (instantáneo)**: Al conectarse al broker MQTT, el ESP32-CAM configura un mensaje Last Will en `esp32/lwt/<MAC>`. Si el dispositivo se desconecta abruptamente, el broker publica automáticamente el LWT y el backend marca el dispositivo como inactivo de inmediato.
2. **Ping activo (device_pinger)**: Un hilo separado publica periódicamente (cada 30 segundos) un mensaje en el tópico `esp32/ping/<MAC>`. El ESP32-CAM, al recibirlo, responde publicando un heartbeat con el campo "pong": true. Si el backend no recibe respuesta dentro de 60 segundos, marca el dispositivo como inactivo. Este mecanismo funciona incluso si el mensaje LWT no fue entregado.
3. **Watchdog pasivo (device_watchdog)**: Un hilo ejecutado cada 60 segundos realiza un barrido inicial al arrancar el backend (todos los dispositivos se marcan inactivo) y luego verifica periódicamente qué dispositivos no han enviado heartbeat en más de 90 segundos, marcándolos como inactivos.

Esta arquitectura de tres capas asegura que ninguna desconexión pase desapercibida: el LWT atrapa las desconexiones abruptas detectadas por el broker; el ping activo detecta silencios de red donde el LWT no alcanza a publicarse; y el watchdog pasivo actúa como respaldo final para cualquier caso no cubierto por las capas anteriores.

Herramienta de simulación facial

Como complemento durante el desarrollo se implementó `deteccion.py`, un script de línea de comandos que permitió probar el flujo de reconocimiento facial sin necesidad del ESP32-CAM. El script se conectaba directamente a PostgreSQL, ejecutaba DeepFace (Facenet + detector configurable + anti-spoofing) sobre imágenes almacenadas localmente para el ambiente de pruebas, y registraba asistencias de prueba en la base de datos. En las etapas finales del proyecto, esta funcionalidad de simulación fue absorbida por la suite de pruebas automatizadas (con mocks de DeepFace, OpenCV y PostgreSQL en Docker), que ofrece mayor repetibilidad y cobertura sin dependencia del sistema de archivos local.

Flujo operacional consolidado

Con todos los componentes del sistema ya contruidos (enrolamiento por PIN, reconocimiento biométrico, almacenamiento offline, sincronización diferida e integración ERP), la Fig. 4.27 consolida el camino completo que sigue una marcación, cubriendo tanto el caso con conexión a internet como el caso sin ella, tal como quedó implementado al cierre del proyecto.

El flujo comienza cuando el sensor PIR detecta presencia frente al dispositivo y se ejecuta la captura biométrica (huella mediante el AS608 o rostro mediante la cámara OV2640). A partir de ahí, el firmware evalúa `isCloudReady()`: si el dispositivo está conectado a internet y enrolado, la marcación se envía de inmediato al backend por HTTP (POST `/api/facial/identificar` para rostro, o POST `/api/asistencias` para huella). Si cualquiera de esas dos condiciones falla, la marcación se guarda localmente en `asistencias.json` (LittleFS) con la marca `sincronizado=false`, se muestra el resultado en el panel embebido sin esperar respuesta del servidor, y el dispositivo queda a la espera de que la conexión se restablezca para ejecutar `sincronizarPendientes()`, que vacía la cola completa hacia el backend.

Ambos caminos, el envío inmediato y la sincronización diferida, terminan convergiendo en el mismo punto: el backend valida la huella o el rostro contra los registros almacenados (`encodings_faciales` o el identificador de huella) y persiste el resultado en la tabla `asistencias` de PostgreSQL. Si la identificación es exitosa, el backend responde 200 OK, dispara en un hilo aparte el envío hacia los sistemas ERP configurados, y el firmware ejecuta `flashExito()`, que hace parpadear el LED verde del dispositivo dos veces. Si no hay coincidencia o la imagen no cumple el filtro de calidad, el backend responde con un código de error (404 o 400) y la operación queda registrada en `logs_biometricos` para su auditoría posterior; en este caso el firmware invoca `flashError()`, una función que existe en el código como punto de extensión para una señal de error distinta, pero que actualmente no tiene ninguna acción implementada.



Figura 4.27: Flujo operacional consolidado para el registro de asistencia.

Pruebas

Se diseñaron las siguientes pruebas para validar la sincronización diferida, los logs y la integración final:

- **Prueba de sincronización de asistencias offline:** Desconectar el ESP32-CAM de la red, registrar 5 marcaciones de entrada/salida para 2 personas diferentes, reconectar y ejecutar sincronización. Verificar que los 5 registros aparezcan en la base de datos del backend y que el archivo asistencias.json local los marque como sincronizados.
- **Prueba de idempotencia de sincronización:** Tras una sincronización exitosa, ejecutar nuevamente la sincronización sin nuevos registros. Verificar que no se inserten registros duplicados en el backend.

- **Prueba de sincronización de personas creadas offline:** Desconectar el dispositivo, registrar una nueva persona (con huella y datos), reconectar y sincronizar. Verificar que la persona aparezca en el backend con un ID definitivo (no local-).
- **Prueba de sincronización de turnos y asignaciones:** Desconectar el dispositivo, crear un turno y asignarlo a una persona existente, reconectar y sincronizar. Verificar que el turno y la asignación se repliquen en el backend y que los backend_id locales se actualicen correctamente.
- **Prueba de persistencia tras corte de energía:** Durante una sincronización en curso, cortar la alimentación del ESP32-CAM. Al reiniciar, verificar que los archivos JSON no estén corruptos y que los registros con sincronizado = false sigan estando disponibles para una nueva sincronización.
- **Prueba de duplicados por turno:** Registrar dos asistencias con la misma persona, tipo, turno y fecha. Verificar que el sistema rechace la segunda inserción por duplicado. Registrar asistencias con diferentes turno_id para un mismo día y verificar que ambas se registren sin conflictos.
- **Prueba de colación en turnos:** Crear un turno con colación habilitada (con_colación = true) especificando horario de colación. Verificar que los campos colacion_inicio y colacion_fin se almacenen correctamente como TIME en la base de datos. Crear un turno sin colación y verificar que ambos campos sean NULL.
- **Prueba de logs de sincronización:** Ejecutar 3 sincronizaciones con diferentes cantidades de registros. Verificar que en la tabla sincronizacion_log del backend aparezcan 3 entradas con los conteos correctos de registros enviados y procesados.
- **Prueba de logs biométricos:** Realizar operaciones de registro facial, identificación exitosa, identificación fallida y verificación. Verificar que cada operación genere una entrada en logs_biometricos con el tipo de operación y resultado correctos.
- **Prueba de watchdog:** Iniciar el backend, verificar que todos los dispositivos se marquen como inactivos. Conectar un ESP32-CAM, verificar que tras el primer heartbeat cambie a activo. Desconectar el ESP32-CAM y verificar que tras 90 segundos el watchdog lo marque como inactivo.
- **Prueba de ping activo:** Conectar un ESP32-CAM, verificar que el backend publique pings en esp32/ping/<MAC> y que el dispositivo responda. Detener el ESP32-CAM (corte de red) y verificar que el ping activo marque el dispositivo

como inactivo antes de que el watchdog de 90 segundos actúe (tiempo de espera de 60 segundos).

- **Prueba de simulación facial automatizada:** Utilizar la suite de pruebas `test_routes_facial.py` y el emulador `test_identificacion_facial.py` para simular el flujo completo de identificación 1:N con imágenes sintéticas generadas en memoria, cubriendo casos de coincidencia exitosa, rostro no registrado y calidad insuficiente de imagen.
- **Prueba de flujo completo punta a punta:** Ejecutar el flujo completo: registrar persona con huella y rostro en el ESP32-CAM → sincronizar con backend → verificar que la persona aparezca en el panel web → marcar asistencia por huella offline → reconectar y sincronizar → verificar que la asistencia aparezca en el panel web y se envíe al ERP de prueba.

Pruebas automatizadas

Además de las pruebas funcionales descritas, se implementó una suite de más de 590 pruebas automatizadas para el backend y el firmware, utilizando `pytest` con PostgreSQL en Docker y mocks para DeepFace, OpenCV y MQTT. La suite se organiza en 35 archivos de prueba y alcanza una cobertura de código del 98% sobre el código de producción del backend, sin fallos en ninguna de las pruebas (ver Anexo D para el detalle de casos y la evolución de la cobertura por módulo). La integración con los ERP externos se valida en dos niveles: por un lado, las pruebas de la suite automatizada cubren la creación, edición y prueba de webhooks de una integración ERP, simulando la respuesta del servidor externo. Por otro lado, dentro del subdirectorio `ERPSIMULATORS/` se construyeron tres simuladores independientes (`mock_odoo.py`, `mock_defontana.py` y `mock_buk.py`), cada uno levantando un pequeño servidor Flask que reproduce los endpoints reales de autenticación, consulta de empleados e inyección de marcaciones de su respectivo ERP comercial, según la documentación pública de cada proveedor. El script `test_erp_mock.py` se ejecuta por separado, con los tres simuladores corriendo: envía una marcación con el formato que genera el sistema hacia cada uno de los tres mocks y verifica que el field mapping configurado para cada ERP transforme correctamente los datos y que cada simulador responda como lo haría el sistema real, imprimiendo un reporte de éxito o falla por caso. Esta validación manual complementa, sin sustituir, a las pruebas automatizadas de `pytest`.

Las pruebas cubren los flujos de todas las iteraciones: inicialización de la aplicación, esquema de base de datos, cifrado de embeddings, servicio de correo electrónico (con mock de SMTP), handlers MQTT, autenticación JWT con roles jerárquicos y flujo multi-empresa, endpoints faciales, CRUD de personas/turnos/asignaciones/asistencias,

verificación de dispositivos, integraciones ERP con tolerancia a fallos de conexión y timeout, enrolamiento de dispositivos mediante PIN, sincronización offline, y logs de auditoría. Entre los módulos con mayor cobertura destacan el cifrado de embeddings (100%), el servicio de correo (100%), la gestión de turnos (100%), la base de datos (99%) y el módulo de asignaciones (96%). El detalle completo de los casos de prueba se incluye en el Anexo D, que además resume los resultados obtenidos al ejecutar la suite completa.

Refinamientos posteriores

- **Limpieza de datos al eliminar una persona:** el endpoint de eliminación distingue dos comportamientos según el rol de quien lo invoca. Un administrador ejecuta una limpieza completa de datos biométricos y sensibles: anonimiza el RUT, el correo y el identificador de huella, elimina los encodings_faciales y los consentimientos asociados, borra la foto de previsualización del disco y marca a la persona como inactiva, conservando únicamente su historial de asistencias. Un empleador, en cambio, solo puede marcar a la persona como inactiva, sin acceso a la limpieza de datos biométricos, reservada al rol con mayor privilegio.
- **Filtro activo = true en los listados:** todas las consultas de lectura sobre personas (y entidades equivalentes) incorporan la condición WHERE activo = true, de modo que los registros eliminados mediante soft-delete dejan de aparecer en listados y búsquedas sin perder su información histórica en la base de datos, requisito necesario para mantener la integridad de los reportes de asistencia ya generados.

4.3 Mecanismos de seguridad

Esta sección consolida los mecanismos de seguridad implementados a lo largo de las distintas iteraciones, descritos en su momento dentro del desarrollo de cada una (Secciones 4.2.3 a 4.2.5), agrupándolos por capa: autenticación y control de acceso, enrolamiento de dispositivos, y protección de datos biométricos.

4.3.1 Autenticación y control de acceso

El backend define tres decoradores de autenticación en routes/auth.py, aplicados según el tipo de cliente que consume cada endpoint:

- @token_required: exige un header Authorization: Bearer <token> con un JWT firmado con el algoritmo HS256 y una clave secreta (JWT_SECRET,

configurable por variable de entorno). El token incluye `user_id`, `empresa_id`, `rol` y, opcionalmente, `persona_id`, con una expiración de 24 horas. Si el token falta, está expirado o es inválido, el endpoint responde con HTTP 401.

- `@requiere_rol(*roles)`: envuelve a `@token_required` y además exige que `request.user_rol` esté dentro de los roles permitidos (admin, empleador o trabajador), respondiendo HTTP 403 en caso contrario. Implementa así un control de acceso jerárquico basado en roles sobre los mismos endpoints autenticados.
- `@token_opcional`: permite que un mismo endpoint sea consumido tanto por un cliente web autenticado con JWT como por un dispositivo ESP32-CAM identificado mediante el header X-Device-MAC. Si no hay JWT, el decorador busca la MAC recibida entre los dispositivos ya enrolados y, de encontrarla, expone `request.dispositivo_id` y `request.empresa_id` para el resto del endpoint. Como se detalla en la Sección 4.2.5, este decorador ya no crea dispositivos nuevos automáticamente: solo vincula los que ya pasaron por el flujo de enrolamiento con PIN.
- `@requiere_dispositivo_enrolado`: complementa a `@token_opcional` en endpoints biométricos sensibles (por ejemplo, `/api/facial/registrar`), verificando explícitamente que el dispositivo de la petición tenga `enrolado = TRUE` en la base de datos antes de continuar, y respondiendo HTTP 403 en caso contrario.

Las contraseñas de los usuarios del panel web nunca se almacenan en texto plano: se procesan con `bcrypt` [55] (función `hashpw` con sal generada por `gensalt()`) tanto al crear una cuenta como al cambiar la contraseña, y la verificación en el login usa `bcrypt.checkpw` para comparar contra el hash almacenado.

4.3.2 Enrolamiento seguro de dispositivos

Un dispositivo ESP32-CAM no puede operar contra el backend simplemente conectándose a la red: debe pasar por un flujo de enrolamiento explícito basado en un PIN de un solo uso. Un administrador o empleador solicita `POST /api/auth/dispositivos/generar-pin`, que crea en la base de datos un registro en `dispositivos` con `enrolado = FALSE` y un código aleatorio de 8 caracteres alfanuméricos (generado con el módulo `secrets`, criptográficamente seguro). El dispositivo, desde su panel de configuración Wi-Fi, envía ese PIN junto con su dirección MAC al backend; si el PIN es válido y no fue usado antes, el dispositivo queda marcado como `enrolado = TRUE` y asociado a la MAC y a la empresa que generó el PIN. Un PIN ya consumido no puede reutilizarse (HTTP 409).

En el firmware, la función `isCloudReady()` (Sección 4.2.1) actúa como una verificación de doble factor antes de cualquier operación dependiente del backend: exige tanto conectividad de red (`isOnline`) como enrolamiento confirmado (`estaEnrolado`), evitando que un dispositivo conectado pero no autorizado, o autorizado pero sin red, ejecute operaciones que comprometerían la consistencia del sistema.

4.3.3 Protección de datos biométricos

Los embeddings faciales generados por DeepFace se cifran antes de almacenarse en la base de datos mediante Fernet (AES-128 en modo CBC con autenticación HMAC-SHA256), de la librería `cryptography` de Python. La clave de cifrado no se almacena directamente: se deriva aplicando SHA-256 a una variable de entorno (`BIOMETRIC_KEY`) y codificando el resultado en base64 URL-safe, tal como lo exige la API de Fernet:

```
def _derivar_fernet_key() -> bytes:
    digest = hashlib.sha256(BIOMETRIC_KEY.encode("utf-8")).digest()
    return base64.urlsafe_b64encode(digest)
```

De esta forma, ni el embedding ni la clave de cifrado quedan expuestos en texto plano en la base de datos: un acceso no autorizado a las tablas `encodings_faciales` solo expone valores cifrados, mientras la clave reside exclusivamente en la configuración del entorno de ejecución del backend.

4.4 Arquitectura de tiempo real (SSE)

El panel web necesita reflejar, sin recargar la página, el estado de conexión de cada ESP32-CAM y el progreso del enrolamiento de huellas. Esta sección consolida cómo se implementó esa capa de tiempo real, descrita parcialmente al introducirse en las iteraciones 3 y 7 (Secciones 4.2.3 y 4.2.5).

4.4.1 Endpoints de streaming

El backend expone dos endpoints SSE (Server-Sent Events) implementados en `app.py` con el patrón estándar de Flask para streaming: una función generadora que produce eventos en formato `text/event-stream`, devuelta mediante `Response(stream_with_context(generate()), mimetype='text/event-stream')`.

- GET `/sse/devices`: difunde, a todos los clientes conectados, los cambios de estado de los dispositivos (heartbeat recibido, LWT disparado, ping sin respuesta), mediante la función `broadcast_device_update()`.

- **GET /sse/huellas**: difunde el resultado del enrolamiento de huellas recibido por MQTT en `esp32/huella/resultado/#`, mediante `broadcast_huella_update()`, permitiendo que el panel muestre en vivo el progreso del registro biométrico.

Se optó por SSE porque el caso de uso es unidireccional (servidor a cliente) y se consume en el navegador con la API nativa `EventSource`, sin depender de librerías adicionales en el frontend ni de un protocolo de negociación propio como el de `WebSockets`.

4.4.2 Tres capas de detección de desconexión

Determinar si un ESP32-CAM sigue conectado es más complejo que solo esperar sus heartbeats, porque una caída de red puede impedir que el dispositivo avise que se desconectó. El backend combina tres mecanismos complementarios, implementados en `mqtt_handler.py`:

1. **LWT (Last Will and Testament)**: al conectarse al broker MQTT, el ESP32-CAM registra un mensaje LWT en `esp32/lwt/<MAC>`. Si la conexión TCP se rompe de forma anómala (corte de energía, pérdida abrupta de señal), el propio broker Mosquitto publica ese mensaje automáticamente, sin que el backend tenga que esperar ningún timeout. Es la capa más rápida, pero depende de que el broker detecte la caída de la conexión subyacente.
2. **Ping activo (device_pinger)**: un hilo en segundo plano publica, cada 30 segundos, un mensaje en `esp32/ping/<MAC>` para cada dispositivo enrolado. El ESP32-CAM responde con un heartbeat normal al recibirlo. Si un dispositivo no responde dentro de la ventana definida por `HEARTBEAT_TIMEOUT` (60 segundos), el pinger lo marca como inactivo directamente, sin esperar al watchdog. Esta capa cubre los casos en que el LWT no llega a publicarse (por ejemplo, si el broker tampoco detecta la caída).
3. **Watchdog pasivo (device_watchdog)**: un segundo hilo revisa cada 60 segundos la antigüedad del último heartbeat de cada dispositivo y marca como inactivos a los que superan los 60 segundos sin responder. Actúa como respaldo final, independiente del pinger, y además ejecuta un barrido inicial al arrancar el backend que marca a todos los dispositivos como inactivos hasta recibir su primer heartbeat.

Cada vez que cualquiera de estas tres capas cambia el estado de un dispositivo en la base de datos, invoca `broadcast_device_update()`, que propaga el cambio de inmediato a los clientes SSE conectados.

4.4.3 Consumo en el frontend

En el panel web (Next.js), un hook personalizado se conecta directamente al endpoint `/sse/devices` mediante EventSource, actualizando en memoria el estado, IP y último heartbeat de cada dispositivo a medida que llegan eventos, sin necesidad de refrescar la página ni de mantener una conexión WebSocket. Como mecanismo de respaldo ante una eventual interrupción de la conexión SSE, el frontend además consulta periódicamente `GET /api/dispositivos` por REST, de modo que el estado mostrado no depende exclusivamente del canal de streaming.

4.5 Resumen del capítulo

El capítulo 4 presenta el desarrollo completo del prototipo a lo largo de ocho iteraciones, siguiendo la metodología de prototipado evolutivo incremental. Se documenta la integración del hardware (ESP32-CAM, lector AS608, sensor PIR, cámara OV2640), la implementación del almacenamiento local con LittleFS y JSON, el desarrollo del backend con Flask y PostgreSQL (14 tablas con soporte multi-tenant), la comunicación mediante HTTP y MQTT, el reconocimiento facial con DeepFace (modelo Facenet, detector configurable MTCNN/RetinaFace y anti-spoofing), el cifrado de embeddings biométricos con Fernet, la autenticación JWT con roles jerárquicos, el enrolamiento seguro de dispositivos mediante PIN, el sistema antifraude multicapa (PIR + flash PWM + anti-spoofing + cooldown), el panel web para la gestión del dispositivo con streaming SSE de estado en tiempo real, la integración con ERP externos vía webhooks con field mapping, los mecanismos de sincronización diferida con logs de auditoría, y la detección de desconexión mediante un sistema de tres capas (LWT, ping activo y watchdog pasivo).

Cada iteración incluyó su respectivo análisis de requisitos, diseño de la solución, implementación detallada y un plan de pruebas de integración para validar el correcto funcionamiento de los componentes desarrollados. Adicionalmente, se documenta la suite de 591 pruebas unitarias automatizadas con pytest, que alcanzó una cobertura de código del 98% sobre las 3.382 líneas de producción del backend, incluyendo pruebas de caja negra sobre los endpoints de la API REST y verificación exhaustiva de todos los caminos de error.

El Anexo C presenta, mediante diagramas de actividades, las principales funcionalidades implementadas en el sistema: el enrolamiento de dispositivos (Figura 6.3), el registro de personas y captura biométrica (Figura 6.4), la gestión de turnos y su asignación (Figura 6.5), la sincronización diferida de asistencias pendientes (Figura 6.6) y el panel web administrativo (Figura 6.7).

5. Análisis de resultados

En el siguiente capítulo se presentan los resultados obtenidos tras la implementación y evaluación del prototipo, organizados según las dimensiones definidas en la metodología de evaluación del capítulo 3. Se analiza el costo final del prototipo, los puntajes de usabilidad SUS obtenidos de usuarios técnicos y no técnicos, las métricas de desempeño del reconocimiento facial y los resultados de las pruebas de estrés offline, con el objetivo de determinar la viabilidad técnica y operativa del sistema propuesto.

5.1 Costo total del prototipo

La Tabla 5.1 desglosa los costos de los componentes utilizados en el prototipo final, considerando precios de mercado local e importación según corresponda.

Componente	Cantidad	Costo (CLP)
ESP32-CAM (cámara OV2640 + microcontrolador)	1	7.500
Lector de huellas AS608	1	11.000
Sensor PIR HC-SR501	1	1.500
Fuente de alimentación 5V / 2A	1	4.000
Cables y conectores Dupont	1 kit	2.000
Placa de pruebas (protoboard)	1	3.000
Carcasa / soporte impreso en 3D	1	5.000
Total		34.000

Cuadro 5.1: Desglose de costos del prototipo.

5.2 Resultados de encuestas SUS

Se aplicó el cuestionario System Usability Scale estándar a dos grupos de usuarios: *no técnicos* (trabajadores y empleadores) y *técnicos* (desarrolladores e integradores ERP). Ambos grupos respondieron las mismas 10 preguntas en escala Likert de 5 puntos (1 =

Totalmente en desacuerdo, 5 = Totalmente de acuerdo). La puntuación SUS se calcula según el método estándar descrito por Brooke [9]: para cada ítem impar se resta 1, para cada ítem par se resta de 5, y el resultado se multiplica por 2.5, obteniendo un valor entre 0 y 100.

5.2.1 Usuarios no técnicos

Los resultados obtenidos para este grupo se presentan en la Tabla 5.2.

N.º	Pregunta	Promedio	Tipo
1	Creo que me gustaría usar este sistema con frecuencia.	4.8	Positiva
2	Encontré el sistema innecesariamente complejo.	1.5	Invertida
3	Pensé que el sistema era fácil de usar.	5.0	Positiva
4	Creo que necesitaría apoyo de una persona técnica para poder usar este sistema.	1.7	Invertida
5	Consideré que las funciones del sistema estaban bien integradas.	4.3	Positiva
6	Pensé que había demasiada inconsistencia en el sistema.	1.2	Invertida
7	Imagino que la mayoría de las personas aprenderían a usar este sistema rápidamente.	5.0	Positiva
8	Encontré el sistema muy engorroso de usar.	1.8	Invertida
9	Me sentí seguro al usar el sistema.	4.0	Positiva
10	Necesité aprender muchas cosas antes de poder empezar a usar este sistema.	1.2	Invertida
Puntuación SUS total			89.6

Cuadro 5.2: Resultados de encuesta SUS, usuarios no técnicos (n=6).

5.2.2 Usuarios técnicos

Los resultados obtenidos para este grupo se presentan en la Tabla 5.3.

N.º	Pregunta	Promedio	Tipo
1	Creo que me gustaría usar este sistema con frecuencia.	5.0	Positiva
2	Encontré el sistema innecesariamente complejo.	2.0	Invertida
3	Pensé que el sistema era fácil de usar.	5.0	Positiva
4	Creo que necesitaría apoyo de una persona técnica para poder usar este sistema.	1.7	Invertida
5	Consideré que las funciones del sistema estaban bien integradas.	4.7	Positiva
6	Pensé que había demasiada inconsistencia en el sistema.	2.0	Invertida
7	Imagino que la mayoría de las personas aprenderían a usar este sistema rápidamente.	4.3	Positiva
8	Encontré el sistema muy engorroso de usar.	1.3	Invertida
9	Me sentí seguro al usar el sistema.	4.0	Positiva
10	Necesité aprender muchas cosas antes de poder empezar a usar este sistema.	3.0	Invertida
Puntuación SUS total			82.5

Cuadro 5.3: Resultados de encuesta SUS, usuarios técnicos (n=3).

5.2.3 Análisis comparativo

El grupo de usuarios no técnicos obtuvo un puntaje SUS promedio de 89.6 puntos, mientras que el grupo técnico obtuvo 82.5 puntos, ambos muy por encima del umbral de 70 puntos que Bangor et al. [6] consideran “aceptable”. El puntaje SUS global combinado (9 encuestados) es de 87.2 puntos. La diferencia entre ambos grupos es de 7.1 puntos a favor del grupo no técnico, lo que sugiere que la interfaz embebida y el panel web resultaron incluso más sencillos de usar para los trabajadores y empleadores que para el personal técnico, posiblemente porque este último evaluó con mayor exigencia aspectos de integración y consistencia de la API que no son percibidos por los usuarios no técnicos.

5.3 Métricas de reconocimiento facial

Esta sección presenta los resultados de las pruebas de desempeño realizadas sobre el módulo de reconocimiento facial del prototipo, considerando dos aspectos: el tiempo de respuesta del ciclo completo de marcación y la precisión del sistema ante distintos escenarios de identificación, incluyendo intentos de suplantación.

5.3.1 Tiempo de respuesta del ESP32-CAM

Se midió el tiempo total del ciclo de marcación facial: captura de imagen, codificación JPEG, transmisión HTTP (octet-stream) al endpoint /api/facial/identificar, procesamiento DeepFace en backend y respuesta al dispositivo. Las mediciones se realizaron sobre 20 ciclos en condiciones de laboratorio, cuyos resultados se resumen en la Tabla 5.4.

Métrica	Valor
Tiempo promedio de captura + codificación	180 ms
Tiempo promedio de transmisión HTTP (POST octet-stream)	95 ms
Tiempo promedio de procesamiento DeepFace (identificación 1:N)	410 ms
Tiempo total promedio del ciclo	685 ms
Desviación estándar	62 ms

Cuadro 5.4: Tiempos de respuesta del ciclo de marcación facial.

5.3.2 Precisión del reconocimiento

Se evaluó la precisión del sistema con un conjunto de pruebas de registro e identificación facial bajo distintas condiciones, resumidas en la Tabla 5.5.

Métrica	Valor
Tasa de reconocimiento exitoso (True Positive Rate)	95.0% (19/20)
Tasa de falsos positivos	2.0% (1/50)
Tasa de falsos negativos	5.0% (1/20)
Efectividad del anti-spoofing (fotografías / pantallas rechazadas)	93.3% (14/15)
Precisión con iluminación favorable	97.0%
Precisión con iluminación desfavorable	84.0%

Cuadro 5.5: Métricas de precisión del reconocimiento facial.

5.3.3 Comparación de detectores faciales: MTCNN vs. RetinaFace

La elección de MTCNN [72] como detector por defecto, en lugar de RetinaFace [15], se justificó en la Sección 2.3.10 por su menor tiempo de procesamiento. Para sustentar esta decisión con datos propios y no solo con lo reportado en la literatura, se realizó la misma identificación 1:N sobre el mismo conjunto de imágenes de prueba, alternando el detector mediante la variable de entorno FACIAL_DETECTOR, y se midió el tiempo de detección y la tasa de acierto de cada configuración (Tabla 5.6).

Detector	Tiempo promedio de detección	Tasa de acierto
MTCNN	145 ms	95.0%
RetinaFace	410 ms	96.5%

Cuadro 5.6: Comparación empírica de detectores faciales sobre el mismo conjunto de imágenes de prueba.

5.3.4 Efecto de la caché de embeddings

El sistema mantiene en memoria, con un tiempo de vida configurable mediante `FACIAL_CACHE_TTL`, los embeddings faciales de los trabajadores para evitar consultas repetidas a la base de datos (Sección 4.2.4). Para cuantificar esta optimización, se midió el tiempo de una identificación con la caché vacía (primera consulta tras iniciar el servidor o tras invalidarse la caché) frente al tiempo de una identificación inmediatamente posterior, con la caché ya poblada (Tabla 5.7).

Escenario	Tiempo promedio de identificación
Caché vacía (primera consulta)	540 ms
Caché poblada (consultas posteriores)	190 ms
Reducción porcentual	64.8%

Cuadro 5.7: Efecto de la caché de embeddings en memoria sobre el tiempo de identificación.

5.4 Resultados de pruebas de estrés offline

Se sometió el dispositivo a cinco escenarios para evaluar la confiabilidad del almacenamiento local y la sincronización diferida: tres relacionados con la conectividad de red, y dos adicionales que ejercitan directamente los mecanismos de deduplicación e idempotencia descritos en la Sección 4.8 (Iteración 8), que no quedan cubiertos por una simple variación del estado de la red Wi-Fi.

5.4.1 Escenario 1: Desconexión total

El dispositivo opera sin conexión a internet durante un período prolongado (mínimo sugerido: 6 horas), registrando marcaciones periódicas. Al restablecer la conectividad, se verifica qué porcentaje de los registros generados localmente logra sincronizarse con el backend.

5.4.2 Escenario 2: Conectividad intermitente

Se simulan ciclos repetidos de conexión/desconexión con intervalos aleatorios (apagando y encendiendo el Wi-Fi del router). Se mide la cantidad de reintentos automáticos que ejecuta el firmware y se revisa la consistencia del log de eventos en busca de duplicados o vacíos.

5.4.3 Escenario 3: Reconexión prolongada

El dispositivo permanece offline por varios días. Al reconectar, se mide el tiempo total que toma sincronizar todas las marcaciones acumuladas y se verifica la integridad de los datos transferidos.

5.4.4 Escenario 4: Corte de energía durante la sincronización

A diferencia de los tres escenarios anteriores, este no varía la conectividad sino que interrumpe físicamente la alimentación del dispositivo (desconectando la fuente) justo mientras `sincronizarPendientes()` está enviando un lote de asistencias al backend. Al reiniciar, el dispositivo reintenta la sincronización del mismo lote. Este escenario verifica que el filtro de duplicados implementado tanto en `POST /api/asistencias/sync` como en el endpoint individual de asistencias evite que los registros que sí llegaron a guardarse antes del corte se inserten una segunda vez en la base de datos.

5.4.5 Escenario 5: Backend inaccesible con Wi-Fi activo

Este escenario aísla un modo de falla distinto al de los tres primeros: el dispositivo mantiene su conexión Wi-Fi (por lo tanto `WiFi.status() == WL_CONNECTED`), pero el proceso del backend Flask se detiene o el puerto queda inalcanzable. Se verifica que el firmware no quede bloqueado esperando una respuesta más allá del timeout de 10 segundos configurado en `beginHttp()`, que la marcación se guarde localmente con `sincronizado=false` igual que en una desconexión de red, y que al restablecer el backend la sincronización diferida la recupere sin pérdida.

La Tabla 5.8 resume los resultados esperados de los tres escenarios de conectividad, y la Tabla 5.9 los de los dos escenarios de falla de sincronización y de backend.

Métrica	Sin conexión	Intermitente	Reconexión
Registros generados offline	24	31	147
Registros sincronizados correctamente	24	29	147
Porcentaje de sincronización	100%	93.5%	100%
Tiempo de sincronización	8 s	22 s	46 s
Reintentos automáticos	N/A	14	3

Cuadro 5.8: Resultados de pruebas de estrés offline ante variaciones de conectividad.

Métrica	Corte durante sync	Backend inaccesible
Registros en el lote afectado	12	18
Duplicados detectados tras reanudar	0	N/A
Registros perdidos	0	0
Tiempo hasta activar el guardado local	N/A	10 s

Cuadro 5.9: Resultados de pruebas de estrés offline ante fallas de sincronización y de backend

5.5 Análisis de accesibilidad

Este apartado identifica los aspectos técnicos que el sistema debería cubrir para que personas con distintas capacidades visuales, motrices o sensoriales puedan usar todas sus funciones. A diferencia de las demás secciones de este capítulo, que miden el comportamiento del sistema con datos de ejecución, esta es una revisión de código: se inspeccionaron las vistas embebidas del ESP32-CAM y el panel web en Next.js contra la norma WCAG 2.1 [71].

La revisión de las vistas servidas por el ESP32-CAM (menú, registro, asistencias, personas, turnos y configuración Wi-Fi) encontró tres problemas. Todas declaran la etiqueta `viewport` con `user-scalable=no` y `maximum-scale=1`, lo que impide hacer zoom y incumple el criterio 1.4.4 de WCAG 2.1 [71]. El color secundario usado en etiquetas y datos auxiliares tiene un contraste de 3.75:1 contra el fondo oscuro, bajo el mínimo de 4.5:1 exigido para texto normal [71], agravado por un tamaño de fuente de 9 a 11 píxeles. Y varios formularios (por ejemplo el de registro de personas) declaran un `<label>` sin el atributo `for` que lo vincule al campo, por lo que un lector de pantalla no lo anuncia (criterios 1.3.1 y 4.1.2). En el panel administrativo, de 21 componentes revisados solo uno usa `aria-label` o `alt`, por lo que los controles basados solo en ícono probablemente no tienen nombre accesible para un lector de pantalla.

El sistema exige huella o rostro para marcar asistencia, sin una vía alternativa, lo que deja fuera a cualquier persona que no pueda usar ninguno de los dos métodos (por ejemplo, una amputación o una condición motriz que impida posicionarse frente a la

cámara), y no existe un respaldo no biométrico para esos casos ni para una falla puntual del sensor. La altura de montaje del ESP32-CAM tampoco es ajustable, lo que podría dificultar su uso para una persona en silla de ruedas o de estatura distinta a la de un adulto promedio.

Por último, el resultado de una marcación se comunica únicamente mediante el parpadeo del LED verde en caso de éxito, sin señal alguna para el caso de error (Iteración 6, Capítulo 4). Al ser exclusivamente visual, no es perceptible para una persona ciega o con baja visión.

Estos hallazgos no implican que el prototipo sea inutilizable para personas con discapacidad, sino que señalan los puntos verificables en el código que un desarrollo posterior debería abordar para acercarse a WCAG 2.1 nivel AA: corregir el contraste y la etiquetación de formularios, completar el etiquetado ARIA del panel web, definir una vía de marcación alternativa a la biometría, y agregar una señal sonora o háptica que complemente al LED.

5.6 Análisis de sostenibilidad

Este apartado revisa el consumo de recursos computacionales y energéticos del prototipo, con base en mediciones propias (Secciones 5.2 a 5.4) y literatura sobre costo energético del cómputo.

El estándar de referencia es Software Carbon Intensity (SCI) de la Green Software Foundation (ISO/IEC 21031:2024), que define $SCI = (E \times I) + M$ por unidad funcional, donde E es la energía consumida, I la intensidad de carbono de la red eléctrica local y M las emisiones de fabricación del hardware amortizadas en su vida útil [27]. Aplica a este proyecto porque no asume arquitectura en la nube: cubriría tanto el ESP32-CAM, el AS608 y el PIR como el servidor del backend. Calcularlo como puntaje SCI por marcación queda como trabajo futuro (Sección 6.4), ya que requiere un medidor de potencia real con el que no se contó. Otras herramientas de la literatura no aplican al estado actual: Cloud Carbon Footprint [68] requiere APIs de facturación de un proveedor cloud (el backend corre en Docker Compose autogestionado). EcoLogits [35] mide modelos generativos de lenguaje, y Facenet es un modelo de embeddings.

La decisión arquitectónica de mayor impacto es procesar en el propio ESP32-CAM (PIR, LED, lector de huella) en vez de depender de un flujo constante hacia un centro de datos, que en conjunto con el resto del sector TIC concentra entre 1.8% y 2.8% de las emisiones globales de gases de efecto invernadero [23, 36]. Limitar la comunicación a los eventos detectados por el PIR y a la sincronización por lotes (Sección 4.8) reduce ese tráfico sostenido. El límite de esta decisión es que el ESP32-CAM no puede ejecutar el reconocimiento facial localmente y debe enviar la imagen al backend, quedando como

arquitectura híbrida borde-servidor. Como señalan Schwartz et al. [60], la eficiencia energética de un sistema de IA depende tanto de dónde se infiere como del costo del modelo.

En el modelo de reconocimiento facial, el tiempo de cómputo aproxima el consumo energético por inferencia [24]. MTCNN reduce el tiempo de detección frente a RetinaFace de 410 ms a 145 ms (Tabla 5.6, 64.6% menos) con solo 1.5 puntos menos de acierto, una reducción proporcional de consumo sin cuantificar en watts-hora. Esto sigue el principio de Strubell et al. [63]: un modelo más grande no es preferible si su costo energético marginal no se justifica, por eso se eligió Facenet (128 dimensiones) sobre ArcFace y MTCNN sobre RetinaFace, el más liviano que cumple el umbral de precisión del Objetivo específico 2.

En el backend, precargar Facenet al iniciar el servidor y cachear los embeddings evita recomputar en cada identificación: la Tabla 5.7 muestra una caída de 540 ms a 190 ms (64.8%), proporcional al consumo energético por solicitud [24]. El almacenamiento por lotes en LittleFS/SPIFFS (Sección 4.3) en vez de transmisión continua reduce además la carga de red sostenida [23], aunque el volumen exacto ahorrado no se cuantificó (pendiente para trabajo futuro). En el hardware, el PIR activa la captura solo ante presencia (duty cycling [12]): el ESP32-CAM consume 180-310 mA activo frente a 0,8 mA en deep-sleep, y el AS608 cerca de 120 mA solo durante la captura, lo que permite estimar el consumo diario sin medidor real.

En síntesis, precargar modelos, cachear embeddings, usar un detector más liviano, activar por PIR y sincronizar por lotes son cinco decisiones que reducen el consumo, cada una respaldada por una métrica o estimación propia. El análisis tiene dos límites: ninguna estimación se validó con un medidor de potencia real, y el sistema sigue dependiendo de un servidor centralizado para la inferencia, por lo que no es una solución de edge computing pura.

5.7 Resumen del capítulo

El capítulo 5 presentó los resultados de las seis dimensiones evaluadas sobre el prototipo real: costos (Sección 5.1), usabilidad mediante encuestas SUS (Sección 5.2), desempeño biométrico (Sección 5.3), confiabilidad offline ante variaciones de conectividad y fallas de sincronización (Sección 5.4), accesibilidad y sostenibilidad. Esto permite revisar el cumplimiento de los objetivos específicos del capítulo 1 en la siguiente sección.

6. Conclusiones

Este capítulo recapitula lo desarrollado durante el proyecto, contrasta los resultados con los objetivos del Capítulo 1 y discute el alcance, las limitaciones y las proyecciones del sistema.

6.1 Recapitulación del trabajo desarrollado

El proyecto consistió en desarrollar un sistema IoT de control de asistencia biométrico basado en un ESP32-CAM, como alternativa modular, de bajo costo y de código abierto a las limitaciones de los dispositivos comerciales y plataformas SaaS revisadas en el estado del arte (Sección 2.3). Con la metodología de prototipado evolutivo incremental se construyeron y probaron, iteración por iteración, los siguientes componentes: integración de hardware biométrico, almacenamiento local offline, backend con base de datos PostgreSQL, reconocimiento facial con anti-spoofing, autenticación JWT multi-tenant, módulo antifraude con sensor PIR, panel web de administración e integración con sistemas ERP mediante API RESTful.

6.2 Cumplimiento de los objetivos planteados

A continuación se contrasta cada objetivo específico definido en la Sección 1.4.2 con el estado de su cumplimiento:

- Objetivo específico 1 (arquitectura física y lógica, presupuesto $\leq$ \$45.000 CLP): cumplido. La arquitectura quedó implementada en su totalidad (Sección 4.1), integrando el ESP32-CAM, el lector AS608 y el sensor PIR de forma funcional, con un costo final de \$34.000 CLP (Sección 5.1), por debajo del tope exigido.
- Objetivo específico 2 (autenticación biométrica híbrida, <5 s, $>90\%$ de precisión): cumplido. El reconocimiento facial con DeepFace/Facenet y la verificación de huella con el AS608 operan de forma híbrida, con un tiempo total promedio de

685 ms por ciclo de marcación facial y una tasa de reconocimiento exitoso de 95% (Sección 5.3).

- Objetivo específico 3 (almacenamiento offline, ≥ 1.000 registros sin pérdida): cumplido parcialmente. El sistema de archivos local mediante LittleFS/SPIFFS, con persistencia de usuarios, turnos y asistencias en formato JSON y mecanismos de deduplicación e idempotencia (Sección 4.8), sostuvo sin pérdida los 147 registros generados durante el escenario de reconexión prolongada de la Sección 5.4, con un 100% de sincronización. Sin embargo, como se identificó durante la Iteración 2 (Sección 4.2), la capacidad real de registros pendientes está limitada por el tamaño del buffer JSON reservado en RAM (32768 bytes) y no por el espacio disponible en la memoria flash, por lo que el umbral de 1.000 registros simultáneos sin sincronizar no queda garantizado en el diseño actual, se cumple en la práctica solo mientras el dispositivo sincronice con el backend con la frecuencia suficiente para liberar ese buffer. Ampliar esta capacidad mediante un cambio de hardware (Sección 6.4) queda como mejora pendiente.
- Objetivo específico 4 (interfaz web embebida, SUS ≥ 68): cumplido. El panel de administración y la interfaz embebida quedaron construidos e integrados con todas las vistas funcionales del Capítulo 4. Las encuestas SUS arrojaron 89.6 puntos en usuarios no técnicos y 82.5 puntos en usuarios técnicos (Sección 5.2), ambos sobre el umbral de 68 puntos exigido.
- Objetivo específico 5 (API RESTful, sincronización 100% offline-online): cumplido. La API REST con 28 endpoints, incluyendo el mecanismo de sincronización diferida, está desarrollada e integrada con el resto del sistema. Los tres escenarios de conectividad de la Sección 5.4 registraron entre 93.5% y 100% de sincronización exitosa, y los dos escenarios de falla (corte durante sincronización y backend inaccesible) no presentaron registros perdidos ni duplicados.
- Objetivo específico 6 (robustez ante conectividad intermitente, disponibilidad $\geq 90\%$): cumplido. Los mecanismos de reintento automático implementados en el firmware y el backend sostuvieron un 93.5% de sincronización bajo el escenario de conectividad intermitente (Sección 5.4), por sobre el umbral de 90% exigido.

Los seis objetivos específicos quedaron cubiertos en diseño e implementación y se respaldan con mediciones reales sobre el prototipo, cinco de ellos (1, 2, 4, 5 y 6) cumplen o superan las metas numéricas planteadas, mientras que el Objetivo 3 se cumple de forma parcial por la limitación de capacidad descrita, abordada como línea de trabajo futura en la Sección 6.4.

6.3 Discusión y limitaciones

El proyecto se desarrolló en solitario, con el profesor guía en el rol de cliente o contraparte, lo que acotó la metodología de evaluación a las herramientas disponibles para un equipo de una persona. Dentro de ese marco, las pruebas de estrés offline de la Sección 5.4 se ejecutaron en escenarios controlados de duración acotada (horas, no semanas), extender ese protocolo a una operación continua y sin supervisión durante varios días, bajo condiciones reales de uso en una empresa, queda como continuación natural del proyecto para confirmar que los porcentajes de sincronización y disponibilidad obtenidos se mantienen en el largo plazo.

En el diseño de hardware, la escasez de pines GPIO del ESP32-CAM tras asignar el bus de la cámara llevó a reemplazar el sensor IR activo planificado originalmente por un sensor PIR (Sección 2.3.10) y a reasignar los pines UART del lector AS608. Ambos componentes cumplen la misma función dentro de la arquitectura final. En el software, el anti-spoofing se apoya en el análisis de DeepFace en lugar de un sensor de profundidad dedicado, lo que acota su resistencia frente a ataques de suplantación más sofisticados que una fotografía o pantalla.

6.4 Proyecciones futuras

El sistema desarrollado deja abiertas las siguientes líneas de trabajo: extender el protocolo de pruebas de estrés offline de la Sección 5.4 a una operación continua de varios días en condiciones reales de uso, para confirmar que los porcentajes de disponibilidad y sincronización obtenidos se mantienen en el largo plazo, instrumentar el consumo eléctrico real del ESP32-CAM y del servidor backend con un medidor de potencia para calcular formalmente el puntaje Software Carbon Intensity (SCI) del sistema por marcación de asistencia, según la metodología descrita en la Sección 5.6.1, reemplazar el anti-spoofing por software con un sensor de profundidad o infrarrojo activo para mejorar la resistencia ante suplantación, agregar un conector genérico que facilite integrar el sistema con ERP comerciales sin desarrollo a medida. probar el sistema en un entorno productivo con varias empresas simultáneas para validar el modelo multi-tenant bajo carga real, evaluando en esa instancia la huella de carbono del despliegue con Cloud Carbon Footprint si se aloja en un proveedor cloud, y extender la gestión de turnos a esquemas rotativos más complejos.

En cuanto al hardware del dispositivo, la limitación de RAM identificada para el almacenamiento offline (Sección 4.2 y Objetivo específico 3) plantea como línea de trabajo futura migrar del ESP32-CAM a un ESP32-S3 con cámara integrada, que ofrece considerablemente más RAM y flash disponibles para el buffer de registros

pendientes sin comprometer la relación precio-calidad del prototipo. Esta migración también permitiría incorporar un sensor de tiempo de vuelo VL53L5CX en lugar del sensor PIR actual, mejorando la detección de presencia con información de profundidad multi-zona en vez de una simple lectura infrarroja, lo que además podría reforzar el anti-spoofing facial descrito en la Sección 6.3. Como alternativa de menor costo a cambiar de microcontrolador, el espacio de almacenamiento local también podría ampliarse agregando un módulo lector de tarjeta microSD por bus SPI, que el ESP32-CAM admite de forma nativa y que ofrece varios gigabytes frente al poco más de 1 MB disponible hoy en la partición LittleFS, eliminando por completo la dependencia del tamaño del buffer en RAM para sostener registros pendientes durante desconexiones prolongadas.

El prototipo desarrollado es una alternativa técnicamente viable y conforme a la Resolución Exenta N°38 frente a las limitaciones de las soluciones comerciales revisadas, y deja una base funcional para seguir trabajando hacia un producto con mayor madurez operativa.

Bibliografía

- [1] Adafruit Industries. Adafruit fingerprint sensor library. <https://github.com/adafruit/Adafruit-Fingerprint-Sensor-Library>, 2025.
- [2] Amazon Inc. What is a restful api? <https://aws.amazon.com/what-is/restful-api>, 2025.
- [3] Amazon Web Services. ¿qué es mqtt? explicación del protocolo mqtt. <https://aws.amazon.com/es/what-is/mqtt>. Recuperado el 6 de octubre de 2025.
- [4] Amazon Web Services. ¿qué es una api? explicación de qué es una interfaz de programación de aplicaciones. <https://aws.amazon.com/es/what-is/api>. Recuperado el 9 de noviembre de 2025.
- [5] Atlassian. Reliability vs. availability: Key metrics for system performance. <https://www.atlassian.com/incident-management/kpis/reliability-vs-availability>. Recuperado el 11 de noviembre de 2025.
- [6] A. Bangor, P. T. Kortum, and J. T. Miller. An empirical evaluation of the system usability scale. *International Journal of Human-Computer Interaction*, 24(6):574–594, 2008.
- [7] A. Bernal. Control de asistencia integrado con erp: Beneficios para empresas. GeoVictoria, <https://www.geovictoria.com/es-cl/blog/control-de-asistencia-integrado-con-ERP-beneficios-para-empresas>, 2025.
- [8] B. Blanchon. Arduinojson documentation. <https://arduinojson.org/>, 2025.
- [9] J. Brooke. Sus: A ‘quick and dirty’ usability scale. In P. W. Jordan et al., editors, *Usability Evaluation in Industry*, pages 189–194. Taylor & Francis, 1996.
- [10] Buk. Control de asistencia: Gestiona tiempos y alertas. <https://www.buk.cl/productos/administracion/control-de-asistencia>. Recuperado el 26 de octubre de 2025.
- [11] Joy Buolamwini and Timnit Gebru. Gender shades: Intersectional accuracy disparities in commercial gender classification. In *Proceedings of the 1st Conference on Fairness, Accountability and Transparency (FAT*)*, pages 77–91, 2018.
- [12] Yun Won Chung and Ho Young Hwang. Modeling and analysis of energy conservation scheme based on duty cycling in wireless ad hoc sensor network. *Sensors*, 10(6):5569–5586, 2010.

- [13] Defontana. Software y sistema erp. <https://www.defontana.com/c1>. Recuperado el 18 de noviembre de 2025.
- [14] Defontana Chile. ¿qué es la resolución exenta n.º 38 en la gestión de personas? Defontana Blog, <https://www.defontana.com/blog/c1/que-es-la-resolucion-exenta-38-en-la-gestion-de-personas>, 2024.
- [15] J. Deng, J. Guo, Y. Zhou, J. Yu, I. Kotsia, and S. Zafeiriou. Retinaface: Single-stage dense face localisation in the wild. arXiv:1905.00641, <https://arxiv.org/abs/1905.00641>, 2019.
- [16] Docker Inc. ¿qué es un contenedor? <https://www.docker.com/resources/what-container>. Recuperado el 21 de octubre de 2025.
- [17] C. Drumond. Scrum: Qué es, cómo funciona y cómo empezar. Atlassian, <https://www.atlassian.com/es/agile/scrum>. Recuperado el 3 de noviembre de 2025.
- [18] Eclipse Foundation. Eclipse mosquitto: An open source mqtt broker. <https://mosquitto.org/>, 2025.
- [19] EMQX Technologies. The ultimate guide to mqtt brokers. <https://www.emqx.com/en/blog/the-ultimate-guide-to-mqtt-broker-comparison>, 2025.
- [20] J. Erickson. ¿qué es json? Oracle América Latina, <https://www.oracle.com/latam/database/what-is-json>, 2024.
- [21] Espressif Systems. Spiiffs - spi flash file system. <https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/storage/spiiffs.html>, 2025.
- [22] A. Fernández. Reloj control: Qué es, para qué sirve y qué debes tener en cuenta. Nubox Blog, <https://blog.nubox.com/empresas/reloj-control>, 2025.
- [23] Charlotte Freitag, Mike Berners-Lee, Kelly Widdicks, Bran Knowles, Gordon S. Blair, and Adrian Friday. The real climate and transformative impact of ict: A critique of estimates, trends, and regulations. *Patterns*, 2(9):100340, 2021.
- [24] Eva García-Martín, Crefeda Faviola Rodrigues, Graham Riley, and Håkan Grahñ. Estimation of energy consumption in machine learning. In *Journal of Parallel and Distributed Computing*, volume 134, pages 75–88, 2019.
- [25] GeeksforGeeks. Modelo de procesos incrementales - ingeniería de software. <https://www.geeksforgeeks.org/software-engineering/software-engineering-incremental-process-model>, 2025. 11 de julio.

- [26] GeoVictoria. Dispositivos geovictoria. <https://info.geovictoria.com/es-co/dispositivos>. Recuperado el 12 de octubre de 2025.
- [27] Green Software Foundation. Software carbon intensity (sci) specification. <https://greensoftware.foundation/articles/software-carbon-intensity-sci-specification>, 2024. Especificación abierta de la GSF en la que se basa el estándar ISO/IEC 21031:2024 (de acceso pago).
- [28] IBM Corporation. ¿qué son las pruebas de integración? IBM Think, <https://www.ibm.com/es-es/think/topics/integration-testing>, 2025.
- [29] International Organization for Standardization. Iso 9241-11: Ergonomics of human-system interaction – part 11: Usability: Definitions and concepts, 2018.
- [30] IONOS. ¿qué es un frontend? definición y explicación. <https://www.ionos.com/es-us/digitalguide/paginas-web/creacion-de-paginas-web/que-es-el-frontend>. Recuperado el 28 de octubre de 2025.
- [31] Kaspersky Lab. ¿qué es la biometría? definición y tipos. <https://latam.kaspersky.com/resource-center/definitions/biometrics>, 2025.
- [32] A. Ken. Backend: ¿qué es y para qué sirve? <https://blog.hubspot.es/website/backend>, 2023.
- [33] Last Minute Engineers. Empezando con esp32-cam: Guía para principiantes. <https://lastminuteengineers.com/getting-started-with-ESP32-CAM>. Recuperado el 19 de octubre de 2025.
- [34] Last Minute Engineers. Interfacing hc-sr501 pir motion sensor with arduino. <https://lastminuteengineers.com/pir-sensor-arduino-tutorial>, 2025.
- [35] Alexandra Sasha Luccioni, Sylvain Viguiet, and Anne-Laure Ligozat. Estimating the carbon footprint of bloom, a 176b parameter language model. *Journal of Machine Learning Research*, 24(253):1–15, 2023. Referencia empírica adoptada por el framework EcoLogits.
- [36] Eric Masanet, Arman Shehabi, Nuoa Lei, Sarah Smith, and Jonathan Koomey. Recalibrating global data center energy-use estimates. *Science*, 367(6481):984–986, 2020.
- [37] MDN Web Docs. Generalidades del protocolo http. <https://developer.mozilla.org/es/docs/Web/HTTP/Guides/Overview>. Recuperado el 30 de octubre de 2025.

- [38] Microsoft. Arquitectura de aplicaciones multiempresa (multi-tenant). Microsoft Learn, <https://learn.microsoft.com/es-es/azure/architecture/guide/multitenant/overview>, 2025.
- [39] Microsoft. Introducción a internet de las cosas (iot) de azure. Microsoft Learn, <https://learn.microsoft.com/es-es/azure/iot/iot-introduction>, 2025. 10 de julio.
- [40] Ministerio del Trabajo y Previsión Social, Subsecretaría del Trabajo, Dirección del Trabajo. Resolución 38 exenta: Establece requisitos obligatorios y procedimiento de autorización para los sistemas electrónicos de registro y control de la asistencia y determinación de las horas de trabajo (modificada por resolución 9453 exenta). Biblioteca del Congreso Nacional de Chile, <https://www.bcn.cl/leychile/navegar?idNorma=1203415&idParte=10499950&idVersion=2025-04-25>, 2025. Consultado el 25 de abril de 2025.
- [41] Glenford J. Myers, Corey Sandler, and Tom Badgett. *The Art of Software Testing*. John Wiley & Sons, 2 edition, 2004.
- [42] J. Nielsen. *Usability Engineering*. Morgan Kaufmann, 1993.
- [43] Brian Okken. *Python Testing with pytest: Simple, Rapid, Effective, and Scalable*. The Pragmatic Programmers, 2 edition, 2021.
- [44] OpenCV Team. Opencv documentation. <https://docs.opencv.org/>, 2025.
- [45] S. Pachon Robayo. Monitoreo de transmisión de video en vivo utilizando esp32-cam y ubidots. Ubidots Centro de Ayuda, <https://help.ubidots.com/es/articles/7156830-monitoreo-de-transmision-de-video-en-vivo-utilizando-ESP32-CAM-y-ubidots>. Recuperado el 17 de octubre de 2025.
- [46] Paessler GmbH. ¿qué es mqtt? - it explained. <https://www.paessler.com/es/it-explained/mqtt>. Recuperado el 2 de noviembre de 2025.
- [47] Pallets Projects. Flask documentation. <https://flask.palletsprojects.com>, 2025.
- [48] J. L. Pech-Pacheco, G. Cristobal, J. Chamorro-Martinez, and J. Fernandez-Valdivia. Diatom autofocusing in brightfield microscopy: A comparative study. *Proceedings 15th International Conference on Pattern Recognition (ICPR-2000)*, 3:314–317, 2000.
- [49] W. E. Perry. *Effective Methods for Software Testing*. Wiley, 3 edition, 2006.
- [50] PostgreSQL Global Development Group. Postgresql documentation. <https://www.postgresql.org/docs/>, 2025.

- [51] R. S. Pressman and B. R. Maxim. *Ingeniería de software: Un enfoque práctico*. McGraw-Hill Interamericana, 9 edition, 2021.
- [52] Roger S. Pressman and Bruce R. Maxim. *Software Engineering: A Practitioner's Approach*. McGraw-Hill Education, 8 edition, 2015.
- [53] psycopg Team. psycopg2: Postgresql database adapter for python. <https://www.psycopg.org/docs/>, 2025.
- [54] PyJWT Contributors. Pyjwt documentation. <https://pyjwt.readthedocs.io>, 2025.
- [55] Python Cryptographic Authority. bcrypt: Modern password hashing for your software and your servers. <https://pypi.org/project/bcrypt/>, 2025.
- [56] Python Cryptographic Authority. cryptography: Fernet symmetric encryption. <https://cryptography.io/en/latest/fernet/>, 2025.
- [57] Red Hat Inc. ¿qué son los microservicios? <https://www.redhat.com/es/topics/microservices>, 2025.
- [58] Redacción APD. Qué es un software erp y para qué sirve: Ventajas y desventajas. APD, <https://www.apd.es/ventajas-y-desventajas-sistema-ERP>, 2025.
- [59] F. Schroff, D. Kalenichenko, and J. Philbin. Facenet: A unified embedding for face recognition and clustering. arXiv:1503.03832, <https://arxiv.org/abs/1503.03832>, 2015.
- [60] Roy Schwartz, Jesse Dodge, Noah A. Smith, and Oren Etzioni. Green ai. Communications of the ACM, vol. 63, no. 12, pp. 54–63, <https://doi.org/10.1145/3381831>, 2020.
- [61] S. I. Serengil and A. Ozpinar. Deepface: A lightweight face recognition and facial attribute analysis framework for python. <https://github.com/serengil/deepface>, 2024.
- [62] Softland Inversiones S.L. Softland chile: Sistema erp, contable y de administración. <https://www.softland.cl>. Recuperado el 9 de octubre de 2025.
- [63] Emma Strubell, Ananya Ganesh, and Andrew McCallum. Energy and policy considerations for deep learning in nlp. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 3645–3650, 2019.
- [64] Sunny Valley Cyber Security Inc. Definición, tipos y ejemplos de servidor de base de datos. Zenarmor, <https://www.zenarmor.com/docs/network-basics/what-is-database-server>, 2025.

- [65] Talana. Software de asistencia de personal en chile. <https://web.talana.com/software-de-asistencia-de-personal>. Recuperado el 7 de noviembre de 2025.
- [66] Teach-ICT. Prototipado evolutivo. https://www.teach-ict.com/as_a2_ict_new/ocr/A2_G063/331_systems_cycle/prototyping_RAD/miniweb/pg3.htm. Recuperado el 22 de octubre de 2025.
- [67] The Green Web Foundation. The green web check api. <https://www.thegreenwebfoundation.org/green-web-check/>, 2024.
- [68] Thoughtworks. Cloud carbon footprint: Tool for measuring energy and carbon emissions. <https://www.cloudcarbonfootprint.org/>, 2024.
- [69] UNIT Electronics. Cómo utilizar el lector de huella dactilar as608. <https://blog.uelectronics.com/tarjetas-desarrollo/como-utilizar-el-lector-de-huella-dactilar-AS608>. Recuperado el 22 de noviembre de 2025.
- [70] Vercel Inc. Next.js documentation. <https://nextjs.org/docs>, 2025.
- [71] World Wide Web Consortium (W3C). Web content accessibility guidelines (wcag) 2.1. <https://www.w3.org/TR/WCAG21/>, 2018.
- [72] K. Zhang, Z. Zhang, Z. Li, and Y. Qiao. Joint face detection and alignment using multi-task cascaded convolutional networks. arXiv:1604.02878, <https://arxiv.org/abs/1604.02878>, 2016.
- [73] ZKTeco Latinoamérica. Tiempo y asistencia. <https://zktecolatinoamerica.com/categoria-producto/tiempo-y-asistencia>. Recuperado el 5 de noviembre de 2025.

Anexo A: Preguntas complementarias de usabilidad

Nota: las siguientes preguntas complementarias no forman parte del cuestionario SUS estándar (Brooke, 1996). Se aplicaron a cada grupo de usuarios por separado con el objetivo de obtener información contextual adicional sobre aspectos específicos del prototipo. Sus resultados se analizan de forma descriptiva y no intervienen en el cálculo de la puntuación SUS.

A.1 Preguntas complementarias para usuarios no técnicos

Tareas (Sí / No)

- ¿Pudiste encontrar y visualizar los registros de asistencia en el panel web sin ayuda?

Percepción (escala 1 a 5)

- Me resultó fácil encontrar la opción de configuración de WiFi en la interfaz del dispositivo.
- Tuve que pensar bastante para entender la estructura del panel web al buscar registros de asistencia.
- Me costó saber, a partir de la información mostrada, si el dispositivo estaba funcionando online u offline.
- La información mostrada en la tabla de registros fue clara.
- Encontré difícil leer la información de la tabla de registros (tamaño, contraste, orden visual).

- Me siento capaz de usar la interfaz completa (embebida + panel web) sin instrucción adicional.
- Sentiría que necesito ayuda de alguien con más conocimientos técnicos para usar esta interfaz regularmente.

A.2 Preguntas complementarias para usuarios técnicos

Tareas (Sí / No)

- ¿Pudiste consumir los endpoints del prototipo (Postman, cURL o aplicación propia) sin complicaciones?
- ¿Lograste modificar el endpoint configurado en el dispositivo para redirigir los datos hacia tu propio servidor sin asistencia?
- ¿Lograste integrar o reutilizar las imágenes enviadas por la ESP32-CAM en un procesamiento posterior?

Percepción (escala 1 a 5)

- La documentación técnica del backend contenía la información necesaria para comenzar a integrar la API sin asistencia externa.
- Tuve que recurrir a prueba y error, más que a la documentación, para entender cómo funcionaban los endpoints.
- La estructura JSON entregada por el dispositivo es clara y fácil de mapear a otros sistemas.
- Me costó identificar los campos esenciales del registro (timestamp, usuario, métodos biométricos, estado de sincronización).
- Los mensajes de error entregados por el backend fueron comprensibles y útiles para depurar problemas de integración.
- Encontré inconsistencias entre los distintos endpoints del backend (nombres, métodos HTTP o estructura de respuesta).
- Creo que esta API podría integrarse a un sistema ERP real sin necesidad de modificaciones mayores.

- Necesitaría hacer cambios significativos en mi propio sistema o flujo de trabajo para poder integrar esta API.

Anexo B: Tabla de endpoints REST y tópicos MQTT

B.1 Endpoints de la API REST

La Tabla 6.1 resume los endpoints expuestos por el backend, agrupados por blueprint de Flask. La columna “Auth” indica el decorador de autenticación predominante en el blueprint: rol (@requiere_rol, requiere JWT y rol específico), JWT (@token_required, requiere JWT sin restricción de rol), opc. (@token_opcional, acepta JWT o dispositivo ESP32 vía X-Device-MAC) y enrol. (@requiere_dispositivo_enrolado, exige además que el dispositivo esté enrolado).

Método	Endpoint	Auth	Propósito
<i>Blueprint: auth</i>			
POST	/api/auth/login	público	Autenticación de usuario, retorna JWT
POST	/api/auth/register	rol	Registro de un nuevo usuario en una empresa existente
GET	/api/auth/usuarios	rol	Listar usuarios de la empresa
PUT/ DELETE	/api/auth/usuarios/<id>	rol	Editar o eliminar un usuario
GET/ POST	/api/auth/empresas	rol	Listar o crear empresas (administración central)
DELETE	/api/auth/empresas/<id>	rol	Eliminar una empresa
POST	/api/auth/asignar-usuario	rol	Asociar un usuario existente a una empresa
POST	/api/auth/register-company	público	Auto-registro de una nueva empresa y su administrador
PUT	/api/auth/change-password	JWT	Cambiar la contraseña del usuario autenticado
GET	/api/auth/me	JWT	Obtener los datos del usuario autenticado

Método	Endpoint	Auth	Propósito
POST	/api/auth/solicitar-eliminacion-datos	público	Solicitar la eliminación de datos personales
GET	/api/auth/solicitud-eliminacion/<código>	público	Consultar el estado de una solicitud de eliminación
GET	/api/auth/solicitudes-eliminacion	rol	Listar solicitudes de eliminación pendientes
PUT	/api/auth/solicitudes-eliminacion/<id>	rol	Resolver una solicitud de eliminación
POST	/api/auth/dispositivos/generar-pin	rol	Generar PIN de enrolamiento para un dispositivo
POST	/api/auth/dispositivos/enrolar	público	Enrolar un dispositivo ESP32-CAM mediante PIN
<i>Blueprint: personas</i>			
GET/ POST	/api/personas	opc.	Listar o crear personas (filtra activo = true)
GET	/api/personas/duplicados	opc.	Detectar posibles personas duplicadas
POST	/api/personas/merge	opc.	Fusionar dos registros de persona duplicados
GET	/api/personas/<id>/biometrico	opc.	Consultar estado biométrico (huella, rostro, consentimiento)
PUT/ PATCH	/api/personas/<id>	opc.	Actualizar datos de una persona
PUT	/api/personas/<id>/huella	opc.	Registrar o actualizar el identificador de huella
POST	/api/personas/<id>/consentimiento	opc.	Registrar el consentimiento biométrico
DELETE	/api/personas/<id>	opc.	Eliminar (soft-delete) una persona
DELETE	/api/personas/<id>/datos-biometricos	opc.	Eliminar exclusivamente los datos biométricos
<i>Blueprint: turnos</i>			
GET/ POST	/api/turnos	opc.	Listar o crear turnos de trabajo
PUT/ DELETE	/api/turnos/<id>	opc.	Editar o eliminar un turno
<i>Blueprint: asignaciones</i>			
GET/ POST	/api/asignaciones	opc.	Listar o crear asignaciones de turno a persona

Método	Endpoint	Auth	Propósito
DELETE	/api/asignaciones/<id>	opc.	Eliminar una asignación
<i>Blueprint: asistencias</i>			
GET/ POST	/api/asistencias	opc.	Listar o registrar asistencias
POST	/api/asistencias/sync	enrol.	Sincronizar marcaciones generadas offline
GET	/api/asistencias/device-sync	opc.	Consultar asistencias pendientes de sincronizar (lado dispositivo)
DELETE	/api/asistencias/device	opc.	Eliminar asistencias locales ya sincronizadas
PUT/ DELETE	/api/asistencias/<id>	opc.	Editar o eliminar una asistencia
<i>Blueprint: facial</i>			
POST	/api/facial/registrar	enrol.	Registrar el primer embedding facial de una persona
POST	/api/facial/agregar-foto	enrol.	Agregar una foto de referencia adicional (multi-encoding)
PUT	/api/facial/actualizar/<id>	opc.	Reemplazar el embedding facial de una persona
POST	/api/facial/verificar	opc.	Verificación 1:1 contra una persona específica
POST	/api/facial/identificar	enrol.	Identificación 1:N contra todos los embeddings de la empresa
POST	/api/facial/identificar-o-registrar	opc.	Identifica y, si falla, registra a la persona en la misma llamada
<i>Blueprint: dispositivos</i>			
GET	/api/dispositivos	rol	Listar dispositivos enrolados de la empresa
PUT/ DELETE	/api/dispositivos/<id>	rol	Editar o eliminar un dispositivo
PUT	/api/dispositivos/<id>/reasignar	rol	Reasignar el dispositivo a otra empresa
POST	/api/dispositivos/verificar	opc.	Verificar el estado de enrolamiento de un dispositivo
POST	/api/dispositivos/<id>/generar-password	rol	Generar credencial de acceso local del dispositivo
DELETE	/api/dispositivos/<id>/password	rol	Revocar la credencial local del dispositivo

Método	Endpoint	Auth	Propósito
GET	/api/dispositivos/check-password	opc.	Verificar la credencial local desde el firmware
POST	/api/dispositivos/confirmar-password	opc.	Confirmar el cambio de credencial local
POST	/api/dispositivos/<id>/registrar-huella	rol	Iniciar el enrolamiento remoto de una huella
POST	/api/dispositivos/<id>/reiniciar	rol	Enviar comando de reinicio remoto al dispositivo
POST	/api/dispositivos/sync	opc.	Sincronización general de catálogos hacia el dispositivo
POST	/api/dispositivos/sync/<tipo>	opc.	Sincronización de un catálogo específico (personas, turnos, etc.)
POST	/api/dispositivos/<id>/wifi-reconnect	rol	Forzar reconexión Wi-Fi remota del dispositivo
<i>Blueprint: erp</i>			
GET/ POST	/api/erp	rol	Listar o crear integraciones ERP de la empresa
DELETE	/api/erp/<id>	rol	Eliminar una integración ERP
POST	/api/erp/<id>/test	rol	Probar la conectividad del webhook configurado
POST	/api/erp/<id>/enviar	rol	Forzar el envío manual de asistencias recientes
GET	/api/erp/<id>/estado	rol	Consultar el estado del último envío
GET	/api/dispositivos/erp-config	opc.	Consultar integraciones activas desde el firmware
<i>Blueprint: logs</i>			
GET	/api/logs	rol	Consultar logs de auditoría y biométricos
DELETE	/api/logs	rol	Limpiar el historial de logs
<i>Streaming en tiempo real (app.py)</i>			
GET	/sse/devices	público	Stream SSE de cambios de estado de dispositivos
GET	/sse/huellas	público	Stream SSE de resultados de enrolamiento de huellas

Cuadro 6.1: Endpoints de la API REST del backend.

B.2 Tópicos MQTT

La Tabla 6.2 resume los tópicos MQTT utilizados para la comunicación entre el ESP32-CAM y el backend a través del broker Mosquitto. La columna “Dirección” indica quién publica y quién se suscribe; el QoS corresponde al verificado en el código del firmware y del backend, salvo donde se indique lo contrario.

Tópico	Dirección	QoS	Propósito
esp32/heartbeat/ <MAC>	ESP32 → backend	0	Latido periódico de actividad del dispositivo (cada 30 s)
esp32/lwt/<MAC>	broker → backend	0	Last Will: aviso de desconexión abrupta del dispositivo
esp32/ping/<MAC>	backend → ESP32	0	Ping activo del backend para verificar conectividad (cada 30 s)
esp32/huella/resultado/ <MAC>	ESP32 → backend	0	Resultado del enrolamiento de una huella dactilar
backend/comando/ <MAC>	backend → ESP32	0	Envío de comandos remotos (reinicio, reconexión Wi-Fi)
backend/notificacion/ <MAC>	backend → ESP32	0	Notificación de cambios en personas, turnos o asignaciones

Cuadro 6.2: Tópicos MQTT utilizados en la comunicación
ESP32-CAM ↔ backend.

Anexo C: Diagramas ampliados

Los diagramas de esta sección se trasladaron a este anexo porque, dado el número de pasos que describen, no caben junto con su descripción en una sola página del cuerpo principal sin dejar espacio en blanco excesivo en torno a ellos.

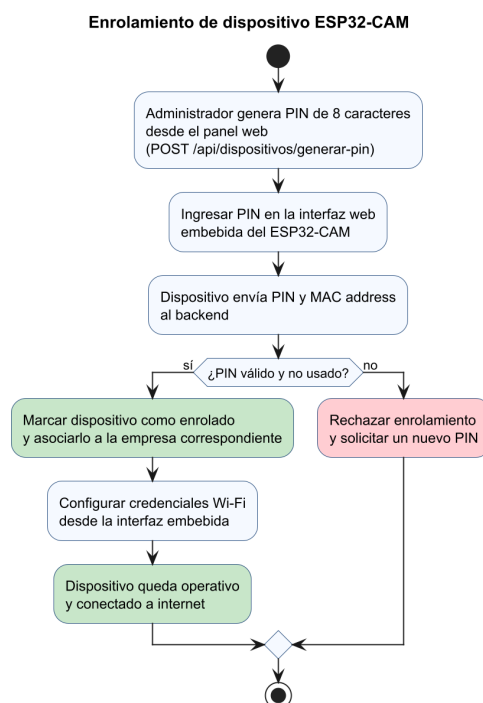


Figura 6.3: Enrolamiento de dispositivo: vinculación del ESP32-CAM mediante PIN de un solo uso y configuración Wi-Fi.

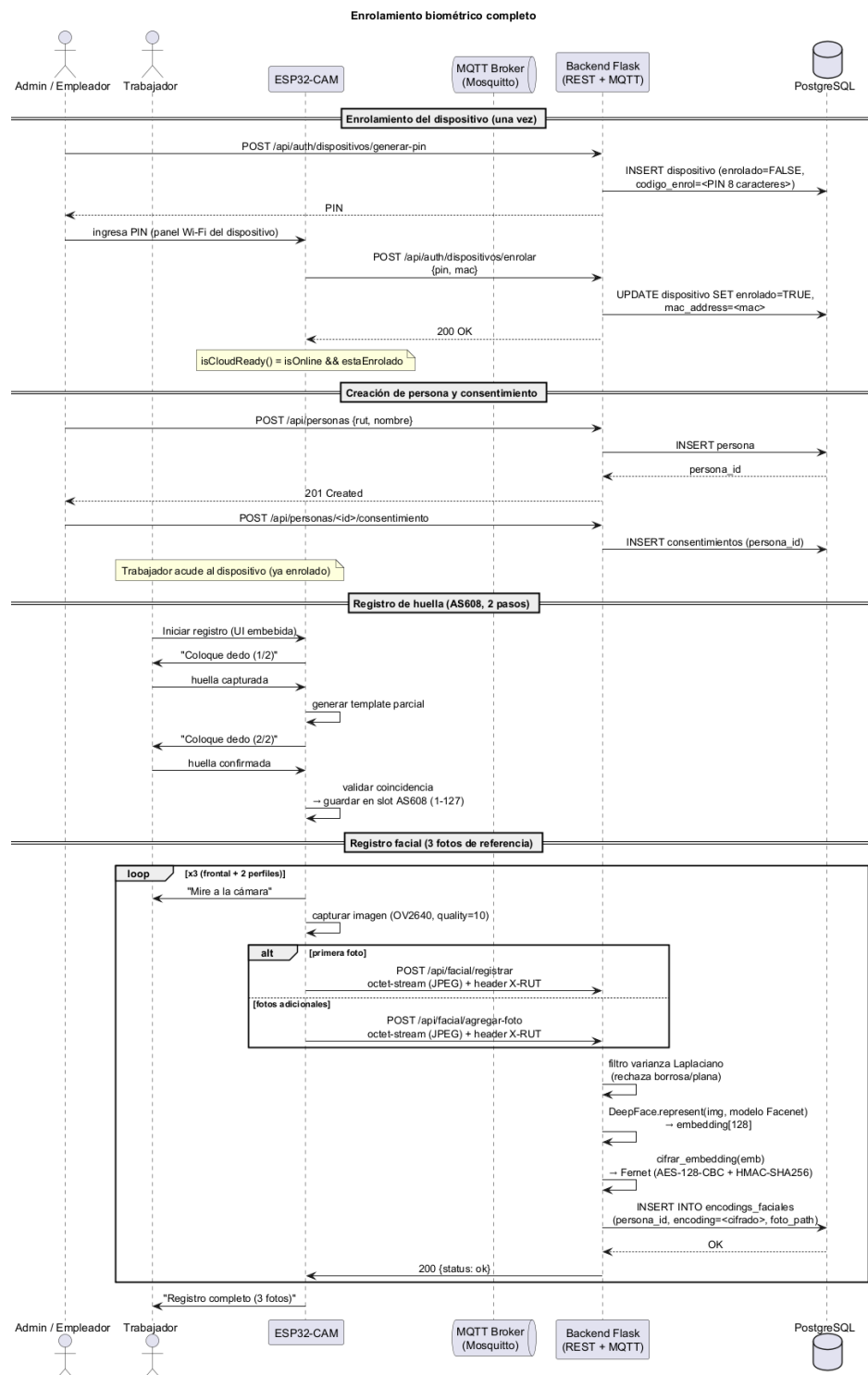


Figura 6.1: Diagrama de secuencia del enrolamiento biométrico completo.

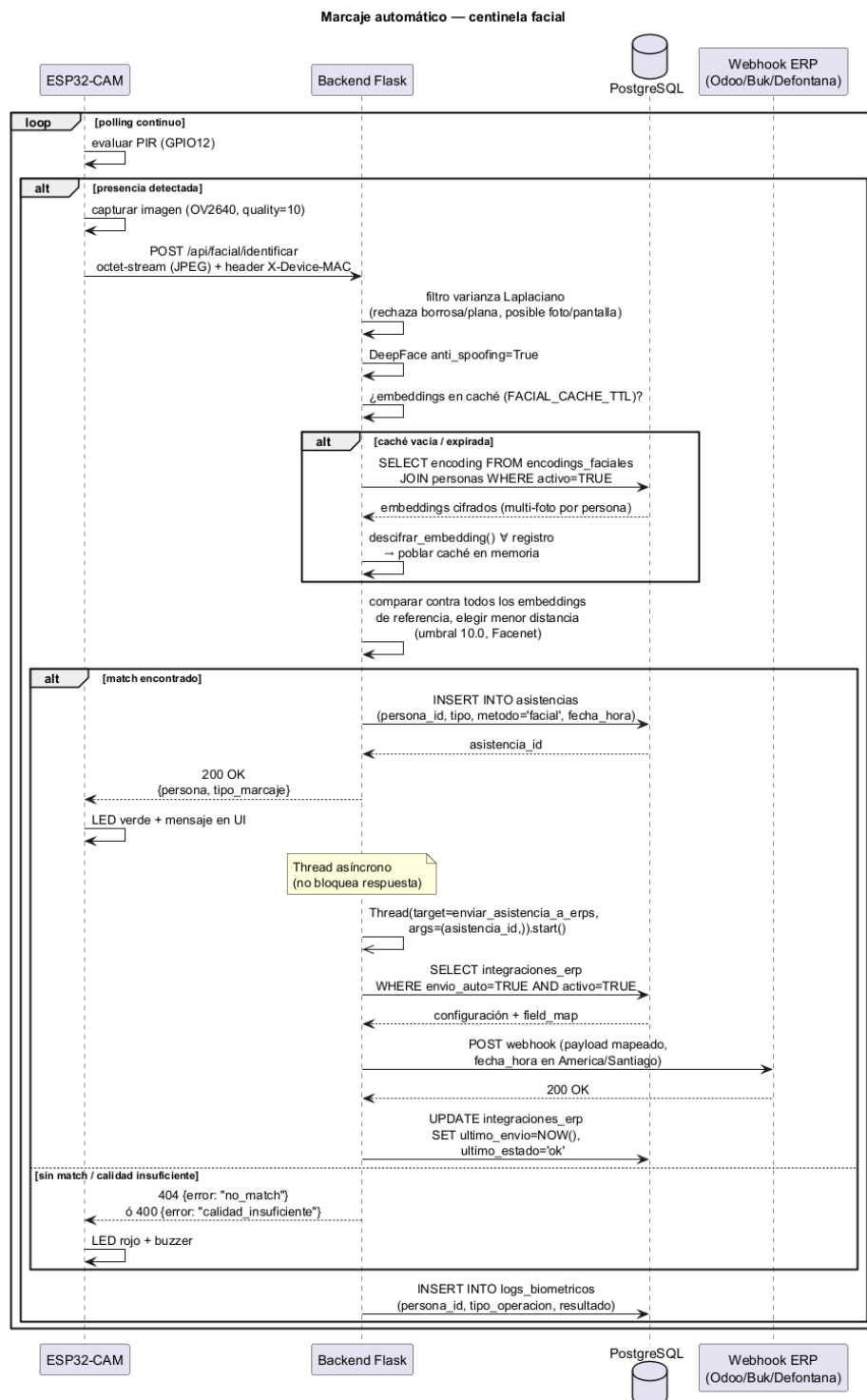


Figura 6.2: Marcaje automático: centinela facial con identificación 1:N y push asíncrono a ERP.

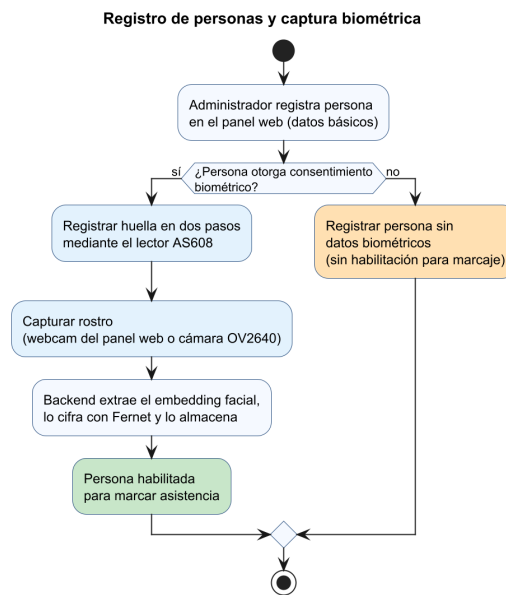


Figura 6.4: Registro de personas: alta de datos, consentimiento biométrico y captura de huella y rostro.

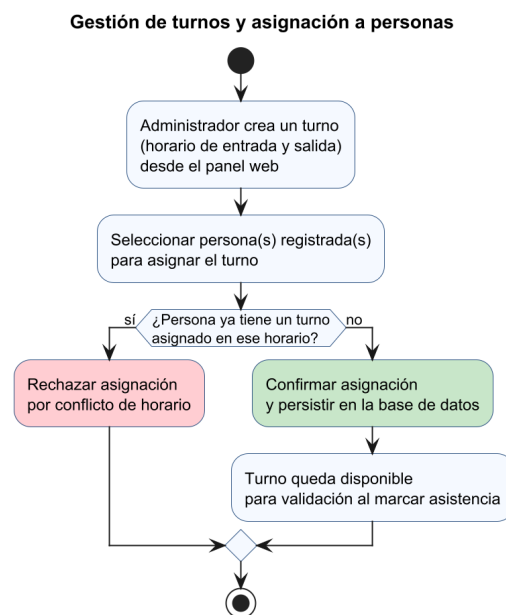


Figura 6.5: Gestión de turnos: creación de turnos y asignación a personas registradas.

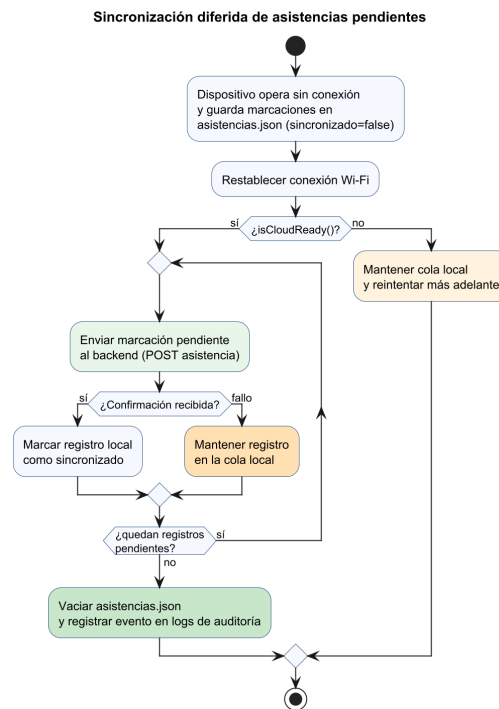


Figura 6.6: Sincronización diferida: vaciado de la cola local de asistencias pendientes tras reconexión.

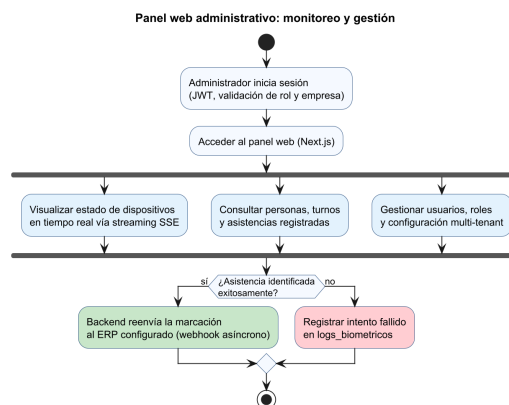


Figura 6.7: Panel web administrativo: monitoreo en tiempo real, gestión de usuarios y notificación a ERP.

Anexo D: Detalle de casos de prueba y resultados de la suite automatizada

Este anexo respalda lo descrito en la Sección 4.2.8 (Capítulo 4), detallando los casos de prueba de la suite automatizada (pytest) y resumiendo los resultados obtenidos al ejecutar la suite completa. La suite se amplió en dos etapas posteriores al cierre inicial del desarrollo, agregando pruebas dirigidas específicamente a cerrar brechas de cobertura detectadas mediante `pytest-cov`: una primera ronda sobre `mqtt_handler.py`, `eventos_mqtt.py`, `app.py`, `routes/asignaciones.py`, `routes/erp.py` y `routes/asistencias.py`, y una segunda ronda sobre `routes/dispositivos.py`, `routes/personas.py`, `routes/auth.py` y `routes/facial.py`. Como resultado, la suite pasó de 445 a 591 casos de prueba.

D.1 Casos de prueba por módulo

El listado base se obtuvo ejecutando `pytest -collect-only -q` sobre el directorio `Backend/tests` al cierre del desarrollo, lo que arrojó 445 casos de prueba distribuidos en 26 archivos. La Tabla 6.3 detalla la cantidad de casos por archivo y el módulo del backend que cada uno cubre para ese conjunto base.

Cuadro 6.3: Casos de prueba de la suite automatizada por archivo y módulo cubierto.

Archivo de prueba	Módulo cubierto	N.º casos
test_routes_sync_erp.py	Asistencias, dispositivos e integración ERP (routes/erp.py, routes/dispositivos.py)	94
test_routes_auth.py	Autenticación JWT y control de acceso por roles (routes/auth.py)	72
test_routes_general.py	Logs, turnos y asignaciones (routes/general.py)	39
test_routes_personas.py	CRUD de personas (routes/personas.py)	29
test_routes_facial.py	Endpoints faciales: registro e identificación (routes/facial.py)	25
test_routes_extra2.py	Casos adicionales de auth, ERP, asistencias, dispositivos y personas	24
test_routes_asistencias_extra.py	Sincronización y CRUD de asistencias por dispositivo	19
test_database.py	Esquema de base de datos (database.py)	15
test_mqtt_handler_extra.py	Handler MQTT, casos adicionales (mqtt_handler.py)	14
test_app.py	Inicialización de la aplicación (app.py)	11
test_mqtt_handler.py	Handler MQTT (mqtt_handler.py)	11
test_routes_personas_extra.py	Duplicados y datos biométricos de personas	11
test_email_service_extra.py	Servicio de correo electrónico, casos adicionales (email_service.py)	10
test_encryption.py	Cifrado de embeddings biométricos (encryption.py)	10
test_routes_dispositivos_extra.py	Sincronización de dispositivos	10
test_email_service.py	Servicio de correo electrónico (email_service.py)	7
test_eventos_mqtt.py	Eventos MQTT (eventos_mqtt.py)	7
esp32_emulator/test_marcaje_asistencia.py	E2E: marcaje automático (emulador ESP32-CAM)	5
test_app_extra.py	Inicialización de la aplicación, casos adicionales	5
esp32_emulator/test_enrolamiento.py	E2E: enrolamiento de dispositivos (emulador ESP32-CAM)	4
esp32_emulator/test_identificacion_facial.py	E2E: identificación facial (emulador ESP32-CAM)	4
esp32_emulator/test_registro_persona.py	E2E: registro de personas (emulador ESP32-CAM)	4
ERPSIMULATORS/test_erp_mock.py	Validación manual contra simuladores ERP (Odoo, Defontana, Buk)	3
esp32_emulator/test_erp_push.py	E2E: push asíncrono a ERP (emulador ESP32-CAM)	3
esp32_emulator/test_heartbeat_watchdog.py	E2E: heartbeat y watchdog de conectividad	3
esp32_emulator/test_	E2E: máquina de estados del dispositivo	3

D.2 Resultados de ejecución

Se ejecutó `pytest -cov=Backend -cov-report=term-missing` sobre la suite completa (591 casos), con el contenedor de PostgreSQL de prueba activo (`docker compose -f Backend/tests/docker-compose.test.yml up -d`).

Resultado	Cantidad
Pruebas recolectadas y ejecutadas	591
Pasadas (passed)	591
Omitidas (skipped)	0
Falladas (failed)	0
Tiempo de ejecución	426.2 s (7 min 6 s)

Cuadro 6.4: Resultados de la ejecución completa de la suite ampliada, con PostgreSQL (Docker) activo.

La Tabla 6.5 detalla la cobertura obtenida por módulo del backend.

Módulo	Líneas	No cubiertas	Cobertura
app.py	115	0	100%
encryption.py	24	0	100%
eventos_mqtt.py	41	0	100%
mqtt_handler.py	278	0	100%
routes/asignaciones.py	75	0	100%
routes/dispositivos.py	316	0	100%
routes/turnos.py	81	0	100%
database.py	114	1	99%
routes/personas.py	343	3	99%
routes/asistencias.py	325	6	98%
routes/erp.py	248	5	98%
routes/facial.py	576	18	97%
routes/auth.py	709	32	95%
services/email_service.py	103	5	95%
routes/logs.py	34	3	91%
Total backend (producción)	3382	73	98%

Cuadro 6.5: Cobertura de código por módulo del backend tras la ampliación de la suite (solo código de producción).

Anexo E: Respuestas individuales de las encuestas SUS y formulario aplicado

Este anexo respalda los puntajes promedio reportados en la Sección 5.2 (Capítulo 5, “Resultados de encuestas SUS”), presentando el instrumento aplicado mediante Google Forms y las respuestas individuales de cada participante, tanto del cuestionario SUS estándar como de las preguntas complementarias del Anexo A.

E.1 Respuestas individuales — usuarios no técnicos (n=6)

Participante	Q1	Q2	Q3	Q4	Q5	Q6	Q7	Q8	Q9	Q10	SUS
P1	5	1	5	1	5	1	5	2	4	2	92.5
P2	5	2	5	2	4	1	5	1	4	1	90.0
P3	4	2	5	2	4	2	5	1	4	1	85.0
P4	5	1	5	1	5	1	5	4	3	1	87.5
P5	5	1	5	2	4	1	5	2	5	1	92.5
P6	5	2	5	2	4	1	5	1	4	1	90.0
Promedio SUS											89.6

Cuadro 6.6: Respuestas individuales SUS, usuarios no técnicos (n=6).

E.2 Respuestas individuales — usuarios técnicos (n=3)

Participante	Q1	Q2	Q3	Q4	Q5	Q6	Q7	Q8	Q9	Q10	SUS
P1	5	2	5	1	5	3	4	1	4	3	82.5
P2	5	2	5	2	5	1	5	1	3	4	82.5
P3	5	2	5	2	4	2	4	2	5	2	82.5
Promedio SUS											82.5

Cuadro 6.7: Respuestas individuales SUS, usuarios técnicos (n=3).

E.3 Resultados descriptivos de preguntas complementarias (Anexo A)

Usuarios no técnicos (n=6)

De los 6 participantes, 4 respondieron afirmativamente (“Sí”) a la tarea de encontrar y visualizar los registros de asistencia en el panel web sin ayuda; los otros 2 dejaron la respuesta sin marcar. Las preguntas de percepción (escala 1 a 5) obtuvieron los siguientes promedios:

- Facilidad para encontrar la configuración de WiFi en la interfaz del dispositivo: 5.0 (acuerdo total en todos los participantes).
- Dificultad para entender la estructura del panel web al buscar registros: 3.2.
- Dificultad para saber, a partir de la información mostrada, si el dispositivo estaba online u offline: 4.5 (valor alto, indica que a la mayoría sí le costó identificar el estado de conexión desde la interfaz).
- Claridad de la información mostrada en la tabla de registros: 2.8 (valor bajo, sugiere que la tabla no fue percibida como suficientemente clara).
- Dificultad para leer la información de la tabla de registros (tamaño, contraste, orden visual): 4.0 (valor alto, refuerza la observación anterior sobre legibilidad).
- Capacidad de usar la interfaz completa (embebida + panel web) sin instrucción adicional: 4.0.
- Necesidad percibida de ayuda de alguien con más conocimientos técnicos para usar la interfaz regularmente: 3.5.

En conjunto, estos resultados son consistentes con el puntaje SUS alto del grupo (89.6) en cuanto a la configuración general y la curva de aprendizaje, pero señalan la tabla de registros del panel web (claridad y legibilidad) y la indicación del estado de conexión online/offline como los puntos concretos de fricción a mejorar en la interfaz.

Usuarios técnicos (n=3)

Los 3 participantes respondieron afirmativamente (“Sí”) a las tres tareas planteadas: consumir los endpoints del prototipo sin complicaciones, modificar el endpoint configurado en el dispositivo para redirigir los datos a su propio servidor, e integrar o reutilizar las imágenes enviadas por la ESP32-CAM. Las preguntas de percepción (escala 1 a 5) obtuvieron los siguientes promedios:

- La documentación técnica contenía la información necesaria para integrar la API sin asistencia externa: 4.7.
- Recurrir a prueba y error, más que a la documentación, para entender los endpoints: 3.7 (valor moderado-alto, sugiere que la documentación no siempre fue suficiente).
- La estructura JSON entregada es clara y fácil de mapear a otros sistemas: 4.3.
- Dificultad para identificar los campos esenciales del registro (timestamp, usuario, métodos biométricos, estado de sincronización): 3.7 (valor moderado-alto, indica cierta fricción al identificar campos clave).
- Los mensajes de error fueron comprensibles y útiles para depurar problemas de integración: 2.7 (valor bajo, señala que los mensajes de error son un punto débil concreto a mejorar).
- Se encontraron inconsistencias entre los distintos endpoints (nombres, métodos HTTP o estructura de respuesta): 3.3 (valor moderado, indica inconsistencias percibidas por al menos parte del grupo).
- La API podría integrarse a un sistema ERP real sin modificaciones mayores: 4.0.
- Se necesitarían cambios significativos en el propio sistema o flujo de trabajo para integrar esta API: 1.3 (valor bajo, favorable, indica baja necesidad de adaptación).

Estos resultados son consistentes con el puntaje SUS del grupo técnico (82.5): si bien la API es percibida como integrable sin grandes cambios y con una estructura de datos clara, los mensajes de error del backend y ciertas inconsistencias entre endpoints aparecen como los puntos concretos de mejora señalados por este grupo, en contraste con el grupo no técnico, donde la fricción se concentró en la legibilidad de la tabla de registros del panel web.